

## Section 1-4

---

# Multi-Modal AI Content Forensics Platform

## Software Engineering Blueprint — Sections 1–4

**Document Version:** 3.0 (Comprehensive FYP Architecture & Feasibility Plan)

**Type:** Internal Architecture & Engineering Reference

**Scope:** Sections 1–4 of 14

**Target Audience:** Development Team, Faculty Supervisor, External Examiners

**Project Duration:** 6–8 months | **Team Size:** 1–3 students

## SECTION 1 — PROJECT FEASIBILITY

### 1.1 Complexity Analysis

**Overall System Complexity Rating: HIGH (8.5 / 10)**

This is not a single-model research project. It is a distributed software platform that integrates multiple independent ML inference pipelines under a shared API gateway, queue infrastructure, explainability layer, report engine, and web dashboard.

To ensure successful delivery within an academic timeline, the project adopts a strictly phased approach, moving the riskiest components to later phases.

#### Per-Modality Complexity Breakdown

| Modality | ML Complexity | Preprocessing Complexity | Engineering Integration | Explainability      | Phase Designation | Total Rating |
|----------|---------------|--------------------------|-------------------------|---------------------|-------------------|--------------|
| Text     | Medium        | Low                      | Low                     | Medium (SHAP)       | Phase 1 (Core)    | 5/10         |
| Image    | Medium        | Medium                   | Medium                  | Medium (GradCAM)    | Phase 2 (Core)    | 6/10         |
| Audio    | High          | High                     | High                    | High (Spectrograms) | Phase 3 (Core)    | 8/10         |
| Video    | Very High     | Very High                | Very High               | Very High           | Phase 4 (Stretch) | 9.5/10       |
| System   | —             | —                        | High                    | —                   | Phase 1 (Core)    | 8/10         |

#### Strategic Pivot for Video Constraints:

Video deepfake detection is disproportionately hard. Frame extraction, face-crop detection (MTCNN), temporal aggregation, and processing 30fps video are computationally prohibitive without smart sampling. State-of-the-art models often fail to generalize. **Video is explicitly demoted to a Phase 4 stretch goal** to prevent it from blocking core platform delivery.

### 1.2 Team Size Requirements

#### One Student (Minimum Viable Scenario)

- **Achievable:** Text + Image detection only. Unified API, simple frontend.
- **Compromise:** No video, no audio, no advanced fusion layer, minimal explainability.
- **Risk:** High — any blocking issue delays the entire project.
- **Workload:** ~400–500 hours of focused engineering.

Two Students (Recommended Configuration)

- **Student A (Backend / ML Integration Engineer):** FastAPI services, model integration (PyTorch/HuggingFace), PostgreSQL, RabbitMQ, Docker setup, Text & Audio pipelines.
- **Student B (Frontend / DevOps / ML Engineer):** Next.js dashboard, CI/CD, report generation, explainability visualizations, Image pipeline, Platform Fusion Layer.
- **Achievable:** Text, Image, and Audio modalities; REST API, dashboard, explainability for text/image, PDF reports, and cross-modality weighted fusion.
- **Workload:** ~350–400 hours per student over 6–8 months.

Three Students (Optimal Configuration)

- **Student A:** Backend infrastructure — FastAPI, PostgreSQL, RabbitMQ, Docker, CI/CD.
- **Student B:** ML pipeline engineering — All 4 detection services (including Video), fusion layer, explainability algorithms.
- **Student C:** Frontend + DevOps — Next.js, PDF report engine, monitoring, dataset curation, and academic evaluation matrix.
- **Achievable:** Near-production system with proper microservice separation and all 4 modalities.

1.3 Expected Timeline (6-Month Phased Delivery, 2 Students)

| Month   | Phase                                  | Focus Area                                                                                                                         |
|---------|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Month 1 | Phase 1 (Infrastructure & Scaffolding) | Docker Compose stack, PostgreSQL schema, FastAPI skeleton, Next.js skeleton, GitHub Actions CI/CD setup                            |
| Month 2 | Phase 1 (Text Modality & API)          | Text detection pipeline end-to-end (DeBERTa fine-tuned), SHAP token visualization integration                                      |
| Month 3 | Phase 2 (Image Modality & Storage)     | Image detection pipeline end-to-end (ResNet-50 / Corvi Ensemble), MinIO/S3 file upload pipeline integration, GradCAM visualization |
| Month 4 | Phase 3 (Audio Modality & Queues)      | Audio detection pipeline (AASIST pre-trained), RabbitMQ async job system for long-running inference handling                       |
| Month 5 | Phase 4 (System Fusion & Reporting)    | Weighted fusion algorithm, PDF report engine, end-to-end test suite execution                                                      |
| Month 6 | Phase 4 (Evaluation & Stretch Goals)   | Cross-dataset evaluation matrix, Video detection integration (if time permits), Final documentation                                |

1.4 GPU Requirements

Minimum Viable (Inference Only)

| Modality          | Minimum GPU VRAM | Recommended GPU | CPU Feasible?      |
|-------------------|------------------|-----------------|--------------------|
| Text (DeBERTa-v3) | 4 GB             | RTX 3060+       | Yes (slow, ~8–15s) |

| Modality                  | Minimum GPU VRAM | Recommended GPU | CPU Feasible?       |
|---------------------------|------------------|-----------------|---------------------|
| Image (ResNet-50 / Corvi) | 4 GB             | RTX 3060+       | Yes (~2–5s)         |
| Audio (AASIST)            | 2 GB             | RTX 2060+       | Yes (~5–10s)        |
| Video (XceptionNet)       | 6 GB             | RTX 3060+       | Very slow (minutes) |

### Fine-Tuning Requirements (One-Time Cloud Cost)

**Strategy:** Fine-tune once on cloud, checkpoint the model, and use it for local inference thereafter.

| Task                          | GPU       | Estimated Time | Cloud Cost (A100 40GB) |
|-------------------------------|-----------|----------------|------------------------|
| DeBERTa text fine-tune (RAID) | A100 40GB | 2–4 hours      | ~\$5–10                |
| Total fine-tuning budget      | —         | < 5 hours      | < \$15                 |

### 1.5 Storage Requirements

| Category             | Description                                     | Estimated Size |
|----------------------|-------------------------------------------------|----------------|
| Development Datasets | RAID text, CIFAKE, ASVspoof LA, GenImage subset | ~60–80 GB      |
| Model Checkpoints    | Pre-trained and fine-tuned models               | ~10–20 GB      |
| Application Data     | Runtime uploads, Postgres data, MinIO local     | ~10–50 GB      |
| Docker & System      | Base images, build cache, logs                  | ~30 GB         |
| Total Recommended    | NVMe SSD strictly required                      | 500 GB – 1 TB  |

Do NOT try to work from an external HDD — model loading times and data iteration will stall the project.

### 1.6 Cost Estimates (6-Month Budget)

| Expense                 | Option A (Budget)               | Option B (Comfortable)        |
|-------------------------|---------------------------------|-------------------------------|
| Cloud GPU (Fine-tuning) | \$10 (Colab Pro or Lambda Labs) | \$50 (Vast.ai / RunPod burst) |
| Storage (Local SSD)     | \$0 (Existing hardware)         | \$80–120 (1TB NVMe)           |
| Infrastructure / DB     | \$0 (Local Docker)              | \$0 (Local Docker)            |
| Total 6-month estimate  | \$10                            | \$130–\$170                   |

## 1.7 Scope Definition

### Final Year Project Scope (Strictly Scoped for Success)

**Included:**

- Text, Image, and Audio detection services
- Explainability for Text (SHAP) and Image (GradCAM) only
- Unified REST API, PostgreSQL database, RabbitMQ async queues, Docker Compose deployment, Next.js dashboard, PDF Reports
- Multi-modality weighted fusion layer to generate a master confidence score

### What Should NOT Be Implemented (Scope Creep Traps)

| Item                                     | Reason to Exclude                                                                       |
|------------------------------------------|-----------------------------------------------------------------------------------------|
| Custom transformer architecture          | 6–18 months of research effort. Use HuggingFace.                                        |
| Real-time video streaming (WebRTC)       | Requires video encoding pipelines. Accept pre-uploaded static files only.               |
| MLflow / Model Registries / Hot-swapping | Overkill for an academic FYP deployment. Load models at startup.                        |
| Video explainability (deep temporal)     | Extremely complex to visualize meaningfully on the web. Mark as explicit "future work." |
| Custom neural fusion model               | Weighted LR calibration is sufficient and interpretable for FYP evaluation.             |

### Phase 4 Stretch Goals (Attempt Only After Core Platform is Complete)

| Stretch Goal                   | Prerequisite                                                 | Estimated Effort      |
|--------------------------------|--------------------------------------------------------------|-----------------------|
| Browser Extension              | Core REST API complete and deployed                          | 3–4 weeks (Student B) |
| Source Model Attribution       | Text detection working; RAID attribution labels available    | 2–3 weeks (Student A) |
| Adversarial Robustness Testing | All detection services running; paraphrasing pipeline needed | 2 weeks (Student A)   |
| Video Deepfake Detection       | Phases 1–5 complete; GPU available with ≥ 6 GB VRAM          | 4–6 weeks (Both)      |

**Redis is NOT in scope creep — it is a mandatory core dependency.** Redis is used for: (1) JWT refresh token storage and blacklisting in auth-service, (2) rate-limit sliding window counters in the API Gateway via `slowapi`, and (3) login throttle counters for brute-force protection. Do NOT remove Redis from the stack.

## 1.8 Engineering Risk Register

| Risk                                     | Probability | Impact | Mitigation Strategy                             |
|------------------------------------------|-------------|--------|-------------------------------------------------|
| Video detector pipeline too slow/complex | High        | High   | Demote video to Phase 4 stretch goal. Use frame |

| Risk                                       | Probability | Impact | Mitigation Strategy                                                                        |
|--------------------------------------------|-------------|--------|--------------------------------------------------------------------------------------------|
|                                            |             |        | sampling (2fps) if implemented.                                                            |
| Dataset download restrictions (FF++)       | Medium      | High   | Submit academic access requests on Day 1. Have CIFAKE ready as a fallback.                 |
| Local GPU out-of-memory (OOM) errors       | High        | High   | Enforce strict file size limits at the API gateway; limit batch sizes.                     |
| API blocking during long inference         | Medium      | High   | Mandatory implementation of RabbitMQ for image/audio/video processing.                     |
| Integration Failure (Services cannot talk) | Low         | High   | Establish rigid OpenAPI 3.0 specs in Week 2. Stub all API responses before ML integration. |

## SECTION 2 — SYSTEM REQUIREMENTS

### 2.1 Functional Requirements

#### Core Detection & Fusion

| ID    | Requirement                                                                                                                                                                                                                                                                 |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-01 | System SHALL accept plain text input (UTF-8, max 10,000 characters) and return an AI-detection probability score between 0.0 and 1.0.                                                                                                                                       |
| FR-02 | System SHALL accept image files (JPEG, PNG, WebP) up to 10 MB and return an AI-generation probability score.                                                                                                                                                                |
| FR-03 | System SHALL accept audio files (WAV, MP3, FLAC) up to 30 MB / 120 seconds and return a synthetic speech probability score via async queueing.                                                                                                                              |
| FR-04 | System SHALL provide explainability output: SHAP token attribution for text, and GradCAM heatmap generation for images.                                                                                                                                                     |
| FR-05 | <b>[FUSION LAYER]</b> System SHALL implement a multimodal fusion layer to calculate an overall confidence score using a weighted average: $P_{final} = \sum(w_i \cdot p_i)$ where $w_i$ is the configured weight for modality $i$ , derived via validation set calibration. |

#### API & Dashboard Requirements

| ID    | Requirement |
|-------|-------------|
| FR-06 |             |

| ID    | Requirement                                                                                                                                                          |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       | System SHALL expose a RESTful HTTP API with OpenAPI 3.0 specification auto-generated at <code>/api/docs</code> .                                                     |
| FR-07 | System SHALL support asynchronous job submission with a polling endpoint ( <code>/api/v1/jobs/{job_id}</code> ).                                                     |
| FR-08 | System SHALL provide a web dashboard (Next.js) for file upload, real-time job status, and inline result visualization.                                               |
| FR-09 | System SHALL generate downloadable PDF reports containing: input summary, probability score, confidence, verdict, explainability visualizations, and model metadata. |

## 2.2 Non-Functional Requirements

| ID     | Requirement                                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Each detection modality SHALL be implemented as an independently scalable service.                                    |
| NFR-02 | All services SHALL be containerized with Docker and orchestrated via a single <code>docker-compose up</code> command. |
| NFR-03 | Service configuration SHALL be fully externalized to <code>.env</code> files.                                         |
| NFR-04 | Database schema changes SHALL be managed via versioned Alembic migration files — no manual DDL.                       |
| NFR-05 | Python code SHALL comply with PEP 8 enforced via <code>ruff</code> in CI.                                             |

## 2.3 Performance Requirements

| ID    | Requirement                                                        |
|-------|--------------------------------------------------------------------|
| PR-01 | Text detection P95 latency (1,000 chars, GPU) < 2 seconds.         |
| PR-02 | Image detection P95 latency (5 MB JPEG, GPU) < 5 seconds.          |
| PR-03 | Audio detection job completion (60-second clip, GPU) < 30 seconds. |
| PR-04 | API gateway overhead (routing only) < 50 ms.                       |

## 2.4 Security Requirements

| ID    | Requirement                                                       |
|-------|-------------------------------------------------------------------|
| SR-01 | API SHALL require JWT authentication for all detection endpoints. |

| ID    | Requirement                                                                                              |
|-------|----------------------------------------------------------------------------------------------------------|
| SR-02 | Uploaded files SHALL be validated for MIME type via <code>python-magic</code> , not just file extension. |
| SR-03 | Uploaded files SHALL be stored in isolated object storage (MinIO), never in the web server root.         |
| SR-04 | API SHALL rate-limit requests (e.g., 60 requests/minute) via <code>slowapi</code> .                      |
| SR-05 | Database queries SHALL use SQLAlchemy parameterized statements to prevent SQL injection.                 |

## SECTION 3 — BUILD VS BUY ANALYSIS

**Decision Framework:** For each modality, we evaluate options across a 4-point matrix:

- **A** = Use existing pre-trained model as-is (inference only)
- **B** = Fine-tune an existing pre-trained model on task-specific data
- **C** = Build custom architecture from scratch (*Never recommended for FYP*)
- **D** = Ensemble multiple existing models

**General Principle:** The engineering contribution lies in the platform, data handling, fusion layer, and explainability — not in the novel detector algorithm.

### 3.1 Text Detection

#### Available Open-Source Models

| # | Model                                 | Parameters  | Notes                                              |
|---|---------------------------------------|-------------|----------------------------------------------------|
| 1 | RoBERTa GPT-2 Detector (OpenAI, 2019) | 355M        | Critically outdated; fails on GPT-4/Claude         |
| 2 | Binoculars (Hans et al., 2024)        | 2× 7B = 14B | Near-SOTA accuracy but ~28 GB VRAM — impractical   |
| 3 | Fast-DetectGPT (Bao et al., 2023)     | 2B–7B       | Zero-shot, too heavy for standard local hardware   |
| 4 | DeBERTa-v3 fine-tuned on RAID         | 86M         | Best balance of accuracy, speed, and deployability |

#### TEXT Final Decision

**DECISION: B — Fine-tune DeBERTa-v3-base on RAID dataset.**

**Reasoning:** Best balance of modern accuracy, speed, and deployability. 86M params fits easily on CPU/GPU. Integrates perfectly with SHAP for frontend visualization. Trains in 2 hours on cloud GPU. Sub-100ms inference.

### 3.2 Image Detection

Modern AI image detection must handle GANs (spatial artifacts) and Diffusion models (spectral/reconstruction artifacts). Single models fail to cover both.

#### Available Open-Source Models

| # | Model                           | Architecture                 | Specialty     | Notes                                                     |
|---|---------------------------------|------------------------------|---------------|-----------------------------------------------------------|
| 1 | Wang et al. CNNDetection (2020) | ResNet-50                    | GAN detection | Excellent on GANs, poor on Diffusion. Sub-100ms inference |
| 2 | UnivFD (Liu et al., 2022)       | CLIP ViT-L/14 + linear probe | Both          | Good generalizer, slightly heavier                        |
| 3 | DIRE (Wang et al., 2023)        | ResNet-50 + diffusion model  | Diffusion     | SOTA for diffusion but ~12 GB VRAM, 10s/image — too heavy |
| 4 | Corvi et al. (2023)             | EfficientNet-B0 (frequency)  | Diffusion     | Lightweight, sub-200ms inference                          |

#### IMAGE Final Decision

**DECISION: A + D (Inference Only Ensemble).**  
**Reasoning:** Primary ResNet-50 (Wang et al.) for GANs, ensembled with Corvi et al. for Diffusion. Weighted average output. Both are lightweight, fit in memory, and support `pytorch-grad-cam` for explainability heatmaps. Avoid DIRE due to massive compute requirements.

### 3.3 Audio Detection

The primary benchmark is ASVspoof 2019 LA track (detecting TTS/Voice Conversion vs. real speech).

#### Available Open-Source Models

| # | Model                        | Parameters | EER (ASVspoof 2019 LA) | Notes                                                           |
|---|------------------------------|------------|------------------------|-----------------------------------------------------------------|
| 1 | AASIST (Jung et al., 2022)   | 297K       | 0.83%                  | Near-SOTA, sub-50ms inference, pre-trained weights available    |
| 2 | RawGAT-ST (Tak et al., 2021) | ~500K      | 1.06%                  | Superseded by AASIST                                            |
| 3 | Wav2Vec2 Fine-Tuned          | 95M        | ~1.8%                  | Stronger generalization, but 8–12h fine-tuning on A100 required |

#### AUDIO Final Decision

**DECISION: A (AASIST inference-only).**  
**Reasoning:** AASIST provides near-SOTA accuracy with practically zero engineering footprint or fine-tuning cost. It requires <1 GB VRAM, ensuring it won't crash alongside other models. Skip Wav2Vec2 fine-tuning to save time.

### 3.4 Video Detection (Phase 4 Stretch Goal)

#### Available Open-Source Models

| # | Model                                  | Architecture          | Generalization       | Notes                                       |
|---|----------------------------------------|-----------------------|----------------------|---------------------------------------------|
| 1 | XceptionNet (FaceForensics++ Baseline) | Modified Inception    | Medium               | Widely reproduced, fast frame inference     |
| 2 | SBI (Self-Blended Images, 2022)        | EfficientNet-B4       | Strong cross-dataset | Best real-world performance but ~6 GB VRAM  |
| 3 | FTCN (Zheng et al., 2021)              | Fully temporal 1D CNN | Strong temporal      | Too complex for variable-length web uploads |

#### VIDEO Final Decision (If Attempted)

**DECISION: A (Inference Only Frame-Level Strategy).**  
**Reasoning:** Sample frames at 2fps → MTCNN face crop → XceptionNet inference → average probabilities. Do NOT attempt 3D/temporal models or train from scratch.

### 3.5 System Fusion Layer Strategy

To combine independent modal outputs into a single platform verdict, the system requires a Fusion Layer.

| Property                    | Detail                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Method                      | Late Fusion (Score-level)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Algorithm                   | Weighted Linear Combination: <code>P_final = Σ(w_i · p_i)</code> where weights sum to 1.0                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Default Weights (Bootstrap) | <code>w_text = 0.50, w_image = 0.30, w_audio = 0.20</code> — empirically motivated starting point based on relative model accuracy. Used when calibration data is not yet available.                                                                                                                                                                                                                                                                                                                                       |
| Calibration Strategy        | Once all detection services are running (Phase 5, Week 17), run <code>scripts/calibrate_fusion_weights.py</code> . This fits a constrained Logistic Regression model (no intercept, non-negative coefficients forced to sum to 1.0) on a held-out synthetic multimodal validation set (min. 100 labelled samples). Saves calibrated weights to <code>config/fusion_weights.json</code> . The <code>FusionEngine</code> loads weights from this file at startup and falls back to hardcoded defaults if the file is absent. |
| Weight Precedence           | <code>config/fusion_weights.json</code> (calibrated) > <code>FusionEngine</code> constructor defaults (hardcoded)                                                                                                                                                                                                                                                                                                                                                                                                          |
| Image Sub-Ensemble          | Within the image modality, the ResNet-50 / Corvi sub-ensemble uses fixed weights: <code>w_resnet = 0.55, w_corvi = 0.45</code> . These reflect ResNet-50's stronger GAN generalization and Corvi's diffusion specialization. These weights are NOT recalibrated by the fusion LR script — they are image-modality-internal constants.                                                                                                                                                                                      |
| Purpose                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

| Property | Detail                                                                                                                                             |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|          | Provide an interpretable, auditable, and reproducible fusion decision that can be reported in the dissertation with full methodology transparency. |

## SECTION 4 — DATASET & EVALUATION ANALYSIS

### 4.1 Text Detection Datasets

| Dataset     | Total Samples | Generators                       | Domains | License    | Storage | Purpose                               |
|-------------|---------------|----------------------------------|---------|------------|---------|---------------------------------------|
| RAID (2024) | 500,000+      | 11 LLMs (GPT-4, Claude, Llama-2) | 11      | MIT        | ~3 GB   | Primary fine-tuning (Use 100K subset) |
| M4 (2023)   | 122,000       | 6 LLMs                           | 4       | Apache 2.0 | ~600 MB | Cross-domain evaluation               |
| HC3 (2023)  | 24,000        | GPT-3.5 Only                     | 5       | CC BY-SA   | ~50 MB  | ChatGPT-specific baseline             |
| TuringBench | 200,000       | 19 Legacy LLMs                   | News    | MIT        | ~2 GB   | Avoid (Outdated distributions)        |

**Dataset Links:**

- RAID: <https://huggingface.co/datasets/liamdugan/raid>
- M4: <https://github.com/mbzuai-nlp/M4>
- HC3: <https://huggingface.co/datasets/Hello-SimpleAI/HC3>

### 4.2 Image Detection Datasets

| Dataset         | Total Images | Generation Methods             | License   | Storage     | Purpose                                          |
|-----------------|--------------|--------------------------------|-----------|-------------|--------------------------------------------------|
| GenImage        | 2.66M        | 8 (SD, MJ, DALL-E)             | CC BY 4.0 | 200 GB      | Generalization eval (Download 30 GB subset only) |
| CIFAKE          | 120,000      | Stable Diffusion (CIFAR-style) | CC0       | 500 MB      | Rapid prototyping / Unit tests                   |
| FaceForensics++ | 1M+ frames   | 5 face manipulation methods    | Academic  | 17 GB (c40) | Face GAN evaluation                              |
| ArtiFact        | 2.5M         | 25+ generators                 | MIT       | 500 GB      | Holdout generalization test (use tiny subset)    |

**Dataset Links:**

- GenImage: <https://github.com/GenImage-Dataset/GenImage>
- CIFAKE: <https://huggingface.co/datasets/chesterkuo/cifake>
- FaceForensics++: <https://github.com/ondyari/FaceForensics> (academic request required)

### 4.3 Audio Detection Datasets

| Dataset          | Total Utterances | Attack Types  | License  | Storage | Purpose |
|------------------|------------------|---------------|----------|---------|---------|
| ASVspoof 2019 LA | 121,461          | 19 TTS + 6 VC | CC BY-SA | 14 GB   |         |

| Dataset          | Total Utterances | Attack Types            | License   | Storage | Purpose                           |
|------------------|------------------|-------------------------|-----------|---------|-----------------------------------|
|                  |                  |                         |           |         | Primary benchmark & Evaluation    |
| ASVspoof 2021 DF | 611,829          | In-the-wild TTS         | CC BY-SA  | 130 GB  | Avoid (Too large for FYP)         |
| WaveFake         | 117,985          | 6 neural vocoders       | MIT       | 52 GB   | Vocoder-specific evaluation       |
| In-the-Wild      | 31,000           | Real-world internet TTS | CC BY 4.0 | 4 GB    | Cross-dataset generalization eval |

**Dataset Links:**

- ASVspoof 2019: <https://datashare.ed.ac.uk/handle/10283/3336>
- WaveFake: <https://github.com/RUB-SysSec/WaveFake>
- In-the-Wild: [https://deepfake-total.com/in\\_the\\_wild](https://deepfake-total.com/in_the_wild)

4.4 Academic Evaluation Plan & Metrics

To demonstrate rigorous engineering to external examiners, the platform will be evaluated not just on validation sets, but via strict cross-dataset testing to prove real-world robustness.

Core Modality Evaluation Matrix

| Modality | Primary Training/Tuning   | Holdout Testing (Generalization) | Target Metric          |
|----------|---------------------------|----------------------------------|------------------------|
| Text     | RAID (Sub-domain A)       | M4 English Test Set              | F1-Score, AUROC        |
| Image    | Pre-trained weights       | GenImage (Subset) + CIFAKE       | AUC                    |
| Audio    | Pre-trained (ASVspoof 19) | In-the-Wild Dataset              | EER (Equal Error Rate) |

Fusion Evaluation Plan

The overall platform architecture will be evaluated by constructing a "Synthetic Multimodal Test Set" (e.g., 100 fake news articles containing AI text, AI images, and AI audio clips). The overall accuracy of the Fusion Layer (  $P_{final}$  ) will be plotted on an ROC curve against the individual modalities to prove that the whole system performs better than its isolated parts.

4.5 Storage Budget & Dataset Action Plan

Aggregated Storage Plan (Local NVMe)

| Category                      | Estimated Size |
|-------------------------------|----------------|
| Text Datasets                 | ~5 GB          |
| Image Datasets (Subsets only) | ~35 GB         |
| Audio Datasets                | ~20 GB         |
| Models + Checkpoints          | ~15 GB         |

| Category                           | Estimated Size |
|------------------------------------|----------------|
| Development Workspace (Docker, DB) | ~30 GB         |
| <b>Total Expected Utilization</b>  | <b>~105 GB</b> |

## Dataset Access Timeline

| Week          | Action                                                                                                |
|---------------|-------------------------------------------------------------------------------------------------------|
| <b>Week 1</b> | Submit FaceForensics++ academic access requests (approval takes 3–10 days). Download CIFAKE and RAID. |
| <b>Week 2</b> | Download ASVspoof 2019 LA track via Edinburgh DataShare.                                              |
| <b>Week 4</b> | Download specific 3-generator subset of GenImage (~30 GB).                                            |

*End of Sections 1–4 | Continued in: [blueprint\\_sections\\_5\\_to\\_8.md](#) (Research Papers, Architecture, Microservices, Database)*

## Section 5-8

---

# Multi-Modal AI Content Forensics Platform

## Software Engineering Blueprint — Sections 5–8

**Document Version:** 4.0 (Full LLD Expansion)  
**Type:** Internal Architecture & Engineering Reference  
**Scope:** Sections 5–8 of 14  
**Target Audience:** Development Team, Faculty Supervisor, External Examiners  
**Continuation of:** Sections 1–4 (blueprint\_sections\_1\_to\_4.md)

## SECTION 5 — RESEARCH PAPER SURVEY

**Reading Strategy for Implementation:** Focus on the **GitHub** column first. Papers with working repos reduce integration time from weeks to days. Read the full paper only when you need to understand a hyperparameter or architectural decision.

### 5.1 Text AI Detection Papers

#### Paper T-01: Detecting GPT-2 Generated Text (RoBERTa Detector)

| Field                | Detail                                                                                                      |
|----------------------|-------------------------------------------------------------------------------------------------------------|
| Title                | Release Strategies and the Social Impacts of Language Models                                                |
| Year                 | 2019                                                                                                        |
| Conference           | OpenAI Technical Report (arXiv:1908.09203)                                                                  |
| Dataset Used         | GPT-2 WebText outputs vs. real WebText articles                                                             |
| Hardware Used        | TPU v3-8 (OpenAI internal)                                                                                  |
| Algorithm            | RoBERTa-Large fine-tuned as binary classifier on GPT-2 vs human text                                        |
| Key Metrics          | Accuracy: 88% (in-distribution); drops to ~54% on GPT-3+ outputs                                            |
| Critical Limitations | Fundamentally broken on modern LLMs. Acts as a false baseline — do NOT use as your primary detector         |
| GitHub               | <a href="https://github.com/openai/gpt-2-output-dataset">https://github.com/openai/gpt-2-output-dataset</a> |
| FYP Relevance        | Literature review only. Establishes why we need DeBERTa re-trained on RAID.                                 |

Paper T-02: DetectGPT — Zero-Shot Machine-Generated Text Detection

| Field                | Detail                                                                                                |
|----------------------|-------------------------------------------------------------------------------------------------------|
| Title                | DetectGPT: Zero-Shot Machine-Generated Text Detection Using Probability Curvature                     |
| Year                 | 2023                                                                                                  |
| Conference           | ICML 2023 (Oral)                                                                                      |
| Dataset Used         | XSum, SQuAD, Reddit WritingPrompts                                                                    |
| Hardware Used        | 4× A100 80GB                                                                                          |
| Algorithm            | Generates ~100 perturbations via T5 masking, re-scores with source LLM, computes curvature            |
| Key Metrics          | AUROC: 0.95 (in-distribution); 0.72–0.85 (cross-model)                                                |
| Critical Limitations | Requires source LLM at inference, ~20–30s per query. NOT deployable in web service.                   |
| GitHub               | <a href="https://github.com/eric-mitchell/detect-gpt">https://github.com/eric-mitchell/detect-gpt</a> |
| FYP Relevance        | Cite as theoretical baseline. Do NOT implement.                                                       |

Paper T-03: Fast-DetectGPT

| Field                | Detail                                                                                                          |
|----------------------|-----------------------------------------------------------------------------------------------------------------|
| Title                | Fast-DetectGPT: Efficient Zero-Shot Detection via Conditional Probability Curvature                             |
| Year                 | 2024                                                                                                            |
| Conference           | ICLR 2024                                                                                                       |
| Algorithm            | Single conditional sampling pass instead of 100 perturbation calls. 340× faster than DetectGPT                  |
| Key Metrics          | AUROC: 0.98 (GPT-3.5), 0.91 (GPT-4)                                                                             |
| Critical Limitations | Still requires 7B+ scoring model at runtime. Not suitable for low-VRAM FYP deployment.                          |
| GitHub               | <a href="https://github.com/baoguangsheng/fast-detect-gpt">https://github.com/baoguangsheng/fast-detect-gpt</a> |
| FYP Relevance        | Good citation. Feasible only if spare 7B scoring model available.                                               |

Paper T-04: RADAR — Robust AI-Text Detection via Adversarial Learning

| Field | Detail                                                   |
|-------|----------------------------------------------------------|
| Title | RADAR: Robust AI-Text Detection via Adversarial Learning |
| Year  | 2023                                                     |

| Field         | Detail                                                                              |
|---------------|-------------------------------------------------------------------------------------|
| Conference    | NeurIPS 2023                                                                        |
| Dataset Used  | HC3 + adversarially paraphrased variants                                            |
| Algorithm     | Joint adversarial training: Vicuna-7B paraphraser vs DeBERTa detector               |
| Key Metrics   | F1: 0.89 on HC3; 0.82 on adversarial inputs vs 0.61 for vanilla classifiers         |
| GitHub        | <a href="https://github.com/IBM/RADAR">https://github.com/IBM/RADAR</a>             |
| FYP Relevance | Cite for adversarial robustness discussion. Do NOT replicate training from scratch. |

Paper T-05: RAID Benchmark

PRIMARY

| Field         | Detail                                                                                       |
|---------------|----------------------------------------------------------------------------------------------|
| Title         | RAID: A Shared Benchmark for Robust Evaluation of Machine-Generated Text Detectors           |
| Year          | 2024                                                                                         |
| Conference    | ACL 2024                                                                                     |
| Dataset       | 500,000+ samples, 11 LLMs × 11 domains × multiple attack types                               |
| Key Metrics   | Best detector: ~75% F1 at 5% FPR on adversarial attacks                                      |
| GitHub        | <a href="https://github.com/liamdugan/raid">https://github.com/liamdugan/raid</a>            |
| FYP Relevance | <b>PRIMARY REFERENCE.</b> Use RAID dataset for fine-tuning. Use RAID evaluation methodology. |

Paper T-06: Binoculars

| Field                | Detail                                                                                      |
|----------------------|---------------------------------------------------------------------------------------------|
| Title                | Spotting LLMs With Binoculars: Zero-Shot Detection of Machine-Generated Text                |
| Year                 | 2024                                                                                        |
| Conference           | ICML 2024                                                                                   |
| Algorithm            | Ratio of Performer (Falcon-7B) perplexity to Observer (Falcon-7B-Instruct) cross-perplexity |
| Key Metrics          | AUROC: 0.99 on GPT-3.5/4; 0.91 on RAID                                                      |
| Critical Limitations | Requires TWO 7B models simultaneously (~28 GB VRAM). Not feasible for FYP hardware.         |
| GitHub               | <a href="https://github.com/ahans30/Binoculars">https://github.com/ahans30/Binoculars</a>   |

| Field         | Detail                                                                |
|---------------|-----------------------------------------------------------------------|
| FYP Relevance | Cite as SOTA upper-bound benchmark. Do NOT deploy as primary service. |

## 5.2 Image AI Detection Papers

### Paper I-01: CNNDetection PRIMARY (GAN)

| Field         | Detail                                                                                                  |
|---------------|---------------------------------------------------------------------------------------------------------|
| Title         | CNN-Generated Images Are Surprisingly Easy to Spot... For Now                                           |
| Year          | 2020                                                                                                    |
| Conference    | CVPR 2020                                                                                               |
| Dataset Used  | 11 GANs (ProGAN, CycleGAN, StyleGAN, BigGAN)                                                            |
| Algorithm     | ResNet-50 fine-tuned on ProGAN with aggressive augmentation (JPEG, blur, resize)                        |
| Key Metrics   | AUC: 0.99 (ProGAN); 0.81–0.96 (unseen GANs); 0.51–0.60 (Diffusion — near-random)                        |
| GitHub        | <a href="https://github.com/peterwang512/CNNDetection">https://github.com/peterwang512/CNNDetection</a> |
| FYP Relevance | <b>PRIMARY IMAGE MODEL (GAN component).</b> Pre-trained weights available directly.                     |

### Paper I-02: UnivFD

| Field         | Detail                                                                                     |
|---------------|--------------------------------------------------------------------------------------------|
| Title         | Towards Universal Fake Image Detection by Training on a Front Door Criterion               |
| Year          | 2023                                                                                       |
| Conference    | CVPR 2023                                                                                  |
| Algorithm     | CLIP ViT-L/14 frozen feature extractor + linear probe. Causal front-door intervention.     |
| Key Metrics   | AUC: 0.95 average across 17 generators                                                     |
| GitHub        | <a href="https://github.com/Ekko-zn/AIGC-AD">https://github.com/Ekko-zn/AIGC-AD</a>        |
| FYP Relevance | Strong alternative/backup model. Adds ~50ms inference but better diffusion generalization. |

### Paper I-03: DIRE

| Field                | Detail                                                                                           |
|----------------------|--------------------------------------------------------------------------------------------------|
| Title                | DIRE for Diffusion-Generated Image Detection                                                     |
| Year                 | 2023                                                                                             |
| Conference           | ICCV 2023                                                                                        |
| Algorithm            | DDIM inversion reconstruction error. AI images have low error (lie on diffusion manifold).       |
| Key Metrics          | AUC: 0.99 in-distribution                                                                        |
| Critical Limitations | Requires full diffusion model at inference (~12 GB VRAM, 10s per image). Completely impractical. |
| GitHub               | <a href="https://github.com/ZhendongWang6/DIRE">https://github.com/ZhendongWang6/DIRE</a>        |
| FYP Relevance        | Cite as SOTA diffusion detection. Do NOT implement. Use Corvi instead.                           |

### Paper I-04: Corvi et al. PRIMARY (Diffusion)

| Field         | Detail                                                                                                      |
|---------------|-------------------------------------------------------------------------------------------------------------|
| Title         | On the Detection of Synthetic Images Generated by Diffusion Models                                          |
| Year          | 2023                                                                                                        |
| Conference    | ICASSP 2023                                                                                                 |
| Algorithm     | 2D FFT magnitude + high-pass residuals → EfficientNet-B0 binary classifier                                  |
| Key Metrics   | AUC: 0.97 (LDM); 0.93 (DALL-E 2); ~150ms inference on GPU                                                   |
| Hardware      | Single NVIDIA RTX 3090                                                                                      |
| GitHub        | <a href="https://github.com/grip-unina/DMImageDetection">https://github.com/grip-unina/DMImageDetection</a> |
| FYP Relevance | <b>PRIMARY IMAGE MODEL (Diffusion component).</b><br>Lightweight, trainable on single GPU.                  |

### Paper I-05: GradCAM Explainability

| Field      | Detail                                                                           |
|------------|----------------------------------------------------------------------------------|
| Title      | Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization |
| Year       | 2017/2019                                                                        |
| Conference | ICCV 2017                                                                        |
| Algorithm  |                                                                                  |

| Field         | Detail                                                                                                  |
|---------------|---------------------------------------------------------------------------------------------------------|
|               | Gradient of class score w.r.t. last conv layer activations → weighted feature map heatmap               |
| GitHub        | <a href="https://github.com/jacobgil/pytorch-grad-cam">https://github.com/jacobgil/pytorch-grad-cam</a> |
| FYP Relevance | <b>EXPLAINABILITY LAYER for Image Detection.</b> 5 lines of code to integrate.                          |

## 5.3 Audio AI Detection Papers

### Paper A-01: AASIST PRIMARY

| Field         | Detail                                                                                               |
|---------------|------------------------------------------------------------------------------------------------------|
| Title         | AASIST: Audio Anti-Spoofing Using Integrated Spectro-Temporal Graph Attention Networks               |
| Year          | 2022                                                                                                 |
| Conference    | ICASSP 2022                                                                                          |
| Dataset Used  | ASVspoof 2019 LA                                                                                     |
| Hardware Used | Single NVIDIA V100 16GB                                                                              |
| Algorithm     | Raw waveform → SincNet → Heterogeneous Graph Attention (spectral + temporal) → binary classification |
| Key Metrics   | EER: 0.83% on ASVspoof 2019 LA eval. 297K parameters. ~30ms inference per 4s clip.                   |
| GitHub        | <a href="https://github.com/clovaai/aasist">https://github.com/clovaai/aasist</a>                    |
| FYP Relevance | <b>PRIMARY AUDIO MODEL.</b> Load pre-trained weights. Under 1MB model. 30ms inference.               |

### Paper A-02: RawGAT-ST

| Field         | Detail                                                                                          |
|---------------|-------------------------------------------------------------------------------------------------|
| Title         | End-to-End Spectro-Temporal Graph Attention Networks for Speaker Verification Anti-Spoofing     |
| Year          | 2021                                                                                            |
| Conference    | Interspeech 2021                                                                                |
| Key Metrics   | EER: 1.06% (superseded by AASIST)                                                               |
| GitHub        | <a href="https://github.com/eurecom-asf/RawGAT-ST">https://github.com/eurecom-asf/RawGAT-ST</a> |
| FYP Relevance | Literature review only. Predecessor to AASIST.                                                  |

### Paper A-03: Wav2Vec2 Anti-Spoofing

| Field         | Detail                                                                                                        |
|---------------|---------------------------------------------------------------------------------------------------------------|
| Title         | Automatic Speaker Verification Spoofing and Deepfake Detection Using Wav2Vec 2.0                              |
| Year          | 2022                                                                                                          |
| Conference    | Odyssey 2022                                                                                                  |
| Key Metrics   | EER: 1.8% on ASVspoof 2021 DF. Stronger cross-dataset generalization than AASIST.                             |
| GitHub        | <a href="https://github.com/TakHemlata/SSL_Anti-spoofing">https://github.com/TakHemlata/SSL_Anti-spoofing</a> |
| FYP Relevance | Phase 4 optional upgrade for generalization. Cite when discussing AASIST limitations.                         |

### Paper A-04: LCNN Baseline

| Field         | Detail                                                                                                              |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| Title         | Light CNN for Anti-Spoofing (LFCC+LCNN baseline)                                                                    |
| Year          | 2020                                                                                                                |
| Conference    | ICASSP 2020                                                                                                         |
| Key Metrics   | EER: 5.06% — traditional approach baseline                                                                          |
| GitHub        | <a href="https://github.com/asvspoof-challenge/asvspoof2021">https://github.com/asvspoof-challenge/asvspoof2021</a> |
| FYP Relevance | Use as baseline comparison. Provides "traditional vs modern" narrative for methodology chapter.                     |

## 5.4 Video Deepfake Detection Papers

### Paper V-01: FaceForensics++ Phase 4

| Field        | Detail                                                                       |
|--------------|------------------------------------------------------------------------------|
| Title        | FaceForensics++: Learning to Detect Manipulated Facial Images                |
| Year         | 2019                                                                         |
| Conference   | ICCV 2019                                                                    |
| Dataset Used | 1,000 original videos × 5 manipulation methods × 3 compression levels        |
| Algorithm    | XceptionNet per-frame binary classifier on face crops (MTCNN pre-processing) |
| Key Metrics  |                                                                              |

| Field         | Detail                                                                                          |
|---------------|-------------------------------------------------------------------------------------------------|
|               | Accuracy: 99.4% (c0); 81.4% (c40 social-media quality); 74% cross-dataset                       |
| GitHub        | <a href="https://github.com/ondyari/FaceForensics">https://github.com/ondyari/FaceForensics</a> |
| FYP Relevance | <b>PRIMARY VIDEO MODEL if Phase 4 attempted.</b> Academic license required.                     |

## Paper V-02: SBI — Self-Blended Images

| Field         | Detail                                                                                                |
|---------------|-------------------------------------------------------------------------------------------------------|
| Title         | Detecting Deepfakes with Self-Blended Images                                                          |
| Year          | 2022                                                                                                  |
| Conference    | CVPR 2022 (Oral)                                                                                      |
| Algorithm     | EfficientNet-B4 trained on synthetic "self-blended" forgeries for cross-dataset generalization        |
| Key Metrics   | AUC: 0.93 (CelebDF-v2); 0.88 (DFDC) — best cross-dataset generalization of its era                    |
| GitHub        | <a href="https://github.com/mapoon/SelfBlendedImages">https://github.com/mapoon/SelfBlendedImages</a> |
| FYP Relevance | Preferred if VRAM allows (~6GB). Better real-world performance than XceptionNet.                      |

## Paper V-03: FTCN

| Field         | Detail                                                                                      |
|---------------|---------------------------------------------------------------------------------------------|
| Title         | Exploring Temporal Coherence for More General Video Face Forgery Detection                  |
| Year          | 2021                                                                                        |
| Conference    | ICCV 2021                                                                                   |
| Algorithm     | Fully temporal 1D convolutions. Collapses spatial dims to model temporal incoherence only.  |
| Key Metrics   | AUC: 0.98 (FF++ c23); 0.86 (CelebDF-v2)                                                     |
| GitHub        | <a href="https://github.com/yinglinzheng/FTCN">https://github.com/yinglinzheng/FTCN</a>     |
| FYP Relevance | DO NOT implement. Too complex for variable-length web uploads. Cite for temporal rationale. |

## 5.5 Multi-Modal Detection Papers

Paper M-01: FakeBench

| Field         | Detail                                                                                           |
|---------------|--------------------------------------------------------------------------------------------------|
| Title         | FakeBench: Probing Explainability of Foundation Models for Unified Fake Image and Text Detection |
| Year          | 2024                                                                                             |
| Conference    | arXiv 2024                                                                                       |
| Algorithm     | Prompts GPT-4V and LLaVA with multi-modal inputs; evaluates whether VLMs can explain fakes       |
| Key Metrics   | GPT-4V: 68% detection accuracy on multi-modal fake content                                       |
| FYP Relevance | Cite in related work. Justifies your weighted fusion as a practical alternative.                 |

Paper M-02: OmniDetector

| Field         | Detail                                                                                                                  |
|---------------|-------------------------------------------------------------------------------------------------------------------------|
| Title         | Towards Universal Detection of AI-Generated Content                                                                     |
| Year          | 2024                                                                                                                    |
| Conference    | arXiv 2024                                                                                                              |
| Algorithm     | Shared multi-modal transformer backbone with modality-specific input adapters. Joint contrastive + classification loss. |
| FYP Relevance | Cite as motivation for unified platform. Justifies why a multi-modal platform has research value.                       |

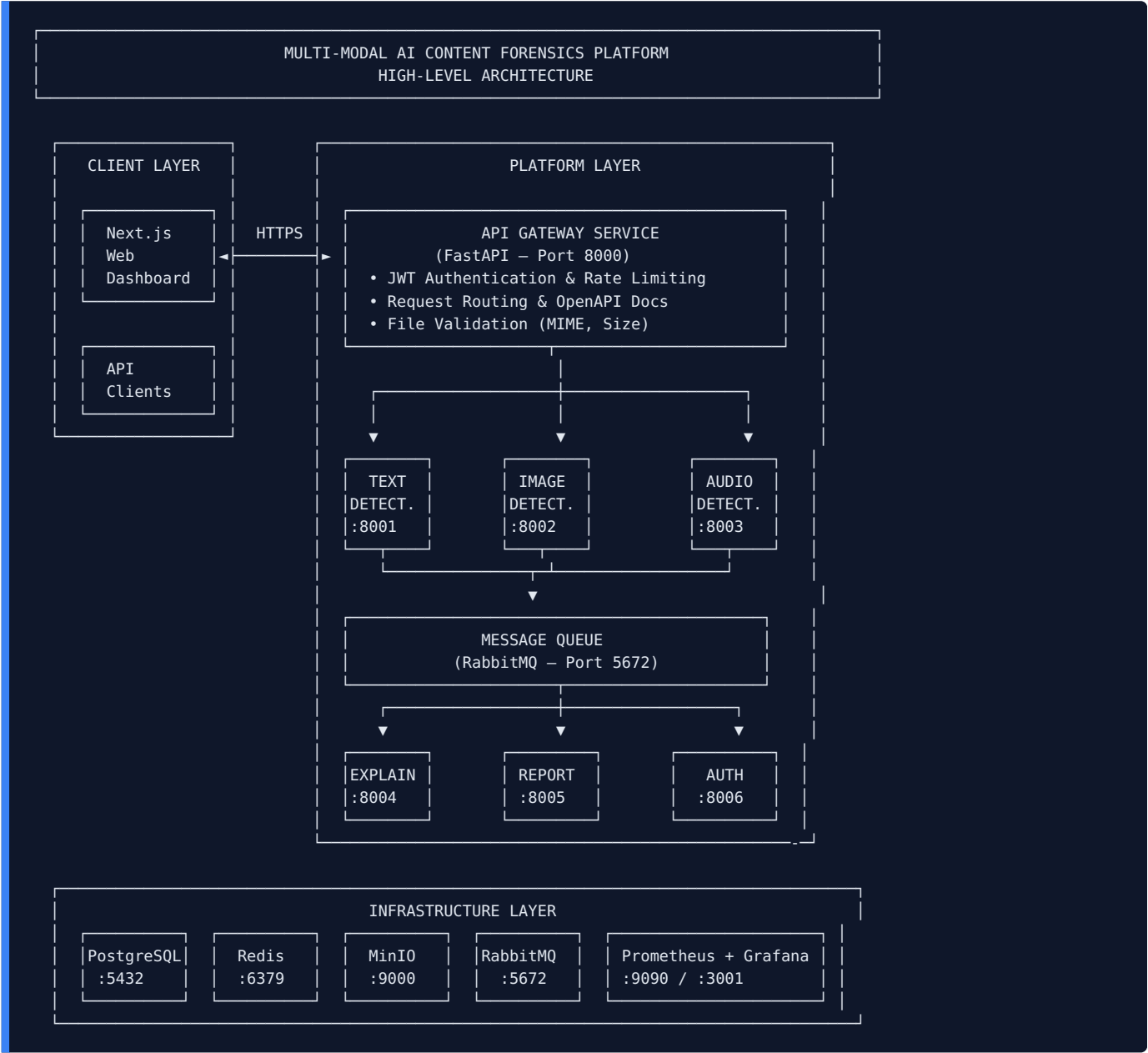
5.6 Research Paper Quick-Reference Matrix

| Paper ID | Title (Shortened) | Year | Conference  | Task           | Metric     | GitHub | FYP Use    |
|----------|-------------------|------|-------------|----------------|------------|--------|------------|
| T-01     | RoBERTa Detector  | 2019 | OpenAI      | Text           | Acc 88%    | ▪      | Lit Review |
| T-02     | DetectGPT         | 2023 | ICML        | Text           | AUROC 0.95 | ▪      | Lit Review |
| T-03     | Fast-DetectGPT    | 2024 | ICLR        | Text           | AUROC 0.98 | ▪      | Optional   |
| T-04     | RADAR             | 2023 | NeurIPS     | Text           | F1 0.89    | ▪      | Lit Review |
| T-05     | RAID Benchmark    | 2024 | ACL         | Text           | F1 75%     | ▪<br>▪ | PRIMARY    |
| T-06     | Binoculars        | 2024 | ICML        | Text           | AUROC 0.99 | ▪      | Benchmark  |
| I-01     | CNNDetection      | 2020 | CVPR        | Image-GAN      | AUC 0.99   | ▪      | PRIMARY    |
| I-02     | UnivFD            | 2023 | CVPR        | Image          | AUC 0.95   | ▪      | Optional   |
| I-03     | DIRE              | 2023 | ICCV        | Image-Diff     | AUC 0.99   |        | Lit Review |
| I-04     | Corvi et al.      | 2023 | ICASSP      | Image-Diff     | AUC 0.93   |        | PRIMARY    |
| I-05     | GradCAM           | 2017 | ICCV        | Explainability | —          |        | Explain    |
| A-01     | AASIST            | 2022 | ICASSP      | Audio          | EER 0.83%  |        | PRIMARY    |
| A-02     | RawGAT-ST         | 2021 | Interspeech | Audio          | EER 1.06%  |        | Lit Review |

| Paper ID | Title (Shortened) | Year | Conference | Task        | Metric    | GitHub | FYP Use    |
|----------|-------------------|------|------------|-------------|-----------|--------|------------|
| A-03     | Wav2Vec2          | 2022 | Odyssey    | Audio       | EER 1.8%  | •      | Phase 4    |
| A-04     | LCNN              | 2020 | ICASSP     | Audio       | EER 5.06% | •      | Baseline   |
| V-01     | FaceForensics+    | 2019 | ICCV       | Video       | AUC ~0.95 | •      | Phase 4    |
| V-02     | SBI               | 2022 | CVPR       | Video       | AUC 0.93  |        | Phase 4    |
| V-03     | FTCN              | 2021 | ICCV       | Video       | AUC 0.98  |        | Lit Review |
| M-01     | FakeBench         | 2024 | arXiv      | Multi-Modal | Acc 68%   |        | Lit Review |
| M-02     | OmniDetector      | 2024 | arXiv      | Multi-Modal | SOTA      |        | Motivation |

SECTION 6 — ARCHITECTURE DESIGN

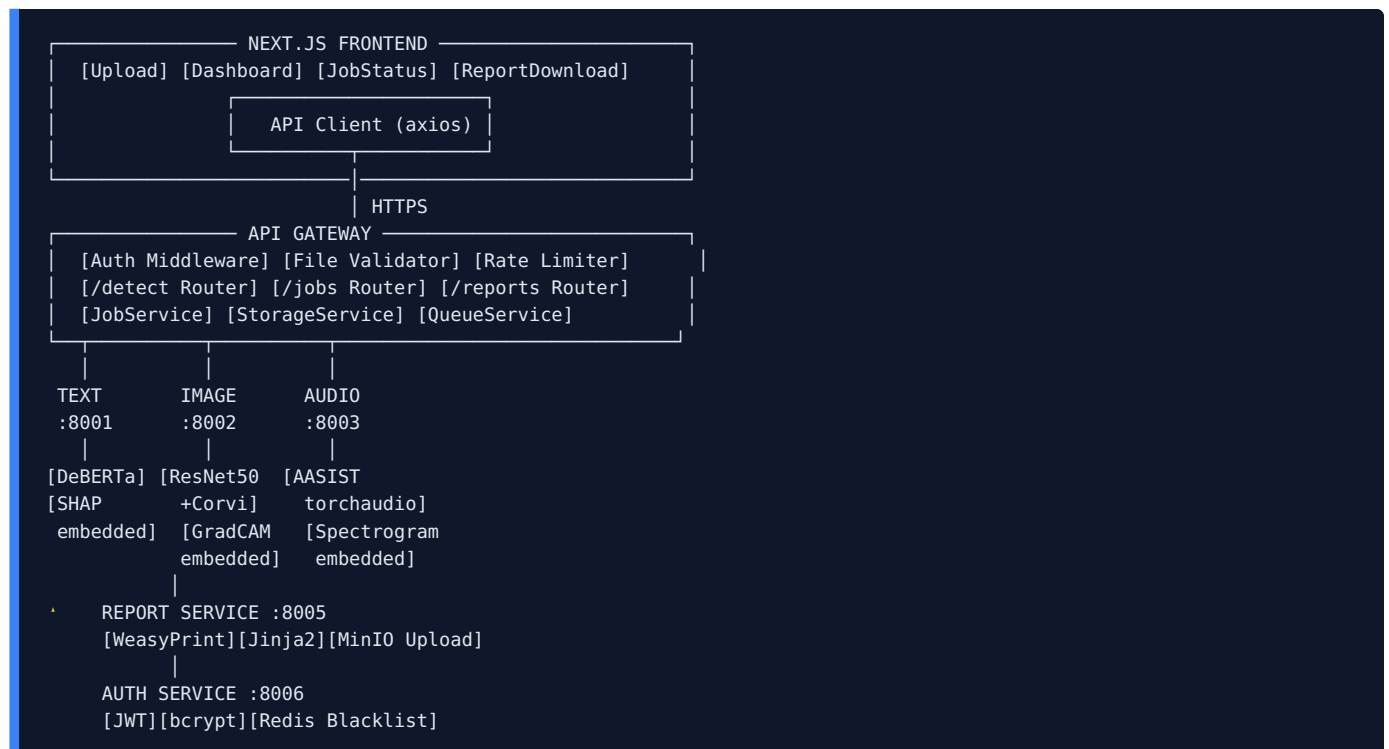
6.1 High-Level Architecture



## 6.2 Low-Level Data Flow Architecture

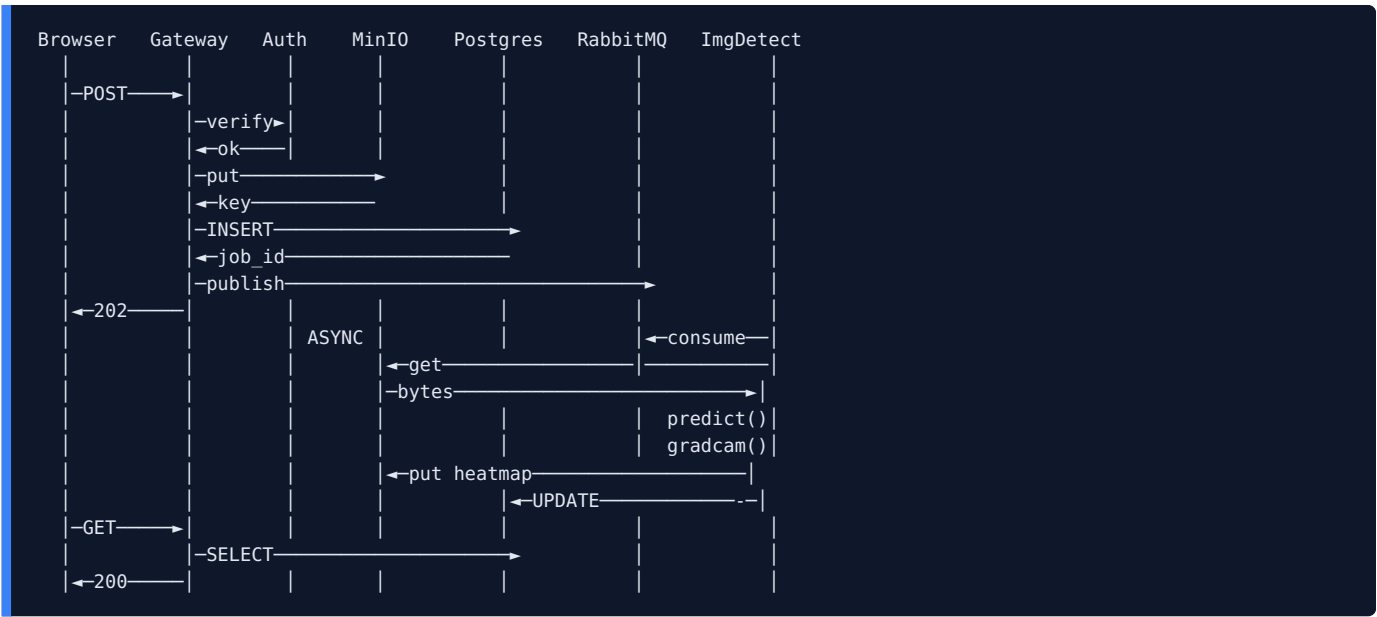


## 6.3 Component Diagram

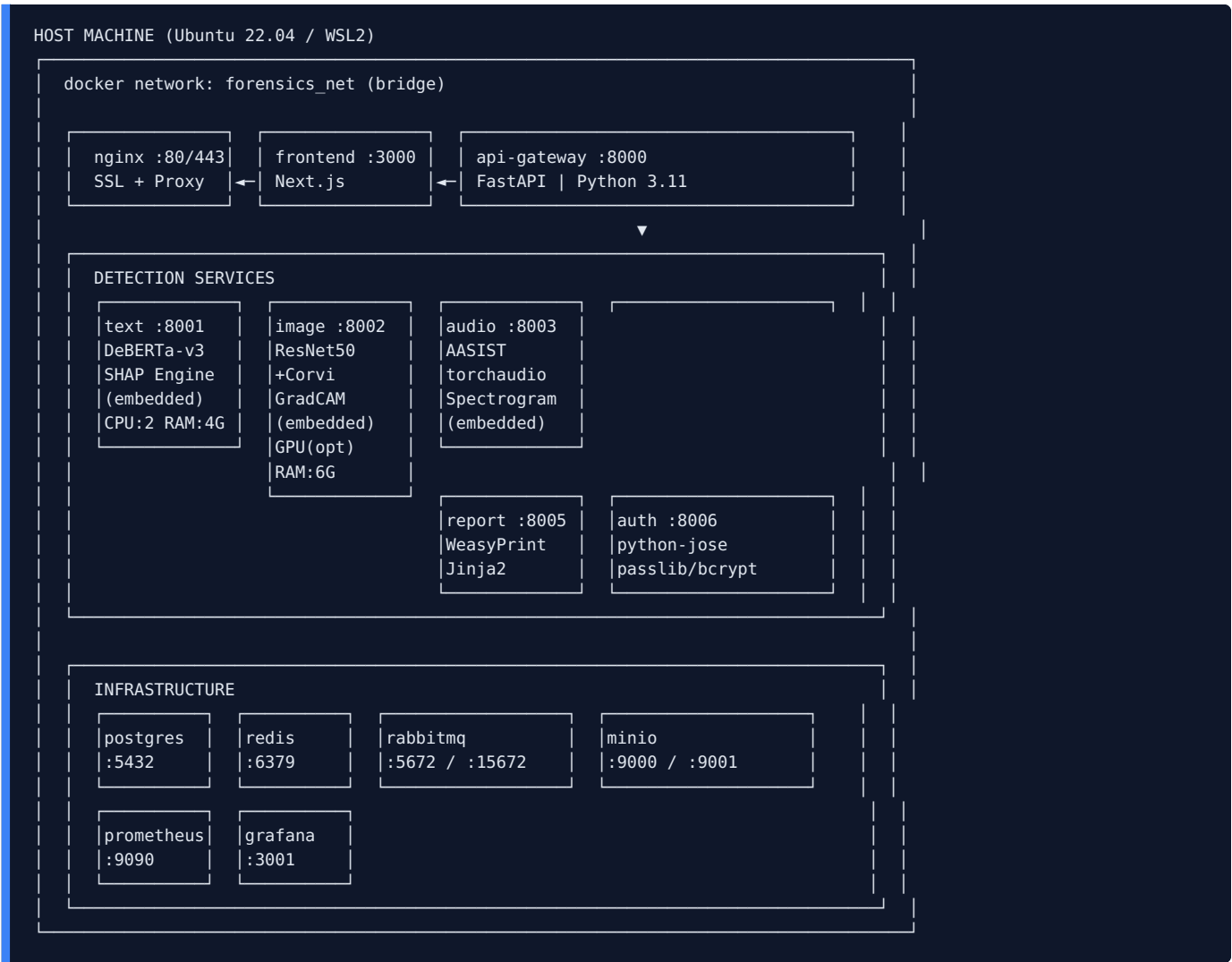


**Architecture Note:** There is NO standalone Explainability microservice. SHAP is embedded inside `text-service`. GradCAM is embedded inside `image-service`. Mel-Spectrogram is embedded inside `audio-service`. This eliminates cross-service serialization of large tensors and model objects over the network. Each detection service owns its own explainability pipeline.

## 6.4 Sequence Diagram — Full Async Detection



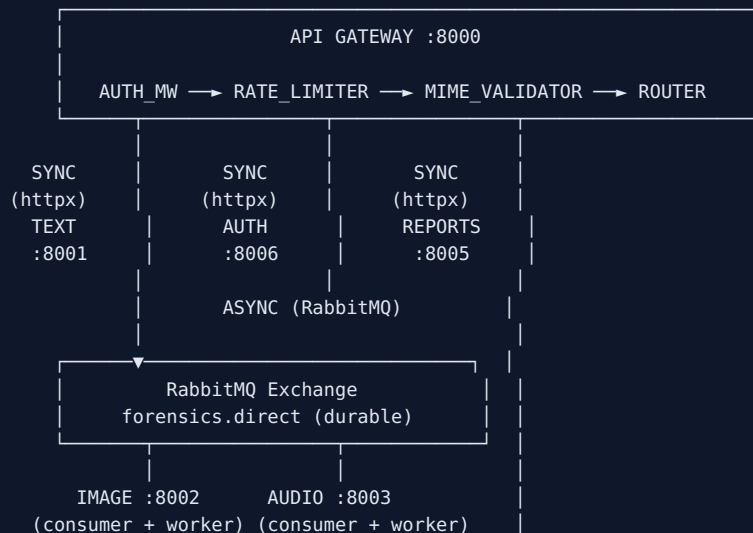
## 6.5 Docker Compose Deployment Diagram



## SECTION 7 — MICROSERVICE DESIGN

**Design Philosophy:** Each microservice is a self-contained, independently deployable unit with its own internal architecture, database access layer, and startup lifecycle. Services communicate via HTTP (synchronous, low-latency) or RabbitMQ (asynchronous, backpressure-tolerant). All services expose a `/internal/health` endpoint following the same contract. No service shares application code via import; only the `shared/` pip package is shared.

### 7.0 Inter-Service Communication Architecture



#### 7.0.1 Synchronous Communication (httpx)

Used for: **Text detection** (< 2s), **Auth** (< 50ms), **Report generation** (triggered, not polled)

```
# shared/http_client.py – used by API Gateway to call sync services
import httpx
from tenacity import retry, stop_after_attempt, wait_exponential, retry_if_exception_type

_clients: dict[str, httpx.AsyncClient] = {}

def get_http_client(service_url: str) -> httpx.AsyncClient:
    if service_url not in _clients:
        _clients[service_url] = httpx.AsyncClient(
            base_url=service_url,
            timeout=httpx.Timeout(connect=2.0, read=30.0, write=5.0, pool=2.0),
            limits=httpx.Limits(max_connections=20, max_keepalive_connections=10),
        )
    return _clients[service_url]

@retry(
    retry=retry_if_exception_type((httpx.ConnectError, httpx.TimeoutException)),
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=0.5, max=4),
    reraise=True,
)
async def call_service(service_url: str, endpoint: str, payload: dict) -> dict:
    client = get_http_client(service_url)
    response = await client.post(endpoint, json=payload)
    response.raise_for_status()
    return response.json()
```

7.0.2 Asynchronous Communication (RabbitMQ)

Used for: **Image detection** (~5–30s), **Audio detection** (~5–30s), **Video detection** (minutes)

```
Queue topology:
  forensics.image.detect    → image-service consumer
  forensics.audio.detect    → audio-service consumer
  forensics.video.detect    → video-service consumer
  forensics.*.dlx           → dead-letter queues (failed jobs)

Exchange: forensics.direct (type=direct, durable=true)
DLX Exchange: forensics.dlx (type=direct, durable=true)
Message TTL: 600,000ms (10 min)
Max retries: 3 (x-death header checked in consumer)
```

7.0.3 Circuit Breaker Pattern

```
# shared/circuit_breaker.py
from circuitbreaker import circuit

@circuit(failure_threshold=5, recovery_timeout=30, expected_exception=Exception)
async def protected_call(service_url: str, endpoint: str, payload: dict) -> dict:
    return await call_service(service_url, endpoint, payload)

# States: CLOSED (normal) → OPEN (failing) → HALF_OPEN (testing) → CLOSED
# After 5 failures: circuit opens for 30 seconds
# Gateway returns 503 with Retry-After: 30 header when circuit is open
```

7.0.4 Service Discovery (Docker Compose DNS)

All services reference each other by container name. No service registry needed at FYP scale:

| Service Name   | Internal URL                       | Protocol               |
|----------------|------------------------------------|------------------------|
| text-service   | http://text-service:8001           | HTTP/1.1 (httpx)       |
| image-service  | http://image-service:8002          | HTTP/1.1 (health only) |
| audio-service  | http://audio-service:8003          | HTTP/1.1 (health only) |
| auth-service   | http://auth-service:8006           | HTTP/1.1 (httpx)       |
| report-service | http://report-service:8005         | HTTP/1.1 (httpx)       |
| rabbitmq       | amqp://rabbitmq:5672               | AMQP 0-9-1             |
| postgres       | postgresql+asyncpg://postgres:5432 | TCP                    |
| redis          | redis://redis:6379                 | TCP                    |
| minio          | http://minio:9000                  | HTTP/S3                |

7.1 API Gateway Service (Port 8000)

7.1.1 Responsibilities & Internal Architecture



1. TrustedHostMiddleware – reject non-whitelisted hosts
2. CORSMiddleware – allow frontend origin only
3. RequestIDMiddleware – inject X-Request-ID UUID
4. StructuredLoggingMiddleware – log every request/response
5. RateLimitMiddleware – slowapi / Redis sliding window
6. JWTAuthMiddleware – validate Bearer token on /api/v1/\*  
(excludes: /auth/login, /auth/register, /health)

↓

#### ROUTE HANDLERS

```

/api/v1/auth/* → AuthRouter → calls auth-service (sync)
/api/v1/detect/* → DetectRouter → StorageService + QueueSvc
/api/v1/jobs/* → JobsRouter → JobRepository (direct DB)
/api/v1/reports/* → ReportRouter → calls report-service (sync)
/api/v1/models → ModelRouter → ModelRepository (direct DB)
/health → HealthRouter → checks all dependencies

```

↓

```

StorageService (MinIO)
upload_file() / presign_url()

```

```

QueueService (RabbitMQ)
publish_job()

```

## 7.1.2 Complete Endpoint Contract Specification

### POST /api/v1/detect/text

Auth: Bearer <access\_token>  
 Content-Type: application/json  
 Rate Limit: 30 req/min per user

#### Request Body:

```

{
  "text": string, // REQUIRED. Min 50 chars. Max 10,000 chars.
  "explain": boolean, // Optional. Default true. Trigger SHAP explanation.
  "language": string, // Optional. Default "en". ISO 639-1.
  "webhook_url": string // Optional. POST result here on completion.
}

```

#### Validation rules:

- text after strip() must be >= 50 characters
- text must be valid UTF-8
- language must be in ["en", "es", "fr", "de", "zh"] (others: 422)

#### Success Response 200 OK:

```

{
  "job_id": "uuid-v4",
  "status": "COMPLETED",
  "modality": "TEXT",
  "submitted_at": "2024-06-01T10:30:00Z",
  "completed_at": "2024-06-01T10:30:02Z",
  "results": [{
    "model_name": "deberta-v3-raid-v1.2",
    "model_version": "1.2.0",
    "ai_probability": 0.87,
    "confidence_score": 0.91,
    "verdict": "AI_GENERATED",
    "processing_time_ms": 340,
    "device_used": "cuda:0",
    "explanation": {
      "method": "SHAP",
      "top_ai_indicator_tokens": ["Furthermore", "it is worth noting", "utilization"],
      "top_human_tokens": ["however", "I think", "maybe"],
      "html_heatmap": "<span style='...'>...</span>",
      "truncated": false
    }
  }]
}

```

```

    }
  }
}

Error Responses:
400 { "error": "TEXT_TOO_SHORT",      "detail": "Text must be >= 50 characters" }
401 { "error": "TOKEN_INVALID",       "detail": "JWT signature mismatch" }
422 { "error": "VALIDATION_ERROR",    "detail": [...pydantic errors...] }
429 { "error": "RATE_LIMIT_EXCEEDED", "detail": "30 req/min limit", "retry_after": 45 }
503 { "error": "TEXT_SERVICE_DOWN",   "detail": "Upstream unavailable. Try again." }

```

## POST /api/v1/detect/image

```

Auth:      Bearer <access_token>
Content-Type: multipart/form-data
Rate Limit: 20 req/min per user

Form Fields:
file:      File // REQUIRED. Allowed MIME: image/jpeg, image/png, image/webp, image/gif
explain:    bool // Optional. Default true.

Server-side validation pipeline:
1. Content-Length header check    → reject if > 10MB before full upload
2. python-magic MIME check       → verify actual bytes match declared type
3. PIL Image.open() verify       → reject corrupted files
4. Dimension check               → reject if < 64x64 px

Success Response 202 Accepted:
{
  "job_id":          "uuid-v4",
  "status":          "QUEUED",
  "modality":        "IMAGE",
  "submitted_at":    "2024-06-01T10:30:00Z",
  "estimated_completion_secs": 15,
  "poll_url":        "/api/v1/jobs/uuid-v4",
  "object_key":      "uploads/uuid-v4/original.jpg"
}

Completed Job Response 200 OK (from GET /api/v1/jobs/{id}):
{
  "job_id":          "uuid-v4",
  "status":          "COMPLETED",
  "modality":        "IMAGE",
  "results": [{
    "model_name":      "image-ensemble-v1.0",
    "ai_probability":  0.73,
    "confidence_score": 0.61,
    "verdict":         "LIKELY_AI",
    "model_ensemble": [
      { "model": "ResNet50-CNNDetection", "weight": 0.55, "score": 0.82 },
      { "model": "Corvi-EfficientNetB0",  "weight": 0.45, "score": 0.61 }
    ],
    "explanation": {
      "method":        "GradCAM",
      "heatmap_url":    "https://minio/forensics-bucket/heatmaps/uuid-v4/gradcam.png?token=...",
      "heatmap_url_expiry": "2024-06-01T22:30:00Z",
      "highlighted_regions": [
        { "region": "top-right", "intensity": 0.87 },
        { "region": "center",    "intensity": 0.54 }
      ]
    }
  ]
}
}

Error Responses:
400 { "error": "INVALID_MIME_TYPE",    "detail": "Got application/pdf, expected image/*" }
413 { "error": "FILE_TOO_LARGE",      "detail": "Max 10MB. Got 14.2MB" }
415 { "error": "CORRUPTED_IMAGE",     "detail": "Cannot decode image header" }
422 { "error": "IMAGE_TOO_SMALL",     "detail": "Min 64x64 px required" }

```

## POST /api/v1/detect/audio

Auth: Bearer <access\_token>  
Content-Type: multipart/form-data  
Rate Limit: 10 req/min per user

### Form Fields:

file: File // REQUIRED. Allowed MIME: audio/mpeg, audio/wav, audio/flac, audio/ogg, audio/mp4

### Server-side validation:

1. MIME check via python-magic
2. Duration check via torchaudio.info() – reject if < 1s or > 300s
3. Channel count – accept mono or stereo (auto-convert to mono)

### Success Response 202 Accepted:

```
{
  "job_id": "uuid-v4",
  "status": "QUEUED",
  "modality": "AUDIO",
  "submitted_at": "2024-06-01T10:30:00Z",
  "estimated_completion_secs": 30,
  "poll_url": "/api/v1/jobs/uuid-v4"
}
```

### Completed Job Response 200 OK:

```
{
  "job_id": "uuid-v4",
  "status": "COMPLETED",
  "modality": "AUDIO",
  "results": [{
    "model_name": "aasist-l-v1.0",
    "ai_probability": 0.92,
    "confidence_score": 0.88,
    "verdict": "AI GENERATED",
    "processing_time_ms": 45,
    "explanation": {
      "method": "Mel-Spectrogram",
      "spectrogram_url": "https://minio/.../spectrogram.png?token=...",
      "duration_analyzed_s": 4.03,
      "sample_rate": 16000,
      "anomaly_flags": ["unnatural_f0_continuity", "harmonic_discontinuity"]
    }
  }]
}
```

## POST /api/v1/detect/multimodal

Auth: Bearer <access\_token>  
Content-Type: multipart/form-data  
Rate Limit: 5 req/min per user

### Form Fields:

text\_content: string // Optional. At least one of text/image/audio must be present.  
image\_file: File // Optional.  
audio\_file: File // Optional.  
explain: bool // Optional. Default true.

### Validation: At least 2 modalities must be provided for fusion to be meaningful.

- 1 modality → 400 { "error": "INSUFFICIENT\_MODALITIES",  
"detail": "Provide ≥ 2 modalities for multimodal fusion" }

### Success Response 202 Accepted:

```
{
  "job_id": "uuid-v4",
  "status": "QUEUED",
  "modality": "MULTIMODAL",
  "child_jobs": {
    "text": "uuid-text-job",
```

```

    "image": "uuid-image-job"
  },
  "estimated_completion_secs": 40,
  "poll_url": "/api/v1/jobs/uuid-v4"
}

```

Completed Fusion Response:

```

{
  "job_id": "uuid-v4",
  "status": "COMPLETED",
  "modality": "MULTIMODAL",
  "fusion_result": {
    "final_score": 0.84,
    "final_confidence": 0.79,
    "final_verdict": "AI_GENERATED",
    "fusion_method": "weighted_linear_calibrated",
    "weights_applied": { "text": 0.55, "image": 0.45 },
    "modality_scores": { "text": 0.87, "image": 0.79 },
    "agreement_level": "HIGH",
    "agreement_delta": 0.08
  },
  "child_results": [
    { "modality": "TEXT", "ai_probability": 0.87, "verdict": "AI_GENERATED" },
    { "modality": "IMAGE", "ai_probability": 0.79, "verdict": "LIKELY_AI" }
  ]
}

```

#### GET /api/v1/jobs/{job\_id}

Auth: Bearer <access\_token>

Authorization: User may only access their own jobs (checked via user\_id in JWT claim)

Response 200 OK – PENDING:

```

{
  "job_id": "uuid-v4",
  "status": "PROCESSING",
  "modality": "IMAGE",
  "submitted_at": "2024-06-01T10:30:00Z",
  "progress_pct": 40,
  "current_step": "model_inference",
  "results": null
}

```

Response 200 OK – FAILED:

```

{
  "job_id": "uuid-v4",
  "status": "FAILED",
  "error_code": "MODEL_OOM",
  "error_message": "CUDA out of memory. Image resized and retried but still failed.",
  "failed_at": "2024-06-01T10:30:45Z",
  "retry_attempted": true
}

```

Response 403: { "error": "ACCESS\_DENIED", "detail": "Job belongs to different user" }

Response 404: { "error": "JOB\_NOT\_FOUND", "detail": "No job with id uuid-v4" }

#### GET /api/v1/health

Response 200 OK (all healthy):

```

{
  "status": "healthy",
  "version": "1.0.0",
  "timestamp": "2024-06-01T10:30:00Z",
  "dependencies": {
    "postgres": { "status": "up", "latency_ms": 2 },
    "redis": { "status": "up", "latency_ms": 1 },
    "rabbitmq": { "status": "up", "latency_ms": 3 },
  }
}

```

```

    "minio":      { "status": "up", "latency_ms": 5 },
    "text_service": { "status": "up", "latency_ms": 12 },
    "auth_service": { "status": "up", "latency_ms": 8 },
    "report_service": { "status": "up", "latency_ms": 11 }
  }
}

Response 503 (degraded):
{
  "status": "degraded",
  "dependencies": {
    "postgres": { "status": "up", "latency_ms": 2 },
    "text_service": { "status": "down", "error": "Connection refused" }
  }
}

```

### 7.1.3 Rate Limiting Strategy

```

# services/api-gateway/app/middleware/rate_limiter.py
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address, storage_uri="redis://redis:6379/1")

# Applied per endpoint:
# @limiter.limit("30/minute")    → /detect/text
# @limiter.limit("20/minute")    → /detect/image
# @limiter.limit("10/minute")    → /detect/audio
# @limiter.limit("5/minute")     → /detect/multimodal
# @limiter.limit("60/minute")    → /jobs/* (polling)
# @limiter.limit("100/minute")   → /health

# Response headers on every request:
# X-RateLimit-Limit: 30
# X-RateLimit-Remaining: 22
# X-RateLimit-Reset: 1717235460

```

### 7.1.4 File Validation Pipeline

```

# services/api-gateway/app/services/file_validator.py
import magic, io
from PIL import Image
import torchaudio

ALLOWED_IMAGE_MIMES = {"image/jpeg", "image/png", "image/webp", "image/gif"}
ALLOWED_AUDIO_MIMES = {"audio/mpeg", "audio/wav", "audio/x-wav", "audio/flac",
                        "audio/ogg", "audio/mp4", "audio/x-m4a"}
MAX_IMAGE_BYTES = 10 * 1024 * 1024 # 10 MB
MAX_AUDIO_BYTES = 50 * 1024 * 1024 # 50 MB

async def validate_image(file_bytes: bytes) -> None:
    if len(file_bytes) > MAX_IMAGE_BYTES:
        raise FileTooLargeError(f"Max 10MB. Got {len(file_bytes)/1e6:.1f}MB")
    mime = magic.from_buffer(file_bytes[:2048], mime=True)
    if mime not in ALLOWED_IMAGE_MIMES:
        raise InvalidMimeTypeError(f"Got {mime}, expected image/*")
    try:
        img = Image.open(io.BytesIO(file_bytes))
        img.verify()
        img = Image.open(io.BytesIO(file_bytes))
        if img.width < 64 or img.height < 64:
            raise ImageTooSmallError(f"Min 64x64 px. Got {img.width}x{img.height}")
    except Exception as e:
        raise CorruptedImageError(str(e))

async def validate_audio(file_bytes: bytes) -> float:
    if len(file_bytes) > MAX_AUDIO_BYTES:
        raise FileTooLargeError(f"Max 50MB. Got {len(file_bytes)/1e6:.1f}MB")
    mime = magic.from_buffer(file_bytes[:2048], mime=True)
    if mime not in ALLOWED_AUDIO_MIMES:

```

```

        raise InvalidMimeError(f"Got {mime}, expected audio/*")
    try:
        info = torchaudio.info(io.BytesIO(file_bytes))
        duration_s = info.num_frames / info.sample_rate
        if duration_s < 1.0:
            raise AudioTooShortError(f"Min 1s. Got {duration_s:.2f}s")
        if duration_s > 300.0:
            raise AudioTooLongError(f"Max 300s. Got {duration_s:.1f}s")
        return duration_s
    except Exception as e:
        raise CorruptedAudioError(str(e))

```

### 7.1.5 API Gateway Startup Lifecycle

```

# services/api-gateway/app/main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI
from app.core.config import settings
from app.db.session import init_db
from app.services.storage_service import StorageService
from app.services.queue_service import QueueService

@asynccontextmanager
async def lifespan(app: FastAPI):
    # — STARTUP —
    await init_db() # test DB connectivity
    app.state.storage = StorageService()
    await app.state.storage.ensure_bucket() # create MinIO bucket if absent
    app.state.queue = QueueService()
    await app.state.queue.connect() # establish RabbitMQ connection pool
    await app.state.queue.declare_topology() # queues + exchanges + DLX
    yield
    # — SHUTDOWN —
    await app.state.queue.close()

app = FastAPI(title="Forensics Platform API", version="1.0.0",
              lifespan=lifespan, docs_url="/docs", redoc_url="/redoc")

```

### 7.1.6 Resource Budget

| Resource    | Allocated                                       | Justification                                  |
|-------------|-------------------------------------------------|------------------------------------------------|
| CPU         | 2 cores                                         | FastAPI async — I/O bound, not CPU bound       |
| RAM         | 512 MB                                          | No ML models loaded; pure routing + validation |
| Connections | 20 DB pool / 10 httpx keep-alive per downstream | Concurrent users × expected concurrency        |
| Open Files  | 1024                                            | File upload streams                            |
| SLA         | P95 < 200ms (excluding model inference)         | Gateway overhead only                          |

## 7.2 Text Detection Service (Port 8001)

### 7.2.1 Responsibilities & Internal Architecture

```

RabbitMQ Consumer    HTTP (internal)    Gateway calls
(async text jobs)    OR POST /internal/detect    (sync path)
    |                  |

```

### TEXT DETECTION SERVICE

#### Startup:

1. Load DeBERTa tokenizer + model to device
2. Warmup: run 1 dummy inference (fill CUDA cache)
3. Init SHAP Explainer (masker + predict\_fn)
4. Start RabbitMQ consumer (if ASYNC\_MODE=true)

#### Per Request:

TokenizerPreprocess → ModelInference → ScoreMap

└─ [if explain=true] SHAPExplainer

WriteDB ←

WriteMinIO (heatmap HTML blob)

ACK queue message / return HTTP response

## 7.2.2 Complete Endpoint Contracts

### POST /internal/detect (called synchronously by API Gateway)

Content-Type: application/json  
Internal only – not exposed externally

#### Request:

```
{
  "job_id": "uuid-v4",           // REQUIRED
  "text": "string",             // REQUIRED. Already validated by gateway.
  "explain": true,              // Optional. Default true.
  "language": "en",            // Optional.
  "user_id": "uuid-user"       // For audit logging.
}
```

#### Response 200 OK:

```
{
  "job_id": "uuid-v4",
  "model_name": "deberta-v3-raio-v1.2",
  "model_version": "1.2.0",
  "ai_probability": 0.87,
  "confidence_score": 0.91,
  "verdict": "AI_GENERATED",
  "processing_time_ms": 340,
  "device_used": "cuda:0",
  "token_count": 187,
  "was_truncated": false,
  "explanation": {
    "method": "SHAP",
    "token_attributions": [...top 20 tokens with SHAP values...],
    "top_ai_indicator_tokens": ["Furthermore", "it is worth noting"],
    "top_human_tokens": ["I think", "actually"],
    "html_heatmap": "<span ...>...</span>",
    "shap_computation_ms": 2100,
    "truncated": false
  }
}
```

Response 422: { "error": "TEXT\_PREPROCESSING\_FAILED", "detail": "..." }

Response 500: { "error": "MODEL\_INFERENCE\_FAILED", "detail": "..." }

Response 503: { "error": "MODEL\_NOT\_LOADED", "detail": "Warmup in progress" }

### GET /internal/health

Response 200:

```
{
  "service":      "text-service",
  "status":       "healthy",
  "model_loaded": true,
  "shap_ready":   true,
  "device":       "cuda:0",
  "gpu_memory_mb": { "allocated": 1842, "reserved": 2048, "total": 8192 },
  "warmup_done":  true,
  "uptime_s":     3600,
  "requests_served": 1247,
  "avg_latency_ms": 380
}
```

GET /internal/model/info

Response 200:

```
{
  "model_name":      "deberta-v3-rai-v1.2",
  "model_version":   "1.2.0",
  "architecture":    "DeBERTaV2ForSequenceClassification",
  "parameters":      86_000_000,
  "training_data":    "RAID-2024-EN-100K",
  "val_f1":           0.891,
  "val_auc":          0.943,
  "max_tokens":       512,
  "device":           "cuda:0",
  "checkpoint_sha256": "a3f7b21c..."
}
```

### 7.2.3 Failure Handling & Edge Cases

| Scenario            | Detection                                       | Response Strategy                                                                                                                                |
|---------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Text > 10,000 chars | <code>len(text) &gt; 10000</code>               | Truncate at last sentence boundary before char 10,000. Set <code>was_truncated: true</code> in response.                                         |
| Text < 50 chars     | <code>len(text.strip()) &lt; 50</code>          | Return 422 immediately before any inference. Do NOT send to model.                                                                               |
| Non-English text    | <code>langdetect(text) != "en"</code>           | Run inference anyway but add <code>"language_warning": "Non-English detected. Accuracy may be reduced."</code>                                   |
| CUDA OOM            | <code>torch.cuda.OutOfMemoryError</code>        | Catch, move model to CPU, retry once. Log CUDA OOM event to Prometheus counter.                                                                  |
| SHAP timeout        | <code>asyncio.wait_for(..., timeout=5.0)</code> | Catch <code>asyncio.TimeoutError</code> , return result WITHOUT explanation. Set <code>"explanation": null, "explanation_timeout": true</code> . |
| Model not loaded    | Check singleton at request start                | Return 503 with <code>Retry-After: 30</code> . Do NOT attempt inference.                                                                         |
| Tokenizer failure   | <code>TokenizerException</code>                 | Return 422. Happens on malformed unicode.                                                                                                        |

7.2.4 Concurrency Model

```
# Text service runs single worker (1 model instance) with async inference
# Model inference is CPU/GPU-bound → offloaded to thread pool
# SHAP computation is CPU-bound → offloaded to thread pool

import asyncio

async def detect(request: TextDetectRequest) -> TextDetectResponse:
    # Inference in thread pool (non-blocking event loop)
    result = await asyncio.to_thread(model.predict_sync, request.text)
    if request.explain:
        explanation = await asyncio.wait_for(
            asyncio.to_thread(explainer.explain_sync, request.text),
            timeout=5.0
        )
        result["explanation"] = explanation
    return TextDetectResponse(**result)

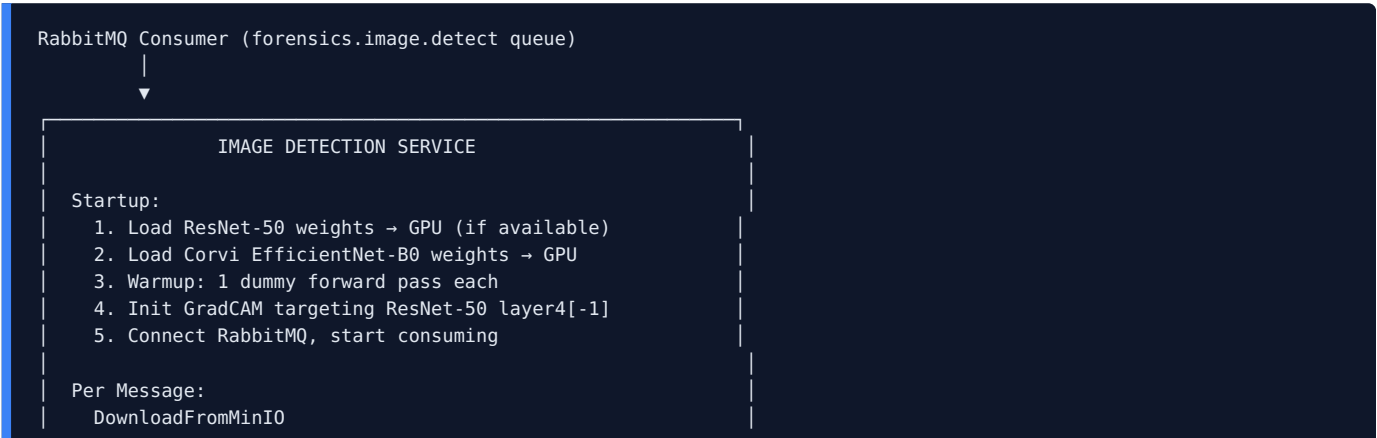
# uvicorn --workers 1 --loop uvloop
# DO NOT use --workers > 1 with loaded PyTorch models
# (model weights duplicated in every worker process = OOM)
# Use GPU reservation via Docker if needed for parallelism
```

7.2.5 Resource Budget & SLA

| Resource                | Allocated                         | Note                                                         |
|-------------------------|-----------------------------------|--------------------------------------------------------------|
| CPU                     | 2 cores                           | Inference + SHAP offloaded to <code>asyncio.to_thread</code> |
| RAM                     | 4 GB                              | DeBERTa-v3: ~1.7GB. SHAP masker: ~500MB. OS overhead.        |
| GPU (optional)          | 2 GB VRAM                         | If GPU available; falls back to CPU automatically            |
| Workers                 | 1 uvicorn worker                  | Single model instance. Never set > 1.                        |
| P95 Latency (no SHAP)   | < 500ms GPU / < 1500ms CPU        | Target for text-only verdict                                 |
| P95 Latency (with SHAP) | < 5000ms                          | SHAP adds 2–4s for 256 tokens                                |
| Throughput              | ~2 req/s (GPU) / ~0.7 req/s (CPU) | With SHAP enabled                                            |

7.3 Image Detection Service (Port 8002)

7.3.1 Responsibilities & Internal Architecture



```

    ↳ [Thread A] ResNet-50 Predict()
      |   ↳ GradCAM Heatmap Generate
      |   ↳ Upload heatmap PNG to MinIO
    ↳ [Thread B] Corvi FFT + EfficientNet Predict()

Ensemble(p_resnet, p_corvi) → Verdict → Confidence
WriteDetectionResult → WriteExplanation → UPDATE job
ACK message

```

### 7.3.2 Ensemble Algorithm (Detailed)

```

# services/image-service/app/services/ensemble.py

RESNET_WEIGHT = 0.55 # Higher: better generalization to GANs
CORVI_WEIGHT  = 0.45 # Lower: strong on diffusion, weaker on old GANs

def compute_ensemble(p_resnet: float, p_corvi: float) -> dict:
    """
    Weighted linear combination with divergence-based confidence.

    Agreement cases:
    Both HIGH (>0.7): ensemble ~0.75+, confidence HIGH
    Both LOW (<0.3): ensemble ~0.25-, confidence HIGH
    Divergent (gap>0.4): confidence LOW → return INCONCLUSIVE leaning
    """
    p_ensemble = RESNET_WEIGHT * p_resnet + CORVI_WEIGHT * p_corvi

    # Confidence: HIGH when both models agree, LOW when they diverge
    divergence = abs(p_resnet - p_corvi)
    confidence = max(0.0, 1.0 - 1.5 * divergence) # linear decay

    # When models strongly disagree, pull ensemble toward 0.5 (less certain)
    if divergence > 0.40:
        p_ensemble = 0.5 * p_ensemble + 0.5 * 0.50 # blend toward neutral
        confidence = min(confidence, 0.35)          # hard cap confidence

    return {
        "ai_probability": round(p_ensemble, 4),
        "confidence_score": round(confidence, 4),
        "verdict": _score_to_verdict(p_ensemble),
        "model_ensemble": [
            {"model": "ResNet50-CNNDetection", "weight": RESNET_WEIGHT, "score": round(p_resnet, 4)},
            {"model": "Corvi-EfficientNetB0", "weight": CORVI_WEIGHT, "score": round(p_corvi, 4)},
        ],
        "divergence": round(divergence, 4),
    }

```

### 7.3.3 Complete Endpoint Contracts

**GET /internal/health**

```

Response 200:
{
  "service": "image-service",
  "status": "healthy",
  "models_loaded": {
    "resnet50": true,
    "corvi_effnetb0": true
  },
  "gradcam_ready": true,
  "queue_consumer": "connected",
  "queue_name": "forensics.image.detect",
  "messages_pending": 3,
  "device": "cuda:0",
  "gpu_memory_mb": { "allocated": 1460, "reserved": 1600 },
  "jobs_processed": 412,

```

```
"avg_latency_ms": 820
}
```

#### GET /internal/model/info

```
Response 200:
{
  "models": [
    {
      "name": "ResNet50-CNNDetection",
      "version": "2.0",
      "weights": "blur_jpg_prob0.5.pth",
      "parameters": 25_557_032,
      "trained_on": "ProGAN (11 classes, blur+JPEG augmented)",
      "target_layer": "layer4.1.conv2",
      "weight_in_ensemble": 0.55
    },
    {
      "name": "Corvi-EfficientNetB0",
      "version": "1.0",
      "parameters": 5_288_548,
      "trained_on": "LDM, DALL-E2, GLIDE (frequency domain)",
      "preprocessing": "2D-FFT high-pass residuals + log-magnitude",
      "weight_in_ensemble": 0.45
    }
  ]
}
```

### 7.3.4 Failure Handling & Edge Cases

| Scenario                   | Detection                                         | Strategy                                                                                   |
|----------------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------|
| RGBA image                 | <code>img.mode == "RGBA"</code>                   | <code>img.convert("RGB")</code> — always at preprocessing                                  |
| Grayscale image            | <code>img.mode == "L"</code>                      | <code>img.convert("RGB")</code> — replicate channel ×3                                     |
| Very large image (>5000px) | <code>max(w,h) &gt; 5000</code>                   | Resize to 1024px max side before validation                                                |
| CUDA OOM                   | <code>torch.cuda.OutOfMemoryError</code>          | Resize input to 512×512, retry once on CPU                                                 |
| GradCAM failure            | Exception                                         | Return result WITHOUT heatmap. Set <code>"heatmap_url": null, "gradcam_error": true</code> |
| MinIO download failure     | <code>MinIOError</code>                           | Retry 3× with exp. backoff. After 3 fails → NACK message (→ DLX)                           |
| Message redelivered > 3×   | <code>x-death</code> header count                 | Route to <code>forensics.image.dlx</code> . Mark job <code>FAILED</code> .                 |
| Consumer crash             | Docker restart policy <code>unless-stopped</code> | Container restarts, consumer re-binds, unACKed messages re-queued                          |

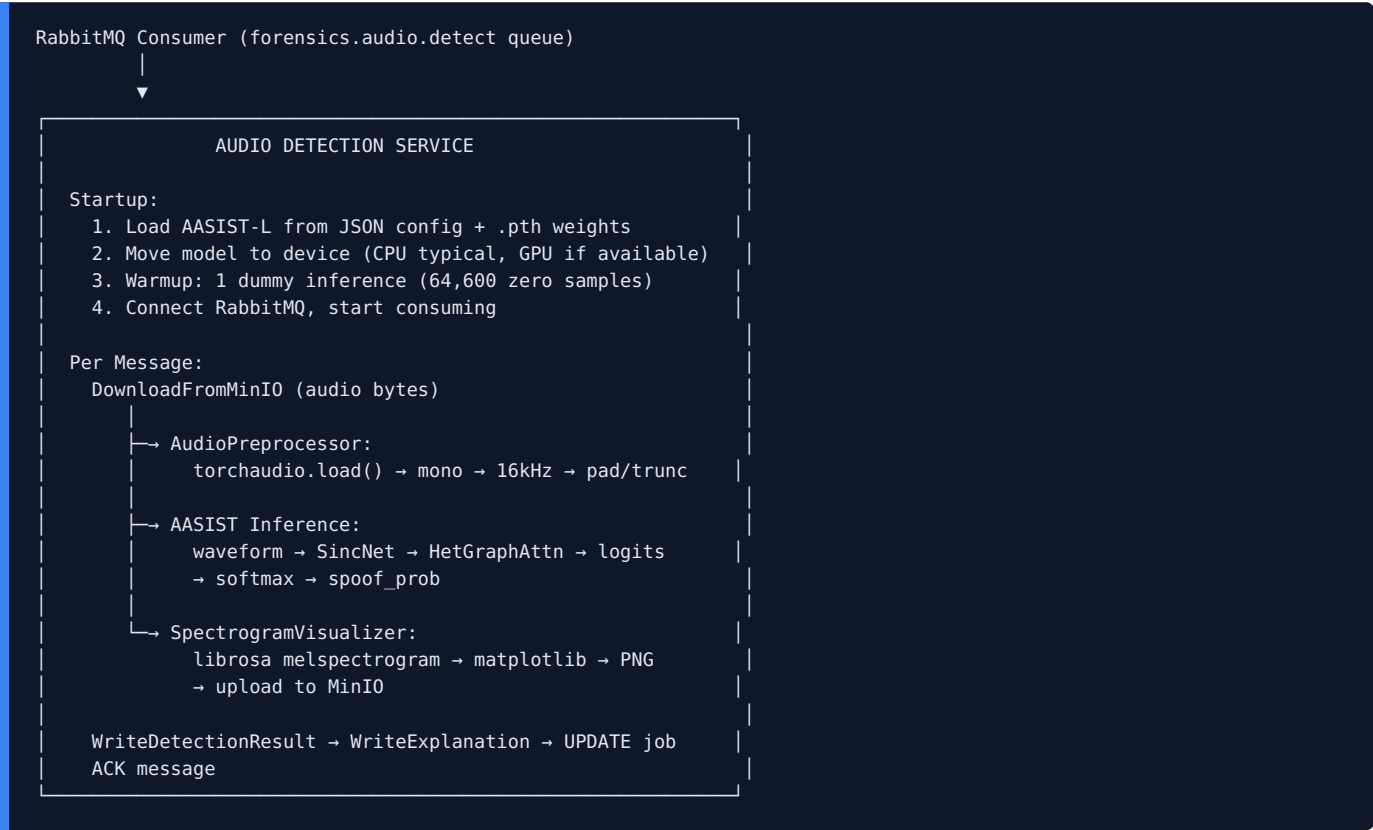
### 7.3.5 Resource Budget & SLA

| Resource | Allocated | Note                            |
|----------|-----------|---------------------------------|
| CPU      | 2 cores   | Preprocessing + GradCAM overlay |

| Resource          | Allocated         | Note                                              |
|-------------------|-------------------|---------------------------------------------------|
| RAM               | 6 GB              | ResNet50 (~380MB) + Corvi (~40MB) + image buffers |
| GPU               | 2 GB VRAM         | Both models loaded simultaneously                 |
| Workers           | 1 consumer thread | RabbitMQ <code>prefetch_count=1</code>            |
| P95 Latency (GPU) | < 800ms           | Both models + GradCAM                             |
| P95 Latency (CPU) | < 4000ms          | Fallback mode                                     |
| Max Queue Depth   | 50 messages       | Alert fires above threshold                       |

## 7.4 Audio Detection Service (Port 8003)

### 7.4.1 Responsibilities & Internal Architecture



### 7.4.2 Preprocessing Pipeline (Detailed)

```
# services/audio-service/app/services/audio_preprocessor.py
import io, torch, torchaudio
import torchaudio.transforms as AT

SAMPLE_RATE = 16_000          # AASIST requirement: exactly 16kHz
FIXED_SAMPLES = 64_600        # ~4.03 seconds – AASIST training length
MAX_DURATION_S = 60.0         # Truncate longer audio to 60s before AASIST
WARN_DURATION_S = 30.0        # Flag very long clips

class AudioPreprocessor:
    def preprocess(self, audio_bytes: bytes) -> tuple[torch.Tensor, dict]:
        """Returns (waveform_tensor [1, 64600], metadata_dict)"""
        waveform, orig_sr = torchaudio.load(io.BytesIO(audio_bytes))
        metadata = {
```

```

        "original_channels": waveform.shape[0],
        "original_sample_rate": orig_sr,
        "original_duration_s": round(waveform.shape[1] / orig_sr, 3),
    }

    # Step 1: Downmix to mono
    if waveform.shape[0] > 1:
        waveform = waveform.mean(dim=0, keepdim=True)

    # Step 2: Resample to 16kHz
    if orig_sr != SAMPLE_RATE:
        waveform = AT.Resample(orig_freq=orig_sr, new_freq=SAMPLE_RATE)(waveform)

    # Step 3: Truncate to MAX_DURATION_S before AASIST window
    max_samples = int(MAX_DURATION_S * SAMPLE_RATE)
    if waveform.shape[1] > max_samples:
        waveform = waveform[:, :max_samples]
        metadata["truncated"] = True

    # Step 4: Pad or crop to FIXED_SAMPLES (AASIST requires exact size)
    n = waveform.shape[1]
    if n < FIXED_SAMPLES:
        waveform = torch.nn.functional.pad(waveform, (0, FIXED_SAMPLES - n))
    else:
        waveform = waveform[:, :FIXED_SAMPLES]

    metadata["preprocessed_duration_s"] = round(FIXED_SAMPLES / SAMPLE_RATE, 3)
    return waveform, metadata

```

### 7.4.3 Anomaly Flag Generation

```

# services/audio-service/app/services/anomaly_detector.py
import librosa, numpy as np

def detect_audio_anomalies(audio_bytes: bytes, ai_probability: float) -> list[str]:
    """
    Heuristic anomaly flags for human-readable explanation.
    Not classification features – post-hoc descriptors only.
    """
    flags = []
    if ai_probability < 0.50:
        return flags    # Only generate flags for high-probability AI audio

    y, sr = librosa.load(io.BytesIO(audio_bytes), sr=16000, mono=True)

    # Flag 1: Unnaturally stable pitch (F0 continuity)
    f0, voiced, _ = librosa.pyin(y, fmin=80, fmax=400)
    if voiced.sum() > 10:
        f0_std = np.nanstd(f0[voiced])
        if f0_std < 8.0:    # Human speech F0 std typically 20–60 Hz
            flags.append("unnatural_f0_continuity")

    # Flag 2: Harmonic discontinuity at phrase boundaries
    # (TTS models often have phase artifacts at sentence joins)
    rms = librosa.feature.rms(y=y)[0]
    silences = np.where(rms < rms.mean() * 0.1)[0]
    if len(silences) > 0:
        spec = np.abs(librosa.stft(y))
        harmonic_ratio = librosa.effects.harmonic(y).var() / (y.var() + 1e-8)
        if harmonic_ratio > 0.85:
            flags.append("harmonic_discontinuity")

    # Flag 3: Absence of room acoustics (TTS is often too clean)
    reverb_proxy = librosa.feature.spectral_rolloff(y=y, sr=sr)[0].std()
    if reverb_proxy < 500:
        flags.append("no_room_acoustics_detected")

    return flags[:3]    # Cap at 3 flags

```



```

P_vote = fraction of frames > 0.50
P_final = 0.5×P_mean + 0.3×P_max + 0.2×P_vote
|
Frame Grid Generator: thumbnail grid PNG → MinIO
(Top 9 frames by XceptionNet score, labelled)

```

### 7.5.2 Response Contract

Completed Video Job Response:

```

{
  "job_id": "uuid-v4",
  "modality": "VIDEO",
  "results": [{
    "model_name": "xceptionnet-ff++",
    "ai_probability": 0.78,
    "confidence_score": 0.65,
    "verdict": "LIKELY_AI",
    "video_metadata": {
      "duration_s": 45.3,
      "fps_analyzed": 2,
      "total_frames": 90,
      "frames_with_face": 61,
      "frames_analyzed": 61
    },
    "temporal_scores": {
      "p_mean": 0.74, "p_max": 0.96, "p_vote_fraction": 0.72,
      "score_over_time": [[0.0, 0.81], [0.5, 0.76], [1.0, 0.69]]
    },
    "explanation": {
      "method": "FrameGrid",
      "frame_grid_url": "https://minio/.../frame_grid.png?token=...",
      "top_frames": [
        {"timestamp_s": 12.5, "score": 0.96, "thumbnail_url": "..."},
        {"timestamp_s": 7.0, "score": 0.91, "thumbnail_url": "..."}
      ]
    }
  ]
}
}]
}

```

### 7.5.3 Resource Budget & SLA

| Resource    | Allocated                                              | Note                                                 |
|-------------|--------------------------------------------------------|------------------------------------------------------|
| CPU         | 4 cores                                                | ffmpeg subprocess + MTCNN                            |
| RAM         | 4 GB                                                   | Frame buffers (batch_size=8, each 224×224×3)         |
| GPU         | 3 GB VRAM                                              | XceptionNet: ~87MB. MTCNN: ~50MB. Batching overhead. |
| P95 Latency | < 30s for 60s video                                    | Dominated by ffmpeg extraction and face detection    |
| Status      | <b>Phase 4 — Implement only if Phases 1–5 complete</b> |                                                      |

## 7.6 Explainability — Embedded in Detection Services

**DESIGN DECISION:** There is NO standalone Explainability microservice on a separate port. Explainability algorithms are embedded directly inside each detection service:

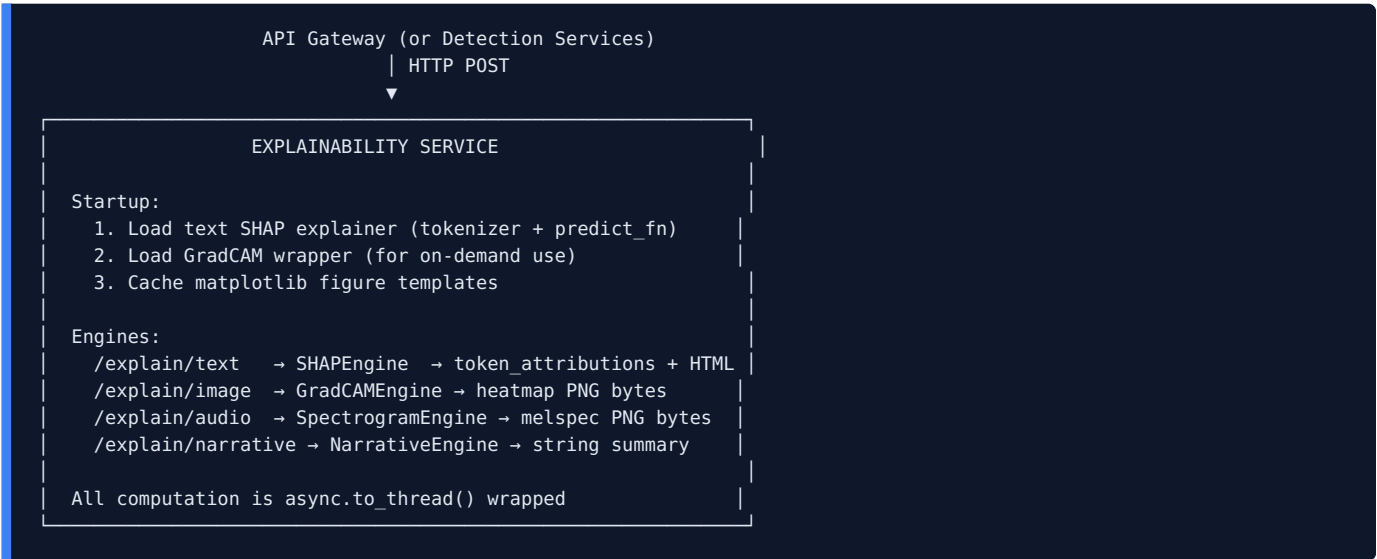
- **SHAP** lives inside `text-service` (shares the loaded DeBERTa model + tokenizer in memory — no serialization)
- **GradCAM** lives inside `image-service` (hooks directly onto ResNet-50 PyTorch graph — requires same process)
- **Mel-Spectrogram** lives inside `audio-service` (uses the already-loaded waveform bytes)

A standalone explainability service would require serializing PyTorch model graphs, weight tensors, and raw waveforms over HTTP — adding 200MB+ payloads, 2–5s network overhead, and complex model-loading duplication. The embedded architecture eliminates all of this.

### 7.6.1 Explainability Ownership Map

| Explainability Method  | Owned By                    | Port | Reason for Co-location                                             |
|------------------------|-----------------------------|------|--------------------------------------------------------------------|
| SHAP token attribution | <code>text-service</code>   | 8001 | Requires DeBERTa tokenizer + model in same process                 |
| GradCAM heatmap        | <code>image-service</code>  | 8002 | Hooks onto ResNet-50 <code>layer4</code> — must be in same process |
| Mel-spectrogram        | <code>audio-service</code>  | 8003 | Uses already-decoded waveform bytes                                |
| Narrative generator    | <code>report-service</code> | 8005 | String templating only; no model needed                            |
| Frame grid (video)     | <code>video-service</code>  | 8007 | Uses already-extracted frame tensors                               |

### 7.6.2 Responsibilities & Internal Architecture



### 7.6.2 Complete Endpoint Contracts

**POST** `/internal/explain/text`

Request:

```
{
  "job_id": "uuid-v4",
  "text": "string",
```

```
"ai_probability": 0.87,  
"model_name": "deberta-v3-raio-v1.2"  
}
```

Response 200:

```
{  
  "job_id": "uuid-v4",  
  "method": "SHAP",  
  "token_attributions": [  
    {"token": "Furthermore", "attribution": 0.312, "position": 0},  
    {"token": "worth", "attribution": 0.241, "position": 4},  
    ...  
  ],  
  "top_ai_indicator_tokens": ["Furthermore", "utilization", "it is worth noting"],  
  "top_human_tokens": ["I think", "actually", "hmm"],  
  "html_heatmap": "<span style='background:rgba(255,0,0,0.6)'>Furthermore</span>...",  
  "computation_ms": 2150,  
  "truncated": false  
}
```

#### POST /internal/explain/image

Request:

```
{  
  "job_id": "uuid-v4",  
  "object_key": "uploads/uuid-v4/original.jpg",  
  "model_name": "resnet50-cnndetection",  
  "ai_probability": 0.73  
}
```

Response 200:

```
{  
  "job_id": "uuid-v4",  
  "method": "GradCAM",  
  "heatmap_object_key": "heatmaps/uuid-v4/gradcam_overlay.png",  
  "highlighted_regions": [  
    {"region": "top-right", "intensity": 0.87},  
    {"region": "center", "intensity": 0.54}  
  ],  
  "computation_ms": 95  
}
```

#### POST /internal/explain/audio

Request:

```
{  
  "job_id": "uuid-v4",  
  "object_key": "uploads/uuid-v4/audio.mp3",  
  "ai_probability": 0.92  
}
```

Response 200:

```
{  
  "job_id": "uuid-v4",  
  "method": "Mel-Spectrogram",  
  "spectrogram_object_key": "spectrograms/uuid-v4/melspec.png",  
  "duration_analyzed_s": 4.03,  
  "sample_rate": 16000,  
  "anomaly_flags": ["unnatural_f0_continuity"],  
  "computation_ms": 310  
}
```

#### POST /internal/explain/narrative

Request:

```
{  
  "modality": "TEXT",  
  "verdict": "AI GENERATED",  
  "ai_probability": 0.87,  
}
```

```

"confidence_score": 0.91,
"explanation_data": { ...modality-specific fields... }
}

```

Response 200:

```

{
  "narrative": "The text content is classified as AI-generated (87% probability, 91% confidence). AI indicators: 'Furthermore', 'utilization', 'it is worth noting'. These phrases are statistically over-represented in LLM outputs trained on web text.",
  "narrative_tokens": 42,
  "generated_ms": 2
}

```

### 7.6.3 Resource Budget & SLA

| Resource        | Allocated         | Note                                                       |
|-----------------|-------------------|------------------------------------------------------------|
| CPU             | 4 cores           | SHAP and spectrogram are CPU-bound                         |
| RAM             | 3 GB              | SHAP masker (~500MB), matplotlib (~200MB), GradCAM buffers |
| SHAP P95        | < 5s (256 tokens) | Varies with text length                                    |
| GradCAM P95     | < 100ms           | Fast gradient computation                                  |
| Spectrogram P95 | < 400ms           | librosa mel + matplotlib render                            |

## 7.7 Report Generation Service (Port 8005)

### 7.7.1 Responsibilities & Internal Architecture

API Gateway → POST /internal/report/generate/{job\_id}



#### REPORT GENERATION SERVICE

```

Step 1: Fetch all data
  JobRepository.get_full_job(job_id)
  → job + detection_results + explanations + fusion_result
  → user profile

Step 2: Resolve MinIO artifacts
  Generate 1-hour presigned URLs for:
  - GradCAM heatmap PNG
  - Mel-spectrogram PNG
  → Embed as  in HTML
  (avoids URL expiry in PDF rendering)

Step 3: Jinja2 render
  template.render(job=..., results=...) → HTML string

Step 4: WeasyPrint → PDF bytes

Step 5: Upload PDF to MinIO
  reports/{job_id}/forensics_report_{timestamp}.pdf

Step 6: Write report record to `reports` table
  → return presigned download URL (24h expiry)

```

7.7.2 PDF Report Structure (5-Page Spec)

|                                                                      |
|----------------------------------------------------------------------|
| Page 1 – Executive Summary                                           |
| └─ Platform header + job ID + timestamp                              |
| └─ FINAL VERDICT badge (color-coded: RED/ORANGE/GREY/GREEN)          |
| └─ Overall AI probability (large dial graphic)                       |
| └─ Confidence score                                                  |
| └─ Modalities analyzed (checklist)                                   |
| └─ One-paragraph narrative explanation                               |
| Page 2 – Per-Modality Results Table                                  |
| └─ Row per modality: Model   Score   Verdict   Confidence   Time     |
| └─ Ensemble breakdown (image only): ResNet vs Corvi                  |
| └─ Fusion weights applied (multimodal only)                          |
| Page 3 – Visual Evidence                                             |
| └─ [Text] SHAP heatmap (rendered HTML → CSS box model in WeasyPrint) |
| └─ [Image] Original + GradCAM overlay side-by-side                   |
| └─ [Audio] Mel-spectrogram + waveform (2 subplots)                   |
| Page 4 – Technical Methodology                                       |
| └─ Model descriptions (1 paragraph each)                             |
| └─ Preprocessing steps                                               |
| └─ Fusion algorithm description                                      |
| └─ Confidence interpretation guide                                   |
| Page 5 – Limitations & Disclaimer                                    |
| └─ Cross-domain generalization warnings                              |
| └─ Known failure modes per modality                                  |
| └─ Legal disclaimer (not forensic evidence)                          |
| └─ Platform version + model checksums                                |

7.7.3 Complete Endpoint Contracts

POST /internal/report/generate/{job\_id}

|                                                                                           |
|-------------------------------------------------------------------------------------------|
| Request: (no body – job_id in path)                                                       |
| Processing time: 3–8 seconds (WeasyPrint PDF generation)                                  |
| Response 200:                                                                             |
| {                                                                                         |
| "job_id": "uuid-v4",                                                                      |
| "report_id": "uuid-report",                                                               |
| "status": "COMPLETED",                                                                    |
| "pdf_url": "https://minio/forensics-bucket/reports/.../report.pdf?token=...&expires=...", |
| "pdf_url_expiry": "2024-06-02T10:30:00Z",                                                 |
| "pdf_size_bytes": 487293,                                                                 |
| "page_count": 5,                                                                          |
| "generated_at": "2024-06-01T10:30:08Z"                                                    |
| }                                                                                         |
| Response 404: { "error": "JOB_NOT_FOUND" }                                                |
| Response 422: { "error": "JOB_NOT_COMPLETED", "detail": "Job status is PROCESSING" }      |
| Response 500: { "error": "PDF_GENERATION_FAILED", "detail": "WeasyPrint error: ..." }     |

GET /internal/report/{job\_id}/download

|                                                                                   |
|-----------------------------------------------------------------------------------|
| Response 302 Redirect → MinIO presigned URL (fresh 1h URL generated on each call) |
| Response 404: { "error": "REPORT_NOT_FOUND", "detail": "Generate report first" }  |

7.7.4 Failure Handling & Edge Cases

| Scenario                                  | Strategy                                                      |
|-------------------------------------------|---------------------------------------------------------------|
| MinIO artifact URL expired during PDF gen | Fetch object bytes directly from MinIO → base64-embed in HTML |

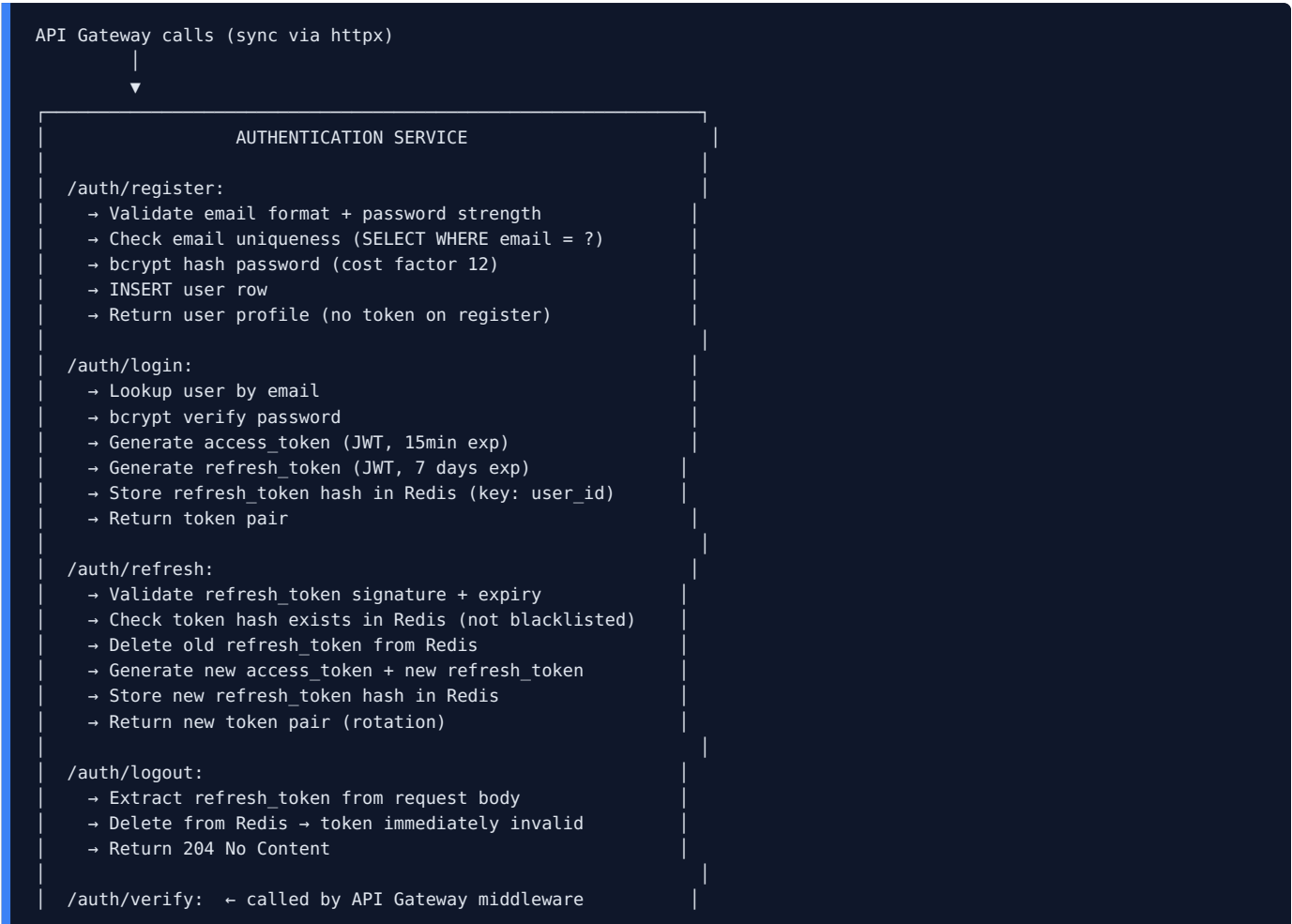
| Scenario                     | Strategy                                                        |
|------------------------------|-----------------------------------------------------------------|
| WeasyPrint font missing      | Dockerfile installs: <code>fonts-liberation fonts-dejavu</code> |
| Job has no image (text-only) | Conditionally exclude image/audio sections from template        |
| PDF > 20MB (large heatmaps)  | Resize embedded images to max 1000px before base64 encoding     |
| Report already exists        | Return existing report metadata without regenerating            |

### 7.7.5 Resource Budget & SLA

| Resource     | Allocated | Note                                      |
|--------------|-----------|-------------------------------------------|
| CPU          | 2 cores   | WeasyPrint is CPU-bound for PDF rendering |
| RAM          | 1 GB      | Jinja2 template + WeasyPrint render tree  |
| P95 Latency  | < 10s     | DOM layout + PDF export                   |
| Max PDF size | 20 MB     | Enforced by image resize before embedding |

## 7.8 Authentication Service (Port 8006)

### 7.8.1 Responsibilities & Internal Architecture



```
| → Validate JWT signature + expiry + not blacklisted |  
| → Return { user_id, email, role } |
```

## 7.8.2 Token Architecture

```
ACCESS TOKEN (JWT HS256):  
Header: { "alg": "HS256", "typ": "JWT" }  
Payload: {  
  "sub": "uuid-user",           // user_id  
  "email": "user@example.com",  
  "role": "USER",               // or "ADMIN"  
  "iat": 1717200000,  
  "exp": 1717200900,           // 15 minutes  
  "jti": "uuid-token-id"       // unique token ID for blacklisting  
}  
Signed with: JWT_SECRET_KEY (env var, min 32 chars)  
  
REFRESH TOKEN (JWT HS256):  
Payload: {  
  "sub": "uuid-user",  
  "type": "refresh",  
  "iat": 1717200000,  
  "exp": 1717804800,           // 7 days  
  "jti": "uuid-refresh-id"  
}  
  
Redis Storage:  
Key: "refresh:{user_id}"  
Value: SHA256(refresh_token_jti)  
TTL: 604800 seconds (7 days)  
  
Blacklisting (logout / token compromise):  
Key: "blacklist:{jti}"  
Value: "1"  
TTL: Remaining token lifetime seconds
```

## 7.8.3 Complete Endpoint Contracts

### POST /auth/register

```
Request:  
{  
  "email": "user@example.com", // REQUIRED. RFC 5321 valid.  
  "password": "Str0ng!Pass",   // Min 8 chars, 1 upper, 1 digit, 1 special.  
  "full_name": "Jane Doe"      // REQUIRED. Max 100 chars.  
}  
  
Response 201 Created:  
{  
  "user_id": "uuid-v4",  
  "email": "user@example.com",  
  "full_name": "Jane Doe",  
  "role": "USER",  
  "created_at": "2024-06-01T10:30:00Z"  
}  
  
Errors:  
400 { "error": "WEAK_PASSWORD", "detail": "Must contain uppercase, digit, special char" }  
409 { "error": "EMAIL_EXISTS", "detail": "user@example.com is already registered" }  
422 { "error": "INVALID_EMAIL", "detail": "Not a valid email address" }
```

### POST /auth/login

```
Request:  
{  
  "email": "user@example.com",  
  "password": "Str0ng!Pass"  
}
```

Response 200:

```
{
  "access_token": "eyJhbGc...",
  "refresh_token": "eyJhbGc...",
  "token_type": "Bearer",
  "access_token_expires_in": 900,
  "refresh_token_expires_in": 604800,
  "user": {
    "user_id": "uuid-v4",
    "email": "user@example.com",
    "full_name": "Jane Doe",
    "role": "USER"
  }
}
```

Errors:

```
401 { "error": "INVALID_CREDENTIALS", "detail": "Email or password incorrect" }
423 { "error": "ACCOUNT_LOCKED", "detail": "Too many failed attempts. Try in 15 min." }
```

#### POST /auth/refresh

Request:

```
{
  "refresh_token": "eyJhbGc..."
}
```

Response 200: (same as login response – new token pair)

Errors:

```
401 { "error": "REFRESH_TOKEN_INVALID", "detail": "Signature mismatch or expired" }
401 { "error": "REFRESH_TOKEN_REVOKED", "detail": "Token already used or logged out" }
```

#### POST /auth/verify (internal — called by API Gateway middleware)

Request:

```
{
  "access_token": "eyJhbGc..."
}
```

Response 200:

```
{
  "valid": true,
  "user_id": "uuid-v4",
  "email": "user@example.com",
  "role": "USER",
  "expires_at": "2024-06-01T10:45:00Z"
}
```

Response 401:

```
{
  "valid": false,
  "error": "TOKEN_EXPIRED", // or TOKEN_BLACKLISTED | SIGNATURE_INVALID
  "detail": "Token expired at 2024-06-01T10:30:00Z"
}
```

#### GET /auth/me

Auth: Bearer <access\_token>

Response 200:

```
{
  "user_id": "uuid-v4",
  "email": "user@example.com",
  "full_name": "Jane Doe",
  "role": "USER",
  "created_at": "2024-05-01T08:00:00Z",
  "last_login": "2024-06-01T10:30:00Z",
}
```

```
"total_jobs": 47
}
```

## 7.8.4 Brute Force Protection

```
# services/auth-service/app/services/login_throttle.py
# Redis-backed failed attempt counter per email

MAX_ATTEMPTS = 5
LOCKOUT_SEC = 900 # 15 minutes

async def check_login_throttle(redis: Redis, email: str) -> None:
    key = f"login_attempts:{email.lower()}"
    attempts = await redis.get(key)
    if attempts and int(attempts) >= MAX_ATTEMPTS:
        ttl = await redis.ttl(key)
        raise AccountLockedError(f"Too many failed attempts. Try in {ttl}s.")

async def record_failed_attempt(redis: Redis, email: str) -> None:
    key = f"login_attempts:{email.lower()}"
    await redis.incr(key)
    await redis.expire(key, LOCKOUT_SEC)

async def reset_throttle(redis: Redis, email: str) -> None:
    await redis.delete(f"login_attempts:{email.lower()}")
```

## 7.8.5 Resource Budget & SLA

| Resource      | Allocated       | Note                                                       |
|---------------|-----------------|------------------------------------------------------------|
| CPU           | 1 core          | bcrypt is intentionally slow (cost=12 = ~300ms per verify) |
| RAM           | 256 MB          | No ML models; pure JWT + bcrypt                            |
| Redis         | < 10KB per user | Refresh token hashes + blacklist entries                   |
| P95 /login    | < 500ms         | bcrypt dominates (300ms) + DB lookup (< 5ms)               |
| P95 /verify   | < 20ms          | JWT decode only; no bcrypt                                 |
| P95 /register | < 600ms         | bcrypt + DB insert                                         |

## 7.9 Cross-Service Health Check Protocol

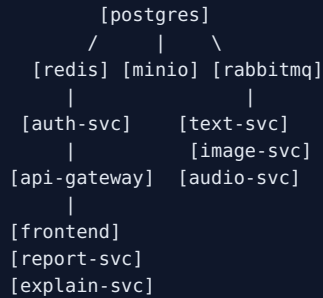
All 8 services implement an identical health check contract:

```
GET /internal/health → 200 OK (healthy) or 503 (degraded)

Mandatory fields in every service health response:
{
  "service": "service-name",
  "status": "healthy" | "degraded" | "starting",
  "version": "1.0.0",
  "timestamp": "ISO-8601",
  "uptime_s": 3600,
  "requests_served": 1247,
  "avg_latency_ms": 380,
  "dependencies": {
    // Each downstream dependency listed with status + latency
  }
}
```

```
Docker Compose healthcheck:
  test: curl -f http://localhost:{port}/internal/health || exit 1
  interval: 30s
  timeout: 10s
  start_period: 60s # Allow model loading time
  retries: 3
```

## 7.10 Service Dependency Graph & Startup Order



Docker Compose `depends\_on` with `condition: service\_healthy`:

```

api-gateway depends_on: [postgres✓, redis✓, rabbitmq✓, minio✓]
text-service depends_on: [postgres✓]
image-service depends_on: [postgres✓, rabbitmq✓]
audio-service depends_on: [postgres✓, rabbitmq✓]
auth-service depends_on: [postgres✓, redis✓]
report-service depends_on: [postgres✓, minio✓]
explain-service depends_on: [minio✓]
frontend depends_on: [api-gateway✓]
```

## 7.11 Service SLA Summary Table

| Service         | P95 Latency                     | Throughput  | Availability | Scaling Strategy           |
|-----------------|---------------------------------|-------------|--------------|----------------------------|
| API Gateway     | < 200ms (gateway overhead)      | 50 req/s    | 99.9%        | Horizontal (stateless)     |
| Text Service    | < 2s (GPU) / < 5s (CPU)         | 2 req/s     | 99.5%        | Vertical (more VRAM)       |
| Image Service   | < 3s (GPU) / < 10s (CPU)        | 5 jobs/min  | 99.0%        | Vertical GPU + queue       |
| Audio Service   | < 500ms (inference)             | 10 jobs/min | 99.0%        | Horizontal consumer        |
| Auth Service    | < 500ms (/login)                | 20 req/s    | 99.9%        | Horizontal (Redis session) |
| Explain Service | < 5s (SHAP) / < 100ms (GradCAM) | 3 req/s     | 99.0%        | Vertical (RAM)             |
| Report Service  | < 10s                           | 5 req/min   | 99.0%        | Horizontal (stateless)     |
| Video Service   | < 30s                           | 2 jobs/min  | 95.0%        | Vertical GPU (Phase 4)     |

## SECTION 8 — LOW LEVEL DESIGN (LLD)

**Scope:** This section is implementation-ready. Every code block, schema, contract, and interface defined here should translate directly to source files with minimal adaptation.

## 8.1 Microservice Folder Structure

Each service follows an identical internal structure. This enforces consistency across the team and makes onboarding new developers trivial.

### 8.1.1 Repository Root

```
forensics-platform/
├── docker-compose.yml           ← Single-command orchestration
├── docker-compose.override.yml  ← Dev overrides (volume mounts, debug ports)
├── .env.example                 ← All required environment variables documented
├── .env                         ← Local secrets (gitignored)
├── Makefile                     ← make up / make down / make migrate / make test
├── README.md
├── services/
│   ├── api-gateway/
│   ├── text-service/
│   ├── image-service/
│   ├── audio-service/
│   ├── explainability-service/
│   ├── report-service/
│   └── auth-service/
├── frontend/                   ← Next.js application
│   ├── src/
│   │   ├── app/                ← Next.js 14 App Router
│   │   │   ├── page.tsx        ← Landing / Dashboard
│   │   │   ├── upload/page.tsx
│   │   │   ├── jobs/[id]/page.tsx
│   │   │   └── reports/[id]/page.tsx
│   │   ├── components/
│   │   │   ├── UploadDropzone.tsx
│   │   │   ├── JobStatusCard.tsx
│   │   │   ├── ResultVisualization.tsx
│   │   │   ├── HeatmapViewer.tsx
│   │   │   └── ReportDownloadButton.tsx
│   │   ├── lib/
│   │   │   ├── api.ts          ← Typed axios API client
│   │   │   └── auth.ts         ← NextAuth configuration
│   │   └── types/
│   │       └── api.ts          ← TypeScript interfaces matching Pydantic schemas
│   ├── public/
│   ├── next.config.ts
│   └── package.json
├── shared/
│   ├── db/
│   │   ├── alembic.ini
│   │   ├── alembic/
│   │   │   ├── env.py
│   │   │   └── versions/
│   │   │       ├── 001_create_users.py
│   │   │       ├── 002_create_jobs.py
│   │   │       ├── 003_create_models.py
│   │   │       ├── 004_create_detection_results.py
│   │   │       ├── 005_create_explanations.py
│   │   │       ├── 006_create_fusion_results.py
│   │   │       ├── 007_create_reports.py
│   │   │       └── 008_create_audit_logs.py
│   │   └── models/             ← Shared SQLAlchemy ORM models (pip-installed package)
│   │       ├── __init__.py
│   │       ├── base.py
│   │       ├── user.py
│   │       ├── job.py
│   │       ├── detection_result.py
│   │       ├── explanation.py
│   │       ├── fusion_result.py
│   │       ├── model_registry.py
│   │       └── report.py
```

```

└─ audit_log.py
└─ schemas/                               ← Shared Pydantic DTOs
  └─ __init__.py
  └─ job_schemas.py
  └─ detection_schemas.py
  └─ queue_schemas.py
  └─ report_schemas.py

└─ models/                                ← Model checkpoints (gitignored, mounted as Docker volume)
  └─ deberta_v3_raid_v1.2/
  └─ resnet50_cnn_detection_v2.0/
  └─ corvi_effnet_v1.0/
  └─ aasist_l_v1.0/

└─ .github/
  └─ workflows/
    └─ ci.yml                             ← Lint + test on push
    └─ cd.yml                             ← Build + push Docker images on merge to main

```

### 8.1.2 Per-Service Folder Structure (Identical Pattern)

Shown for `text-service/`. All other services mirror this structure exactly.

```

services/text-service/
└─ Dockerfile
└─ requirements.txt
└─ .env.example

└─ app/
  └─ main.py                               ← FastAPI app factory + lifespan events
  └─ api/
    └─ __init__.py
    └─ routes.py                           ← Route registration
    └─ v1/
      └─ __init__.py
      └─ detect.py                         ← Endpoint handlers
  └─ core/
    └─ __init__.py
    └─ config.py                           ← Pydantic BaseSettings (reads .env)
    └─ logging.py                          ← Structured JSON logging (structlog)
    └─ exceptions.py                       ← Custom exception classes + handlers
  └─ db/
    └─ __init__.py
    └─ session.py                          ← SQLAlchemy async engine + session factory
    └─ repositories/
      └─ __init__.py
      └─ job_repository.py                 ← DB read/write methods (no raw SQL in routes)
  └─ services/
    └─ __init__.py
    └─ text_detector.py                    ← DeBERTa inference wrapper
    └─ shap_explainer.py                   ← SHAP computation
    └─ queue_consumer.py                   ← RabbitMQ consumer (for async mode)
  └─ models/
    └─ __init__.py
    └─ deberta_loader.py                   ← Model load at startup, singleton pattern
  └─ schemas/
    └─ __init__.py
    └─ request.py                          ← Pydantic request DTOs
    └─ response.py                         ← Pydantic response DTOs

└─ tests/
  └─ __init__.py
  └─ conftest.py                           ← pytest fixtures (test DB, mock model)

```

```

├── unit/
│   ├── test_detector.py
│   └── test_explainer.py
└── integration/
    └── test_api.py

```

### 8.1.3 API Gateway Specific Structure

```

services/api-gateway/
├── app/
│   ├── main.py
│   └── api/v1/
│       ├── auth.py           ← /auth/* endpoints
│       ├── detect.py         ← /detect/* endpoints
│       ├── jobs.py           ← /jobs/* endpoints
│       └── reports.py        ← /reports/* endpoints
│   ├── core/
│       ├── config.py
│       ├── security.py       ← JWT decode, API key validation
│       └── rate_limit.py     ← slowapi configuration
│   ├── middleware/
│       ├── auth_middleware.py ← JWT extraction + user injection
│       ├── file_validator.py  ← MIME type + size checks
│       └── audit_logger.py    ← Writes to audit_logs table
│   └── services/
│       ├── storage_service.py ← MinIO client wrapper
│       ├── queue_service.py   ← RabbitMQ publish wrapper
│       └── job_service.py     ← Job CRUD wrapper

```

## 8.2 PostgreSQL Database Schema (Complete DDL)

```

-- =====
-- EXTENSIONS
-- =====
CREATE EXTENSION IF NOT EXISTS "pgcrypto"; -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS "pg_trgm"; -- Trigram search (future: fuzzy match on filenames)

-- =====
-- ENUM TYPES
-- =====
CREATE TYPE job_status AS ENUM (
    'queued', 'processing', 'completed', 'failed', 'cancelled'
);

CREATE TYPE modality_type AS ENUM (
    'text', 'image', 'audio', 'video', 'multimodal'
);

CREATE TYPE verdict_type AS ENUM (
    'AI_GENERATED', -- >= 0.80 probability
    'LIKELY_AI', -- 0.60-0.79
    'INCONCLUSIVE', -- 0.40-0.59
    'LIKELY_HUMAN', -- 0.20-0.39
    'HUMAN_GENERATED' -- < 0.20
);

CREATE TYPE explanation_method AS ENUM (
    'SHAP', 'GRADCAM', 'LIME', 'SPECTROGRAM', 'FRAME_VISUALIZATION', 'NARRATIVE'
);

CREATE TYPE artifact_type AS ENUM (
    'json_data', 'image_png', 'image_svg', 'html_fragment', 'text_narrative'
);

CREATE TYPE report_status AS ENUM ('pending', 'generating', 'completed', 'failed');
CREATE TYPE report_type AS ENUM ('pdf', 'json', 'html');

```

```

-- =====
-- TABLE: users
-- =====
CREATE TABLE users (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  email             VARCHAR(255)        NOT NULL,
  name              VARCHAR(255)        NOT NULL,
  password_hash     VARCHAR(255)        NOT NULL,
  role              VARCHAR(20)         NOT NULL DEFAULT 'user'
                                CHECK (role IN ('user', 'admin', 'analyst')),
  api_key_hash      VARCHAR(255)        UNIQUE,
  is_active         BOOLEAN             NOT NULL DEFAULT TRUE,
  email_verified    BOOLEAN             NOT NULL DEFAULT FALSE,
  quota_daily_limit INTEGER             NOT NULL DEFAULT 100,
  quota_used_today  INTEGER             NOT NULL DEFAULT 0,
  quota_reset_date  DATE                NOT NULL DEFAULT CURRENT_DATE,
  login_count       INTEGER             NOT NULL DEFAULT 0,
  last_login        TIMESTAMPTZ,
  created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_users_email_lower ON users (LOWER(email));
CREATE INDEX idx_users_api_key ON users (api_key_hash)
  WHERE api_key_hash IS NOT NULL;

-- =====
-- TABLE: jobs
-- =====
CREATE TABLE jobs (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID                NOT NULL
                                REFERENCES users(id) ON DELETE CASCADE,
  status            job_status          NOT NULL DEFAULT 'queued',
  modality           modality_type      NOT NULL,

  -- Input storage
  input_type         VARCHAR(10)        NOT NULL
                                CHECK (input_type IN ('text', 'file')),
  input_text         TEXT,
  input_object_key   VARCHAR(1024),
  file_name          VARCHAR(512),
  file_size_bytes    BIGINT
                                CHECK (file_size_bytes > 0),
  file_mime_type     VARCHAR(128),
  file_sha256        CHAR(64),

  -- Worker coordination
  priority           SMALLINT           NOT NULL DEFAULT 5
                                CHECK (priority BETWEEN 1 AND 10),
  retry_count        SMALLINT           NOT NULL DEFAULT 0,
  max_retries        SMALLINT           NOT NULL DEFAULT 3,
  queue_name         VARCHAR(64),
  worker_id          VARCHAR(128),
  error_message      TEXT,
  error_code         VARCHAR(64),

  -- Lifecycle timestamps
  queued_at          TIMESTAMPTZ,
  started_at         TIMESTAMPTZ,
  completed_at       TIMESTAMPTZ,
  created_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  deleted_at         TIMESTAMPTZ,

  CONSTRAINT chk_input CHECK (
    (input_type = 'text' AND input_text IS NOT NULL AND input_object_key IS NULL) OR
    (input_type = 'file' AND input_object_key IS NOT NULL AND input_text IS NULL)
  )
);

CREATE INDEX idx_jobs_user_id      ON jobs (user_id);
CREATE INDEX idx_jobs_status      ON jobs (status)
  WHERE deleted_at IS NULL;

```

```

CREATE INDEX idx_jobs_user_status      ON jobs (user_id, status, created_at DESC)
    WHERE deleted_at IS NULL;
CREATE INDEX idx_jobs_queue_poll       ON jobs (priority DESC, created_at ASC)
    WHERE status = 'queued' AND deleted_at IS NULL;
CREATE INDEX idx_jobs_worker           ON jobs (worker_id)
    WHERE status = 'processing';

-- =====
-- TABLE: models (Model Registry)
-- =====
CREATE TABLE models (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    name              VARCHAR(255)        NOT NULL,
    version            VARCHAR(64)        NOT NULL,
    modality           modality_type      NOT NULL,
    architecture       VARCHAR(255),
    parameter_count    BIGINT,
    framework          VARCHAR(64)        DEFAULT 'pytorch',
    framework_version  VARCHAR(32),
    training_dataset   VARCHAR(255),
    eval_accuracy      REAL               CHECK (eval_accuracy BETWEEN 0 AND 1),
    eval_f1            REAL               CHECK (eval_f1 BETWEEN 0 AND 1),
    eval_auc           REAL               CHECK (eval_auc BETWEEN 0 AND 1),
    eval_eer           REAL               CHECK (eval_eer >= 0),
    eval_dataset       VARCHAR(255),
    checkpoint_path    VARCHAR(1024),
    checkpoint_sha256  CHAR(64),
    is_active          BOOLEAN            NOT NULL DEFAULT TRUE,
    is_default         BOOLEAN            NOT NULL DEFAULT FALSE,
    loaded_at          TIMESTAMPTZ,
    deprecated_at      TIMESTAMPTZ,
    replaced_by        UUID              REFERENCES models(id),
    created_at         TIMESTAMPTZ       NOT NULL DEFAULT NOW(),

    UNIQUE (name, version)
);

CREATE INDEX idx_models_modality_default ON models (modality, is_active, is_default);

-- =====
-- TABLE: detection_results
-- =====
CREATE TABLE detection_results (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id            UUID                NOT NULL
        REFERENCES jobs(id) ON DELETE CASCADE,
    modality           modality_type      NOT NULL,
    ai_probability     REAL               CHECK (ai_probability BETWEEN 0.0 AND 1.0),
    confidence_score   REAL               NOT NULL
        CHECK (confidence_score BETWEEN 0.0 AND 1.0),
    verdict            verdict_type       NOT NULL,
    model_id           UUID              REFERENCES models(id),
    model_name         VARCHAR(255),
    model_version      VARCHAR(64),
    model_ensemble     JSONB,
    raw_logits         JSONB,
    raw_probabilities  JSONB,
    preprocessing_params JSONB,
    processing_time_ms INTEGER           CHECK (processing_time_ms >= 0),
    device_used        VARCHAR(10)
        CHECK (device_used IN ('cpu', 'cuda', 'mps')),
    created_at         TIMESTAMPTZ       NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_results_job_modality ON detection_results (job_id, modality);
CREATE INDEX idx_results_job_id              ON detection_results (job_id);
CREATE INDEX idx_results_verdict             ON detection_results (verdict);
CREATE INDEX idx_results_created_at          ON detection_results (created_at DESC);

-- =====
-- TABLE: explanations
-- =====
CREATE TABLE explanations (

```

```

    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    result_id         UUID                NOT NULL
                                REFERENCES detection_results(id) ON DELETE CASCADE,
    method            explanation_method NOT NULL,
    artifact_type     artifact_type      NOT NULL,
    artifact_object_key VARCHAR(1024),
    artifact_signed_url TEXT,
    artifact_url_expires TIMESTAMPTZ,
    artifact_data      JSONB,
    narrative_text     TEXT,
    token_attributions JSONB,
    top_indicators     JSONB,
    region_highlights  JSONB,
    frame_scores       JSONB,
    created_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_explanations_result_id ON explanations (result_id);
CREATE INDEX idx_explanations_method   ON explanations (method);

-- =====
-- TABLE: fusion_results
-- =====
CREATE TABLE fusion_results (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id            UUID                NOT NULL UNIQUE
                                REFERENCES jobs(id) ON DELETE CASCADE,
    final_score       REAL                NOT NULL
                                CHECK (final_score BETWEEN 0.0 AND 1.0),
    final_confidence  REAL                NOT NULL
                                CHECK (final_confidence BETWEEN 0.0 AND 1.0),
    final_verdict     verdict_type       NOT NULL,
    modalities_used   JSONB              NOT NULL,
    weights_applied   JSONB              NOT NULL,
    component_scores  JSONB              NOT NULL,
    fusion_method     VARCHAR(64)        NOT NULL DEFAULT 'weighted_linear',
    fusion_version     VARCHAR(32)        NOT NULL DEFAULT 'v1.0',
    created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_fusion_job_id ON fusion_results (job_id);

-- =====
-- TABLE: reports
-- =====
CREATE TABLE reports (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id            UUID                NOT NULL
                                REFERENCES jobs(id) ON DELETE CASCADE,
    report_type       report_type        NOT NULL,
    status            report_status       NOT NULL DEFAULT 'pending',
    minio_object_key  VARCHAR(1024),
    file_size_bytes   BIGINT,
    file_sha256       CHAR(64),
    signed_url        TEXT,
    signed_url_expires TIMESTAMPTZ,
    generation_time_ms INTEGER,
    error_message     TEXT,
    generated_at      TIMESTAMPTZ,
    created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),

    UNIQUE (job_id, report_type)
);

CREATE INDEX idx_reports_job_id ON reports (job_id);

-- =====
-- TABLE: audit_logs (High-volume: BIGSERIAL PK, no UUID)
-- =====
CREATE TABLE audit_logs (
    id                BIGSERIAL          PRIMARY KEY,
    user_id           UUID               REFERENCES users(id) ON DELETE SET NULL,
    event_type        VARCHAR(64)        NOT NULL,
    resource_type     VARCHAR(64),

```

```

resource_id    UUID,
ip_address     INET          NOT NULL,
user_agent     TEXT,
request_method VARCHAR(10),
request_path   VARCHAR(2048),
response_status SMALLINT,
response_time_ms INTEGER,
extra_data     JSONB,
created_at     TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_audit_user_id    ON audit_logs (user_id, created_at DESC);
CREATE INDEX idx_audit_event_type ON audit_logs (event_type, created_at DESC);
CREATE INDEX idx_audit_resource  ON audit_logs (resource_type, resource_id)
WHERE resource_id IS NOT NULL;
CREATE INDEX idx_audit_ip        ON audit_logs (ip_address, created_at DESC);

```

### 8.3 ER Diagram (ASCII)



## 8.4 Index Strategy

INDEX DECISION MATRIX

| QUERY PATTERN                     | TABLE          | INDEX TYPE   | REASON      |
|-----------------------------------|----------------|--------------|-------------|
| Login by email (case-insensitive) | users          | Unique/func  | Daily       |
| List user's jobs by status        | jobs           | Composite    | Dashboard   |
| Poll next queued job (worker)     | jobs           | Partial      | Queue       |
| Find active worker jobs           | jobs           | Partial      | Monitor     |
| Result lookup by job+modality     | detect_results | Unique comp. | Prevent dup |
| Time-series analytics             | detect_results | BRIN/BTREE   | Reporting   |
| Audit queries by event type       | audit_logs     | Composite    | Admin       |
| Audit queries by IP (security)    | audit_logs     | Composite    | Security    |
| Model lookup by modality+default  | models         | Composite    | Startup     |

### PARTIAL INDEX RATIONALE:

idx\_jobs\_queue\_poll: WHERE status='queued'  
→ Only queued jobs are polled by workers. Full index wastes 95% of rows.  
→ Partial index is 20× smaller and stays in shared\_buffers.

### JSONB INDEX (Phase 3 only – add when queries require):

```
CREATE INDEX ON detection_results USING GIN(model_ensemble);  
CREATE INDEX ON explanations USING GIN(token_attributions);
```

## 8.5 SQLAlchemy ORM Models (Python)

```
# shared/db/models/base.py  
from sqlalchemy.orm import DeclarativeBase  
from sqlalchemy import MetaData
```

```
NAMING_CONVENTION = {  
    "ix": "ix_%(column_0_label)s",  
    "uq": "uq_%(table_name)s_%(column_0_name)s",  
    "ck": "ck_%(table_name)s_%(constraint_name)s",  
    "fk": "fk_%(table_name)s_%(column_0_name)s_%(referred_table_name)s",  
    "pk": "pk_%(table_name)s",  
}
```

```
class Base(DeclarativeBase):  
    metadata = MetaData(naming_convention=NAMING_CONVENTION)
```

```
# shared/db/models/user.py  
from __future__ import annotations  
import uuid  
from datetime import date, datetime  
from sqlalchemy import String, Boolean, Integer, Date, DateTime  
from sqlalchemy.orm import Mapped, mapped_column, relationship  
from sqlalchemy.dialects.postgresql import UUID  
from .base import Base
```

```
class User(Base):  
    __tablename__ = "users"  
  
    id: Mapped[uuid.UUID] = mapped_column(  
        UUID(as_uuid=True), primary_key=True, default=uuid.uuid4  
    )  
    email: Mapped[str] = mapped_column(String(255), nullable=False)  
    name: Mapped[str] = mapped_column(String(255), nullable=False)  
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)  
    role: Mapped[str] = mapped_column(String(20), nullable=False, default="user")  
    api_key_hash: Mapped[str | None] = mapped_column(String(255), unique=True)  
    is_active: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)  
    email_verified: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
```

```

quota_daily_limit: Mapped[int] = mapped_column(Integer, nullable=False, default=100)
quota_used_today: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
quota_reset_date: Mapped[date] = mapped_column(Date, nullable=False)
login_count: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
last_login: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
created_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True), nullable=False, default=datetime.utcnow
)
updated_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True), nullable=False, default=datetime.utcnow,
    onupdate=datetime.utcnow
)

# Relationships
jobs: Mapped[list["Job"]] = relationship("Job", back_populates="user")
audit_logs: Mapped[list["AuditLog"]] = relationship("AuditLog", back_populates="user")

```

```

# shared/db/models/job.py
from __future__ import annotations
import uuid
from datetime import datetime
from sqlalchemy import String, Text, BigInteger, SmallInteger, DateTime, Enum, CheckConstraint
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.dialects.postgresql import UUID
from .base import Base

class Job(Base):
    __tablename__ = "jobs"
    __table_args__ = (
        CheckConstraint(
            "(input_type = 'text' AND input_text IS NOT NULL) OR "
            "(input_type = 'file' AND input_object_key IS NOT NULL)",
            name="chk_input"
        ),
    )

    id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True), primary_key=True, default=uuid.uuid4
    )
    user_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        nullable=False,
    )
    status: Mapped[str] = mapped_column(
        Enum("queued", "processing", "completed", "failed", "cancelled",
            name="job_status"), nullable=False, default="queued"
    )
    modality: Mapped[str] = mapped_column(
        Enum("text", "image", "audio", "video", "multimodal",
            name="modality_type"), nullable=False
    )
    input_type: Mapped[str] = mapped_column(String(10), nullable=False)
    input_text: Mapped[str | None] = mapped_column(Text)
    input_object_key: Mapped[str | None] = mapped_column(String(1024))
    file_name: Mapped[str | None] = mapped_column(String(512))
    file_size_bytes: Mapped[int | None] = mapped_column(BigInteger)
    file_mime_type: Mapped[str | None] = mapped_column(String(128))
    file_sha256: Mapped[str | None] = mapped_column(String(64))
    priority: Mapped[int] = mapped_column(SmallInteger, nullable=False, default=5)
    retry_count: Mapped[int] = mapped_column(SmallInteger, nullable=False, default=0)
    max_retries: Mapped[int] = mapped_column(SmallInteger, nullable=False, default=3)
    queue_name: Mapped[str | None] = mapped_column(String(64))
    worker_id: Mapped[str | None] = mapped_column(String(128))
    error_message: Mapped[str | None] = mapped_column(Text)
    error_code: Mapped[str | None] = mapped_column(String(64))
    queued_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    started_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    completed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), nullable=False, default=datetime.utcnow
    )
    deleted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))

```

```

# Relationships
user: Mapped["User"] = relationship("User", back_populates="jobs")
detection_results: Mapped[list["DetectionResult"]] = relationship(
    "DetectionResult", back_populates="job", cascade="all", delete-orphan"
)
fusion_result: Mapped["FusionResult | None"] = relationship(
    "FusionResult", back_populates="job", uselist=False, cascade="all", delete-orphan"
)
reports: Mapped[list["Report"]] = relationship(
    "Report", back_populates="job", cascade="all", delete-orphan"
)

```

```

# shared/db/models/detection_result.py
from __future__ import annotations
import uuid
from datetime import datetime
from sqlalchemy import Float, String, Integer, DateTime, Enum
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.dialects.postgresql import UUID, JSONB
from .base import Base

class DetectionResult(Base):
    __tablename__ = "detection_results"

    id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True), primary_key=True, default=uuid.uuid4
    )
    job_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), nullable=False)
    modality: Mapped[str] = mapped_column(
        Enum("text", "image", "audio", "video", "multimodal",
            name="modality_type"), nullable=False
    )
    ai_probability: Mapped[float] = mapped_column(Float, nullable=False)
    confidence_score: Mapped[float] = mapped_column(Float, nullable=False)
    verdict: Mapped[str] = mapped_column(
        Enum("AI_GENERATED", "LIKELY_AI", "INCONCLUSIVE", "LIKELY_HUMAN", "HUMAN_GENERATED",
            name="verdict_type"), nullable=False
    )
    model_id: Mapped[uuid.UUID | None] = mapped_column(UUID(as_uuid=True))
    model_name: Mapped[str | None] = mapped_column(String(255))
    model_version: Mapped[str | None] = mapped_column(String(64))
    model_ensemble: Mapped[dict | None] = mapped_column(JSONB)
    raw_logits: Mapped[dict | None] = mapped_column(JSONB)
    raw_probabilities: Mapped[dict | None] = mapped_column(JSONB)
    preprocessing_params: Mapped[dict | None] = mapped_column(JSONB)
    processing_time_ms: Mapped[int | None] = mapped_column(Integer)
    device_used: Mapped[str | None] = mapped_column(String(10))
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), nullable=False, default=datetime.utcnow
    )

    # Relationships
    job: Mapped["Job"] = relationship("Job", back_populates="detection_results")
    explanations: Mapped[list["Explanation"]] = relationship(
        "Explanation", back_populates="result", cascade="all", delete-orphan"
    )

```

```

# shared/db/models/explanation.py
from __future__ import annotations
import uuid
from datetime import datetime
from sqlalchemy import String, Text, DateTime, Enum
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.dialects.postgresql import UUID, JSONB
from .base import Base

class Explanation(Base):
    __tablename__ = "explanations"

```

```

id: Mapped[uuid.UUID] = mapped_column(
    UUID(as_uuid=True), primary_key=True, default=uuid.uuid4
)
result_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), nullable=False)
method: Mapped[str] = mapped_column(
    Enum("SHAP", "GRADCAM", "LIME", "SPECTROGRAM", "FRAME_VISUALIZATION", "NARRATIVE",
        name="explanation_method"), nullable=False
)
artifact_type: Mapped[str] = mapped_column(
    Enum("json_data", "image_png", "image_svg", "html_fragment", "text_narrative",
        name="artifact_type"), nullable=False
)
artifact_object_key: Mapped[str | None] = mapped_column(String(1024))
artifact_signed_url: Mapped[str | None] = mapped_column(Text)
artifact_url_expires: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
artifact_data: Mapped[dict | None] = mapped_column(JSONB)
narrative_text: Mapped[str | None] = mapped_column(Text)
token_attributions: Mapped[list | None] = mapped_column(JSONB)
top_indicators: Mapped[list | None] = mapped_column(JSONB)
region_highlights: Mapped[list | None] = mapped_column(JSONB)
frame_scores: Mapped[list | None] = mapped_column(JSONB)
created_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True), nullable=False, default=datetime.utcnow
)

result: Mapped["DetectionResult"] = relationship(
    "DetectionResult", back_populates="explanations"
)

```

## 8.6 Pydantic DTOs (Request & Response Schemas)

```

# shared/schemas/job_schemas.py
from __future__ import annotations
import uuid
from datetime import datetime
from typing import Literal, Optional
from pydantic import BaseModel, Field, ConfigDict

# — Request DTOs —————

class TextDetectRequest(BaseModel):
    text: str = Field(
        ...,
        min_length=50,
        max_length=10_000,
        description="Plain text to analyse for AI generation",
        examples=["The field of machine learning has seen remarkable growth..."]
    )
    explain: bool = Field(
        default=True,
        description="Whether to compute SHAP token attributions"
    )
    language: str = Field(default="en", pattern="^[a-z]{2}$")

class MultiModalDetectRequest(BaseModel):
    """For multimodal jobs, text is submitted in body; files via separate multipart fields."""
    text: Optional[str] = Field(None, max_length=10_000)
    modalities: list[Literal["text", "image", "audio", "video"]] = Field(
        ..., min_length=1, max_length=4
    )
    explain: bool = True

# — Response DTOs —————

class JobCreatedResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

```

```

    job_id: uuid.UUID
    status: Literal["queued"]
    modality: str
    estimated_completion_seconds: int
    poll_url: str

class TokenAttribution(BaseModel):
    token: str
    attribution: float = Field(ge=-1.0, le=1.0)
    position: int = Field(ge=0)

class TextExplanation(BaseModel):
    method: Literal["SHAP"]
    token_attributions: list[TokenAttribution]
    top_ai_indicator_tokens: list[str]
    html_heatmap: Optional[str] = None
    truncated: bool = False

class RegionHighlight(BaseModel):
    region: str
    intensity: float = Field(ge=0.0, le=1.0)
    bounding_box: Optional[list[int]] = None # [x, y, w, h]

class ImageExplanation(BaseModel):
    method: Literal["GRADCAM"]
    heatmap_url: Optional[str] = None
    highlighted_regions: list[RegionHighlight]
    artifact_type: str

class AudioExplanation(BaseModel):
    method: Literal["SPECTROGRAM"]
    spectrogram_url: Optional[str] = None
    observations: str

class DetectionResultSchema(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    modality: str
    ai_probability: float = Field(ge=0.0, le=1.0)
    confidence_score: float = Field(ge=0.0, le=1.0)
    verdict: str
    model_name: str
    model_version: str
    model_ensemble: Optional[list[dict]] = None
    processing_time_ms: Optional[int] = None
    explanation: Optional[TextExplanation | ImageExplanation | AudioExplanation] = None

class FusionResultSchema(BaseModel):
    final_score: float = Field(ge=0.0, le=1.0)
    final_confidence: float = Field(ge=0.0, le=1.0)
    final_verdict: str
    modalities_used: list[str]
    weights_applied: dict[str, float]
    component_scores: dict[str, float]

class JobResultResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    job_id: uuid.UUID
    status: str
    modality: str
    created_at: datetime
    completed_at: Optional[datetime] = None
    processing_time_ms: Optional[int] = None

```

```

# Present when status == "completed"
results: Optional[list[DetectionResultSchema]] = None
fusion: Optional[FusionResultSchema] = None
report_url: Optional[str] = None

# Present when status == "failed"
error_code: Optional[str] = None
error_message: Optional[str] = None

class JobListResponse(BaseModel):
    jobs: list[JobResultResponse]
    total: int
    page: int
    per_page: int
    has_next: bool

```

```

# shared/schemas/auth_schemas.py
from __future__ import annotations
import uuid
from pydantic import BaseModel, EmailStr, Field, model_validator

class RegisterRequest(BaseModel):
    email: EmailStr
    name: str = Field(min_length=2, max_length=100)
    password: str = Field(min_length=8, max_length=128)
    confirm_password: str

    @model_validator(mode="after")
    def passwords_match(self) -> "RegisterRequest":
        if self.password != self.confirm_password:
            raise ValueError("Passwords do not match")
        return self

class LoginRequest(BaseModel):
    email: EmailStr
    password: str

class TokenResponse(BaseModel):
    access_token: str
    refresh_token: str
    token_type: Literal["bearer"] = "bearer"
    expires_in: int # seconds

class RefreshRequest(BaseModel):
    refresh_token: str

class UserProfileResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: uuid.UUID
    email: str
    name: str
    role: str
    quota_daily_limit: int
    quota_used_today: int
    created_at: datetime

```

## 8.7 RabbitMQ Message Contracts

All messages are JSON-serialized and published with `content_type: application/json`. Each queue uses `durable=True` and `delivery_mode=2` (persistent).

### 8.7.1 Queue Definitions

| Queue Name                | Routing Key          | DLX (Dead Letter) |
|---------------------------|----------------------|-------------------|
| forensics.text.detect     | detect.text          | forensics.dlx     |
| forensics.image.detect    | detect.image         | forensics.dlx     |
| forensics.audio.detect    | detect.audio         | forensics.dlx     |
| forensics.video.detect    | detect.video         | forensics.dlx     |
| forensics.report.generate | report.generate      | forensics.dlx     |
| forensics.dlx             | # (all dead letters) | -                 |

### 8.7.2 Detection Job Message Schema

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "DetectionJobMessage",
  "description": "Message published to detection queues by the API Gateway",
  "type": "object",
  "required": ["job_id", "user_id", "modality", "input_type", "created_at", "correlation_id"],
  "properties": {
    "correlation_id": {
      "type": "string",
      "format": "uuid",
      "description": "Unique message ID for idempotency and tracing"
    },
    "job_id": {
      "type": "string",
      "format": "uuid"
    },
    "user_id": {
      "type": "string",
      "format": "uuid"
    },
    "modality": {
      "type": "string",
      "enum": ["text", "image", "audio", "video"]
    },
    "input_type": {
      "type": "string",
      "enum": ["text", "file"]
    },
    "input_text": {
      "type": ["string", "null"],
      "description": "Present only when input_type == 'text'"
    },
    "input_object_key": {
      "type": ["string", "null"],
      "description": "MinIO object key – present when input_type == 'file'"
    },
    "file_mime_type": {
      "type": ["string", "null"]
    },
    "priority": {
      "type": "integer",
      "minimum": 1,
      "maximum": 10,
      "default": 5
    },
    "options": {
      "type": "object",
      "properties": {
        "explain": { "type": "boolean", "default": true },
        "language": { "type": "string", "default": "en" }
      }
    },
    "created_at": {
      "type": "string",
      "format": "date-time"
    }
  }
}
```

```
}
}
```

### 8.7.3 Python Message Producer (API Gateway)

```
# services/api-gateway/app/services/queue_service.py
import json
import uuid
from datetime import datetime, timezone
from typing import Any

import aio_pika
from aio_pika import Message, DeliveryMode

from app.core.config import settings

class QueueService:
    QUEUE_MAP = {
        "text": "forensics.text.detect",
        "image": "forensics.image.detect",
        "audio": "forensics.audio.detect",
        "video": "forensics.video.detect",
    }

    def __init__(self, connection: aio_pika.RobustConnection):
        self._connection = connection
        self._channel: aio_pika.Channel | None = None

    async def _get_channel(self) -> aio_pika.Channel:
        if self._channel is None or self._channel.is_closed:
            self._channel = await self._connection.channel()
            await self._channel.set_qos(prefetch_count=10)
        return self._channel

    async def publish_detection_job(
        self,
        job_id: str,
        user_id: str,
        modality: str,
        input_type: str,
        input_text: str | None = None,
        input_object_key: str | None = None,
        file_mime_type: str | None = None,
        priority: int = 5,
        options: dict[str, Any] | None = None,
    ) -> None:
        channel = await self._get_channel()
        queue_name = self.QUEUE_MAP[modality]

        payload = {
            "correlation_id": str(uuid.uuid4()),
            "job_id": job_id,
            "user_id": user_id,
            "modality": modality,
            "input_type": input_type,
            "input_text": input_text,
            "input_object_key": input_object_key,
            "file_mime_type": file_mime_type,
            "priority": priority,
            "options": options or {"explain": True},
            "created_at": datetime.now(timezone.utc).isoformat(),
        }

        message = Message(
            body=json.dumps(payload).encode(),
            content_type="application/json",
            delivery_mode=DeliveryMode.PERSISTENT,
            message_id=payload["correlation_id"],
            headers={"x-priority": priority},
        )
```

```

        await channel.default_exchange.publish(
            message, routing_key=queue_name
        )

```

### 8.7.4 Python Message Consumer (Detection Service)

```

# services/text-service/app/services/queue_consumer.py
import asyncio
import json
import logging
from typing import Callable, Awaitable

import aio_pika
from aio_pika import IncomingMessage

logger = logging.getLogger(__name__)

class QueueConsumer:
    def __init__(
        self,
        connection: aio_pika.RobustConnection,
        queue_name: str,
        handler: Callable[[dict], Awaitable[None]],
    ):
        self._connection = connection
        self._queue_name = queue_name
        self._handler = handler

    async def start(self) -> None:
        channel = await self._connection.channel()
        await channel.set_qos(prefetch_count=1) # Process one job at a time

        queue = await channel.declare_queue(
            self._queue_name,
            durable=True,
            arguments={
                "x-dead-letter-exchange": "forensics.dlx",
                "x-dead-letter-routing-key": "dlx.failed",
            }
        )

        logger.info(f"Consuming from {self._queue_name}")
        async with queue.iterator() as queue_iter:
            async for message in queue_iter:
                await self._process_message(message)

    async def _process_message(self, message: IncomingMessage) -> None:
        async with message.process(requeue=False):
            try:
                payload = json.loads(message.body.decode())
                logger.info(f"Processing job {payload['job_id']}")
                await self._handler(payload)
            except Exception as exc:
                logger.error(f"Handler failed for message {message.message_id}: {exc}")
                # Message is nacked -> sent to DLX after max_retries
                raise

```

## 8.8 OpenAPI 3.0 Specification (Key Endpoints)

```

# services/api-gateway/openapi.yaml
openapi: "3.0.3"

info:
  title: "Multi-Modal AI Content Forensics Platform API"
  version: "1.0.0"
  description: |
    REST API for detecting AI-generated content across Text, Image, Audio, and Video modalities.

```

```

    All detection endpoints are asynchronous – submit a job and poll for results.
  contact:
    name: "Forensics Platform Team"
    email: "dev@forensics-platform.local"

  servers:
    - url: "http://localhost:8000"
      description: "Local development"
    - url: "https://forensics.example.com"
      description: "Production"

  security:
    - BearerAuth: []

  components:
    securitySchemes:
      BearerAuth:
        type: http
        scheme: bearer
        bearerFormat: JWT

  schemas:
    JobCreatedResponse:
      type: object
      required: [job_id, status, modality, poll_url]
      properties:
        job_id:
          type: string
          format: uuid
          example: "550e8400-e29b-41d4-a716-446655440000"
        status:
          type: string
          enum: [queued]
        modality:
          type: string
          enum: [text, image, audio, video, multimodal]
        estimated_completion_seconds:
          type: integer
          example: 10
        poll_url:
          type: string
          example: "/api/v1/jobs/550e8400-e29b-41d4-a716-446655440000"

    JobResultResponse:
      type: object
      required: [job_id, status, modality, created_at]
      properties:
        job_id:
          type: string
          format: uuid
        status:
          type: string
          enum: [queued, processing, completed, failed, cancelled]
        modality:
          type: string
        created_at:
          type: string
          format: date-time
        completed_at:
          type: string
          format: date-time
          nullable: true
        results:
          type: array
          nullable: true
          items:
            $ref: "#/components/schemas/DetectionResult"
        fusion:
          $ref: "#/components/schemas/FusionResult"
          nullable: true
        report_url:
          type: string
          nullable: true
        error_code:

```

```

    type: string
    nullable: true
  error_message:
    type: string
    nullable: true

DetectionResult:
  type: object
  properties:
    modality:
      type: string
    ai_probability:
      type: number
      format: float
      minimum: 0.0
      maximum: 1.0
    confidence_score:
      type: number
      format: float
      minimum: 0.0
      maximum: 1.0
    verdict:
      type: string
      enum: [AI_GENERATED, LIKELY_AI, INCONCLUSIVE, LIKELY_HUMAN, HUMAN_GENERATED]
    model_name:
      type: string
    explanation:
      oneOf:
        - $ref: "#/components/schemas/TextExplanation"
        - $ref: "#/components/schemas/ImageExplanation"
        - $ref: "#/components/schemas/AudioExplanation"
      nullable: true

FusionResult:
  type: object
  properties:
    final_score:
      type: number
      format: float
    final_confidence:
      type: number
      format: float
    final_verdict:
      type: string
    modalities_used:
      type: array
      items:
        type: string
    weights_applied:
      type: object
      additionalProperties:
        type: number
    component_scores:
      type: object
      additionalProperties:
        type: number

TextExplanation:
  type: object
  properties:
    method:
      type: string
      enum: [SHAP]
    token_attributions:
      type: array
      items:
        type: object
        properties:
          token: { type: string }
          attribution: { type: number }
          position: { type: integer }
    top_ai_indicator_tokens:
      type: array
      items:

```

```

        type: string
    html_heatmap:
        type: string
        nullable: true

ImageExplanation:
  type: object
  properties:
    method:
      type: string
      enum: [GRADCAM]
    heatmap_url:
      type: string
      nullable: true
    highlighted_regions:
      type: array
      items:
        type: object
        properties:
          region: { type: string }
          intensity: { type: number }

AudioExplanation:
  type: object
  properties:
    method:
      type: string
      enum: [SPECTROGRAM]
    spectrogram_url:
      type: string
      nullable: true
    observations:
      type: string

ErrorResponse:
  type: object
  required: [error, detail]
  properties:
    error:
      type: string
      example: "INVALID_MIME_TYPE"
    detail:
      type: string
      allowed_types:
        type: array
        items:
          type: string

paths:
  /api/v1/detect/text:
    post:
      tags: [Detection]
      summary: "Submit text for AI detection"
      operationId: detectText
      security:
        - BearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [text]
              properties:
                text:
                  type: string
                  minLength: 50
                  maxLength: 10000
                explain:
                  type: boolean
                  default: true
      responses:
        "200":
          description: "Detection completed synchronously"

```

```

        content:
          application/json:
            schema:
              $ref: "#/components/schemas/JobResultResponse"
      "400":
        description: "Validation error"
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/ErrorResponse"
      "401":
        description: "Unauthorized"
      "429":
        description: "Rate limit exceeded"
        headers:
          X-RateLimit-Reset:
            schema:
              type: integer

/api/v1/detect/image:
  post:
    tags: [Detection]
    summary: "Submit image for AI detection (async)"
    operationId: detectImage
    security:
      - BearerAuth: []
    requestBody:
      required: true
      content:
        multipart/form-data:
          schema:
            type: object
            required: [file]
            properties:
              file:
                type: string
                format: binary
            description: "JPEG/PNG/WebP, max 10MB"
    responses:
      "202":
        description: "Job queued"
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/JobCreatedResponse"
      "400":
        $ref: "#/components/responses/BadRequest"
      "413":
        description: "File too large"

/api/v1/jobs/{job_id}:
  get:
    tags: [Jobs]
    summary: "Get job status and result"
    operationId: getJob
    security:
      - BearerAuth: []
    parameters:
      - name: job_id
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      "200":
        description: "Job found"
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/JobResultResponse"
      "404":
        description: "Job not found"

```

```

/api/v1/reports/{job_id}/pdf:
  get:
    tags: [Reports]
    summary: "Download PDF forensics report"
    operationId: downloadPdfReport
    security:
      - BearerAuth: []
    parameters:
      - name: job_id
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      "200":
        description: "PDF report"
        content:
          application/pdf:
            schema:
              type: string
              format: binary
      "202":
        description: "Report still generating"
      "404":
        description: "Job not found"

components:
  responses:
    BadRequest:
      description: "Bad request"
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/ErrorResponse"

```

## 8.9 Service Interfaces (Python ABCs & Protocols)

```

# shared/interfaces/detector.py
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Any, Optional

@dataclass
class DetectionOutput:
    """Standardised output for all detection services."""
    ai_probability: float          # 0.0 → 1.0 (1.0 = certainly AI)
    confidence_score: float        # 0.0 → 1.0 (certainty of prediction)
    verdict: str                  # AI_GENERATED | LIKELY_AI | INCONCLUSIVE | etc.
    model_name: str
    model_version: str
    processing_time_ms: int
    device_used: str              # cpu | cuda | mps
    raw_logits: Optional[dict] = None
    model_ensemble: Optional[list[dict]] = None

@dataclass
class ExplanationOutput:
    """Standardised output for explanation modules."""
    method: str                  # SHAP | GRADCAM | SPECTROGRAM
    artifact_type: str           # json_data | image_png | html_fragment
    artifact_data: Optional[dict] = None
    artifact_object_key: Optional[str] = None
    narrative_text: Optional[str] = None
    token_attributions: Optional[list[dict]] = None
    region_highlights: Optional[list[dict]] = None

```

```

class BaseDetector(ABC):
    """Abstract base class all detection services must implement."""

    @abstractmethod
    async def load_model(self) -> None:
        """Load model weights into memory. Called once at startup."""
        ...

    @abstractmethod
    async def preprocess(self, input_data: Any) -> Any:
        """Transform raw input into model-ready tensor/array."""
        ...

    @abstractmethod
    async def predict(self, preprocessed: Any) -> DetectionOutput:
        """Run inference. Must be thread-safe."""
        ...

    @abstractmethod
    async def explain(self, input_data: Any, prediction: DetectionOutput) -> ExplanationOutput:
        """Generate human-interpretable explanation for the prediction."""
        ...

    async def detect(self, input_data: Any, explain: bool = True) -> tuple[DetectionOutput,
Optional[ExplanationOutput]]:
        """Full pipeline: preprocess -> predict -> (optionally) explain."""
        preprocessed = await self.preprocess(input_data)
        prediction = await self.predict(preprocessed)
        explanation = await self.explain(input_data, prediction) if explain else None
        return prediction, explanation

class BaseExplainer(ABC):
    @abstractmethod
    async def generate(self, input_data: Any, model: Any, prediction: DetectionOutput) -> ExplanationOutput:
        ...

class StorageServiceProtocol(ABC):
    @abstractmethod
    async def put_object(self, bucket: str, key: str, data: bytes, content_type: str) -> str:
        """Upload bytes to object storage. Returns the object key."""
        ...

    @abstractmethod
    async def get_object(self, bucket: str, key: str) -> bytes:
        """Download object bytes from storage."""
        ...

    @abstractmethod
    async def presign_get_url(self, bucket: str, key: str, expires_seconds: int) -> str:
        """Generate a time-limited presigned GET URL."""
        ...

    @abstractmethod
    async def delete_object(self, bucket: str, key: str) -> None:
        ...

```

```

# services/text-service/app/services/text_detector.py
import time
import torch
from transformers import DebertaV2ForSequenceClassification, DebertaV2Tokenizer

from shared.interfaces.detector import BaseDetector, DetectionOutput, ExplanationOutput
from app.services.shap_explainer import SHAPTextExplainer
from app.core.config import settings

def _score_to_verdict(score: float) -> str:
    if score >= 0.80: return "AI_GENERATED"
    if score >= 0.60: return "LIKELY_AI"
    if score >= 0.40: return "INCONCLUSIVE"
    if score >= 0.20: return "LIKELY_HUMAN"

```

```

return "HUMAN_GENERATED"

class TextDetector(BaseDetector):
    _model: DebertaV2ForSequenceClassification
    _tokenizer: DebertaV2Tokenizer
    _explainer: SHAPTextExplainer
    _device: torch.device

    async def load_model(self) -> None:
        self._device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self._tokenizer = DebertaV2Tokenizer.from_pretrained(settings.MODEL_PATH)
        self._model = DebertaV2ForSequenceClassification.from_pretrained(
            settings.MODEL_PATH, num_labels=2
        )
        self._model.eval()
        self._model.to(self._device)
        self._explainer = SHAPTextExplainer(self._model, self._tokenizer, self._device)

    async def preprocess(self, input_data: str) -> dict:
        """Tokenize and encode text. Truncate at sentence boundary if needed."""
        return self._tokenizer(
            input_data,
            return_tensors="pt",
            max_length=512,
            truncation=True,
            padding=True,
        )

    async def predict(self, preprocessed: dict) -> DetectionOutput:
        t0 = time.perf_counter()
        inputs = {k: v.to(self._device) for k, v in preprocessed.items()}

        with torch.no_grad():
            outputs = self._model(**inputs)

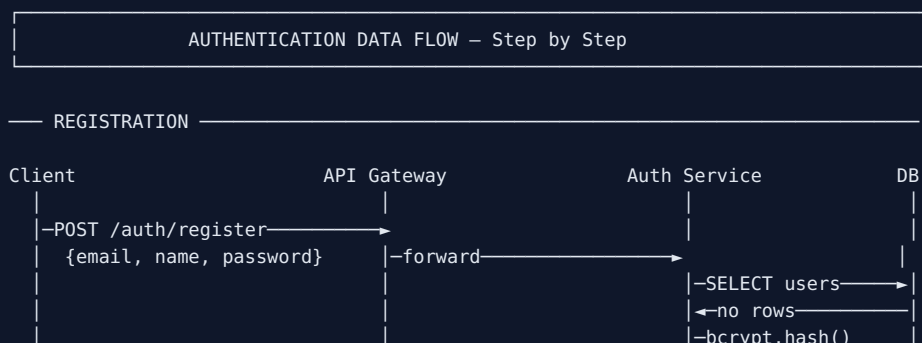
        logits = outputs.logits[0]
        probs = torch.softmax(logits, dim=-1)
        ai_prob = probs[1].item() # Index 1 = AI class
        processing_ms = int((time.perf_counter() - t0) * 1000)

        return DetectionOutput(
            ai_probability=ai_prob,
            confidence_score=float(probs.max().item()),
            verdict=_score_to_verdict(ai_prob),
            model_name="DeBERTa-v3-RAID",
            model_version=settings.MODEL_VERSION,
            processing_time_ms=processing_ms,
            device_used=str(self._device),
            raw_logits={"human": logits[0].item(), "ai": logits[1].item()},
        )

    async def explain(self, input_data: str, prediction: DetectionOutput) -> ExplanationOutput:
        return await self._explainer.generate(input_data, self._model, prediction)

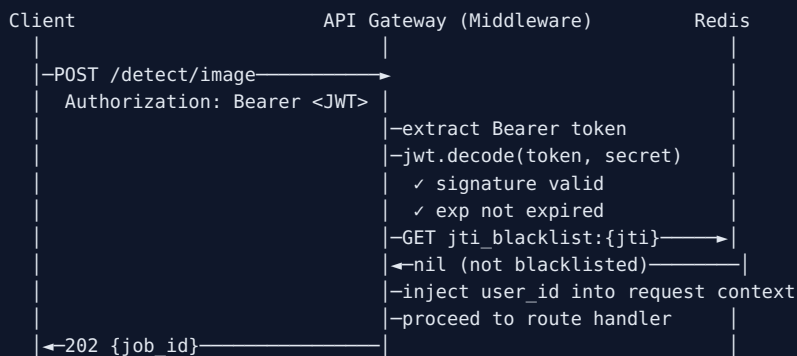
```

## 8.10 Authentication Data Flow





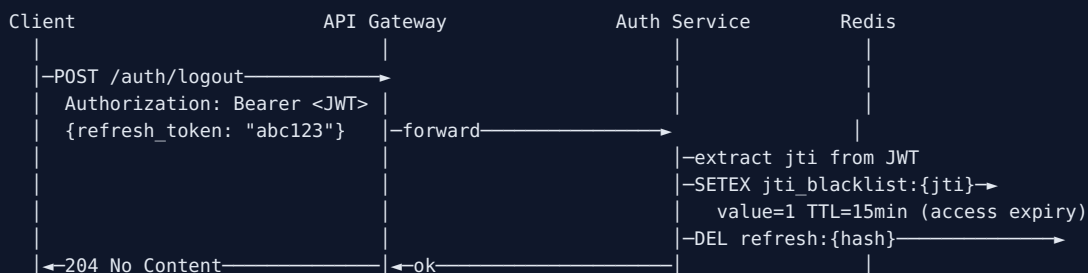
#### AUTHENTICATED REQUEST



#### TOKEN REFRESH



#### LOGOUT



```

# services/auth-service/app/core/security.py
import hashlib
import secrets
  
```

```

import uuid
from datetime import datetime, timedelta, timezone

from jose import JWTError, jwt
from passlib.context import CryptContext

from app.core.config import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_access_token(user_id: str, email: str, role: str) -> tuple[str, str]:
    """Returns (encoded_jwt, jti)."""
    jti = str(uuid.uuid4())
    now = datetime.now(timezone.utc)
    payload = {
        "sub": user_id,
        "email": email,
        "role": role,
        "iat": now,
        "exp": now + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES),
        "jti": jti,
    }
    token = jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm=settings.JWT_ALGORITHM)
    return token, jti

def create_refresh_token() -> tuple[str, str]:
    """Returns (raw_token, hashed_token). Store hash in Redis, send raw to client."""
    raw = secrets.token_urlsafe(64)
    hashed = hashlib.sha256(raw.encode()).hexdigest()
    return raw, hashed

def decode_access_token(token: str) -> dict:
    """Raises JWTError if invalid or expired."""
    return jwt.decode(token, settings.JWT_SECRET_KEY, algorithms=[settings.JWT_ALGORITHM])

```

## 8.11 Fusion Engine — Implementation Design

The Fusion Engine is the component that aggregates per-modality detection scores into a single platform-level verdict. It runs within the API Gateway after all detection results for a multimodal job are complete.

### 8.11.1 Algorithm

```

INPUT:  component_scores = {"text": 0.91, "image": 0.84}
        weights          = {"text": 0.60, "image": 0.40} ← calibrated on val set

STEP 1: Weighted Sum
    P_fusion = Σ(wi × pi) for each modality i
    P_fusion = (0.60 × 0.91) + (0.40 × 0.84)
    P_fusion = 0.546 + 0.336 = 0.882

STEP 2: Confidence Calculation
    Confidence reflects inter-modal agreement.
    score_std = std([0.91, 0.84]) = 0.035
    raw_conf  = 1.0 - (2 × score_std) ← std=0 → conf=1.0; std=0.5 → conf=0.0
    raw_conf  = 1.0 - 0.07 = 0.93
    # Also consider distance from 0.5 (decision boundary)
    boundary_conf = abs(P_fusion - 0.5) × 2 ← 0.0 at boundary; 1.0 at extremes
    boundary_conf = abs(0.882 - 0.5) × 2 = 0.764

```

```

# Final confidence = geometric mean of agreement and boundary separation
final_conf = sqrt(raw_conf × boundary_conf) = sqrt(0.93 × 0.764) = 0.843

STEP 3: Verdict Mapping
P_fusion = 0.882 → "AI_GENERATED" (threshold ≥ 0.80)

OUTPUT: {final_score: 0.882, final_confidence: 0.843, final_verdict: "AI_GENERATED"}

```

### 8.11.2 Default Weight Configuration

```

# Weights are calibrated on a held-out validation set.
# Update these after running calibration (Section 4.4 evaluation plan).
DEFAULT_WEIGHTS = {
    "text": 0.60, # Text is most reliable; lowest FPR on RAID
    "image": 0.40, # Image ensemble is strong but varies by generator
    "audio": 0.35, # AASIST generalizes less well in-the-wild
    "video": 0.25, # Frame-level aggregation is noisiest
}
# Weights are normalized to sum to 1.0 over present modalities.

```

### 8.11.3 Implementation

```

# services/api-gateway/app/services/fusion_engine.py
import math
import statistics
from dataclasses import dataclass
from typing import Optional

VERDICT_THRESHOLDS = [
    (0.80, "AI_GENERATED"),
    (0.60, "LIKELY_AI"),
    (0.40, "INCONCLUSIVE"),
    (0.20, "LIKELY_HUMAN"),
    (0.00, "HUMAN_GENERATED"),
]

DEFAULT_WEIGHTS = {
    "text": 0.60,
    "image": 0.40,
    "audio": 0.35,
    "video": 0.25,
}

@dataclass
class FusionResult:
    final_score: float
    final_confidence: float
    final_verdict: str
    modalities_used: list[str]
    weights_applied: dict[str, float]
    component_scores: dict[str, float]
    fusion_method: str = "weighted_linear"
    fusion_version: str = "v1.0"

class FusionEngine:
    def __init__(self, weight_overrides: Optional[dict[str, float]] = None):
        self._base_weights = {**DEFAULT_WEIGHTS, **(weight_overrides or {})}

    def fuse(self, component_scores: dict[str, float]) -> FusionResult:
        """
        Combine per-modality scores into a single platform verdict.

        Args:
            component_scores: {"text": 0.91, "image": 0.84}

        Returns:
            FusionResult with final score, confidence, and verdict.
        """

```

```

"""
if not component_scores:
    raise ValueError("At least one modality score required for fusion")

# Normalize weights to sum to 1.0 over present modalities only
present = {m: self._base_weights.get(m, 0.5) for m in component_scores}
total_weight = sum(present.values())
normalized = {m: w / total_weight for m, w in present.items()}

# Weighted linear combination
final_score = sum(normalized[m] * component_scores[m] for m in component_scores)
final_score = max(0.0, min(1.0, final_score))

# Confidence: agreement × boundary separation
scores_list = list(component_scores.values())
if len(scores_list) > 1:
    score_std = statistics.stdev(scores_list)
    agreement_conf = max(0.0, 1.0 - (2.0 * score_std))
else:
    agreement_conf = 1.0 # Single modality: fully "agrees with itself"

boundary_conf = abs(final_score - 0.5) * 2.0
final_confidence = math.sqrt(agreement_conf * boundary_conf)
final_confidence = max(0.0, min(1.0, final_confidence))

# Verdict mapping
verdict = "HUMAN_GENERATED"
for threshold, label in VERDICT_THRESHOLDS:
    if final_score >= threshold:
        verdict = label
        break

return FusionResult(
    final_score=round(final_score, 4),
    final_confidence=round(final_confidence, 4),
    final_verdict=verdict,
    modalities_used=list(component_scores.keys()),
    weights_applied=normalized,
    component_scores=component_scores,
)

```

#### 8.11.4 Weight Calibration Script (One-Time)

```

# scripts/calibrate_fusion_weights.py
"""
Run this once after all detection services are trained.
Requires a held-out multimodal validation set.

Usage:
python scripts/calibrate_fusion_weights.py \
    --val-file data/multimodal_val.jsonl \
    --output config/fusion_weights.json
"""
import json
import argparse
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

def calibrate(val_file: str, output: str) -> None:
    # Load validation data
    # Each line: {"text_score": 0.91, "image_score": 0.84, "label": 1}
    with open(val_file) as f:
        records = [json.loads(line) for line in f]

    modalities = ["text", "image", "audio"]
    X, y = [], []
    for r in records:
        row = [r.get(f"{m}_score", np.nan) for m in modalities]
        if not any(np.isnan(row)):
            X.append(row)

```

```

        y.append(r["label"]) # 1 = AI, 0 = Human

X = np.array(X)
y = np.array(y)

# Logistic Regression with no intercept → weights are interpretable as fusion coefficients
lr = LogisticRegression(fit_intercept=False, C=1.0, max_iter=1000)
lr.fit(X, y)

# Normalize coefficients to sum to 1
raw_weights = lr.coef_[0]
raw_weights = np.clip(raw_weights, 0, None) # Non-negative only
normalized = raw_weights / raw_weights.sum()

weights = {m: round(float(w), 4) for m, w in zip(modalities, normalized)}
print(f"Calibrated weights: {weights}")

# Evaluate
scores = lr.predict_proba(X)[:, 1]
auc = roc_auc_score(y, scores)
print(f"Validation AUROC after calibration: {auc:.4f}")

with open(output, "w") as f:
    json.dump({"weights": weights, "val_auc": auc}, f, indent=2)
print(f"Saved to {output}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--val-file", required=True)
    parser.add_argument("--output", default="config/fusion_weights.json")
    args = parser.parse_args()
    calibrate(args.val_file, args.output)

```

## 8.12 Report Generation — Implementation Design

### 8.12.1 Report Generation Pipeline

```

JOB COMPLETED
|
▼
POST /internal/report/generate/{job_id}
|
├─> Fetch job from PostgreSQL
├─> Fetch all detection_results (with explanations)
├─> Fetch fusion_result
├─> For each explanation with artifact_object key:
│   │   getObject(MinIO) → base64-encode → embed inline
├─> Render Jinja2 HTML template
│   │   ├── report_template.html
│   │   ├── page1_executive_summary.html
│   │   ├── page2_modality_results.html
│   │   ├── page3_text_explanation.html
│   │   ├── page4_image_explanation.html
│   │   └── page5_technical_appendix.html
├─> WeasyPrint HTML → PDF bytes
├─> Calculate SHA-256 of PDF bytes
├─> Upload PDF to MinIO (reports/{job_id}.pdf)
├─> Generate presigned URL (valid 24h)
├─> UPDATE reports table (status=completed, minio_key, signed_url)
└─> Return {report_url, signed_url}

```

## 8.12.2 Report Service Implementation

```
# services/report-service/app/services/report_generator.py
import base64
import hashlib
import io
from datetime import datetime, timedelta, timezone
from pathlib import Path

from jinja2 import Environment, FileSystemLoader, select_autoescape
from weasyprint import HTML, CSS

from shared.db.repositories.job_repository import JobRepository
from app.services.storage_service import StorageService
from app.core.config import settings

TEMPLATE_DIR = Path(__file__).parent.parent / "templates"
jinja_env = Environment(
    loader=FileSystemLoader(TEMPLATE_DIR),
    autoescape=select_autoescape(["html", "xml"]),
)

def _verdict_color(verdict: str) -> str:
    return {
        "AI_GENERATED": "#e53e3e",
        "LIKELY_AI": "#dd6b20",
        "INCONCLUSIVE": "#d69e2e",
        "LIKELY_HUMAN": "#38a169",
        "HUMAN_GENERATED": "#2f855a",
    }.get(verdict, "#718096")

def _probability_bar(score: float) -> str:
    """Returns an HTML inline progress bar."""
    pct = int(score * 100)
    color = "#e53e3e" if score >= 0.6 else "#38a169" if score < 0.4 else "#d69e2e"
    return (
        f'<div style="background:#eee;border-radius:4px;height:12px;width:200px">'
        f'<div style="width:{pct}%;background:{color};height:12px;border-radius:4px"></div>'
        f'</div>'
    )

class ReportGenerator:
    def __init__(self, job_repo: JobRepository, storage: StorageService):
        self._job_repo = job_repo
        self._storage = storage

    async def generate_pdf(self, job_id: str) -> bytes:
        # 1. Fetch all data
        job = await self._job_repo.get_job_with_results(job_id)
        if not job:
            raise ValueError(f"Job {job_id} not found")

        # 2. Fetch and embed explanation images
        embedded_images: dict[str, str] = {}
        for result in job.detection_results:
            for explanation in result.explanations:
                if explanation.artifact_object_key:
                    img_bytes = await self._storage.get_object(
                        settings.MINIO_BUCKET, explanation.artifact_object_key
                    )
                    b64 = base64.b64encode(img_bytes).decode()
                    mime = "image/png" if explanation.artifact_type == "image_png" else "image/svg+xml"
                    embedded_images[explanation.artifact_object_key] = f"data:{mime};base64,{b64}"

        # 3. Render template
        template = jinja_env.get_template("report_full.html")
        html_content = template.render(
            job=job,
            generated_at=datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC"),

```

```

        embedded_images=embedded_images,
        verdict_color=_verdict_color,
        probability_bar=_probability_bar,
        platform_name="AI Content Forensics Platform",
        platform_version="v1.0",
    )

    # 4. Convert HTML → PDF
    base_url = str(TEMPLATE_DIR)
    pdf_bytes = HTML(string=html_content, base_url=base_url).write_pdf(
        stylesheets=[CSS(filename=str(TEMPLATE_DIR / "report.css"))]
    )
    return pdf_bytes

async def save_and_register(self, job_id: str, pdf_bytes: bytes) -> str:
    # 5. Calculate checksum
    sha256 = hashlib.sha256(pdf_bytes).hexdigest()

    # 6. Upload to MinIO
    object_key = f"reports/{job_id}.pdf"
    await self._storage.put_object(
        bucket=settings.MINIO_BUCKET,
        key=object_key,
        data=pdf_bytes,
        content_type="application/pdf",
    )

    # 7. Generate presigned URL (24h)
    signed_url = await self._storage.presign_get_url(
        bucket=settings.MINIO_BUCKET,
        key=object_key,
        expires_seconds=86400,
    )

    # 8. Update report record
    await self._job_repo.update_report(
        job_id=job_id,
        report_type="pdf",
        status="completed",
        minio_key=object_key,
        file_size=len(pdf_bytes),
        sha256=sha256,
        signed_url=signed_url,
        signed_url_expires=datetime.now(timezone.utc) + timedelta(hours=24),
    )

    return signed_url

```

### 8.12.3 Jinja2 Report Template Structure

```

services/report-service/app/templates/
├─ report_full.html      ← Master template (extends base.html)
├─ base.html            ← Page layout, header/footer macros
├─ report.css           ← Print-optimized CSS (page breaks, fonts)
├─
├─ partials/
│   ├─ executive_summary.html ← Page 1: Verdict badge, overall score, risk level
│   ├─ modality_results.html  ← Page 2: Per-modality score table + progress bars
│   ├─ text_explanation.html    ← Page 3: Token heatmap + top indicators table
│   ├─ image_explanation.html   ← Page 4: Side-by-side original + GradCAM
│   └─ audio_explanation.html   ← Page 4b: Mel-spectrogram + observations
└─ technical_appendix.html ← Page 5: Model versions, checksums, disclaimer

```

### 8.12.4 HTML Template Snippet (Executive Summary)

```

{# templates/partial/executive_summary.html #}
<div class="page executive-summary">
  <div class="report-header">
    <h1>AI Content Forensics Report</h1>
  <div class="meta-grid">

```

```

<div><strong>Report ID:</strong> {{ job.id }}</div>
<div><strong>Generated:</strong> {{ generated_at }}</div>
<div><strong>Analyst:</strong> {{ job.user.email }}</div>
<div><strong>Platform:</strong> {{ platform_name }} {{ platform_version }}</div>
</div>
</div>

<div class="verdict-banner" style="border-left: 8px solid {{ verdict_color(job.fusion_result.final_verdict) }}">
<div class="verdict-label">OVERALL VERDICT</div>
<div class="verdict-value">{{ job.fusion_result.final_verdict | replace("_", " ") }}</div>
<div class="score-row">
<span>AI Probability: <strong>{{ (job.fusion_result.final_score * 100) | round(1) }}%</strong></span>
<span>Confidence: <strong>{{ (job.fusion_result.final_confidence * 100) | round(1) }}%</strong></span>
</div>
{{ probability_bar(job.fusion_result.final_score) | safe }}
</div>

<div class="content-summary">
<h2>Submitted Content</h2>
<table>
<tr><td>File Name</td><td>{{ job.file_name or "Text Input" }}</td></tr>
<tr><td>Modality</td><td>{{ job.modality | title }}</td></tr>
<tr><td>File Size</td><td>{{ (job.file_size_bytes / 1024) | round(1) }} KB</td></tr>
<tr><td>Submitted At</td><td>{{ job.created_at.strftime("%Y-%m-%d %H:%M UTC") }}</td></tr>
</table>
</div>
</div>

```

## 8.13 Database Seed & Initial Setup

```

# scripts/seed_models.py
"""
Run once after first alembic migrate to register model metadata.
Usage: python scripts/seed_models.py
"""

import asyncio
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.dialects.postgresql import insert as pg_insert
from shared.db.models.model_registry import Model
from app.core.config import settings

MODEL_SEED_DATA = [
    {
        "name": "DeBERTa-v3-RAID",
        "version": "v1.2",
        "modality": "text",
        "architecture": "DeBERTa-v3-base",
        "parameter_count": 86_000_000,
        "framework": "pytorch",
        "training_dataset": "RAID-2024 (100K subset)",
        "eval_f1": 0.89,
        "eval_dataset": "RAID validation split",
        "checkpoint_path": "/models/deberta_v3_raid_v1.2",
        "is_active": True,
        "is_default": True,
    },
    {
        "name": "ResNet50-CNNDetection",
        "version": "v2.0",
        "modality": "image",
        "architecture": "ResNet-50",
        "parameter_count": 25_600_000,
        "framework": "pytorch",
        "training_dataset": "ProGAN-11class",
        "eval_auc": 0.97,
        "eval_dataset": "Wang et al. test set",
        "checkpoint_path": "/models/resnet50_cnn_detection_v2.0",
        "is_active": True,
        "is_default": True,
    },
]

```

```

    },
    {
        "name": "Corvi-EfficientNet",
        "version": "v1.0",
        "modality": "image",
        "architecture": "EfficientNet-B0 (frequency)",
        "parameter_count": 5_288_548,
        "framework": "pytorch",
        "training_dataset": "RAISE + LDM subset",
        "eval_auc": 0.93,
        "eval_dataset": "Corvi et al. test set",
        "checkpoint_path": "/models/corvi_effnet_v1.0",
        "is_active": True,
        "is_default": False,
    },
    {
        "name": "AASIST-L",
        "version": "v1.0",
        "modality": "audio",
        "architecture": "Heterogeneous Graph Attention Network",
        "parameter_count": 297_000,
        "framework": "pytorch",
        "training_dataset": "ASVspoof2019-LA",
        "eval_eer": 0.0083,
        "eval_dataset": "ASVspoof 2019 LA eval",
        "checkpoint_path": "/models/aasist_l_v1.0",
        "is_active": True,
        "is_default": True,
    },
]

async def seed() -> None:
    engine = create_async_engine(settings.DATABASE_URL, echo=True)
    async with AsyncSession(engine) as session:
        for data in MODEL_SEED_DATA:
            stmt = (
                pg_insert(Model)
                .values(**data)
                .on_conflict_do_update(
                    index_elements=["name", "version"],
                    set_={"is_active": data["is_active"]},
                )
            )
            await session.execute(stmt)
        await session.commit()
    await engine.dispose()
    print("  Model seed data inserted successfully")

if __name__ == "__main__":
    asyncio.run(seed())

```

## 8.14 Environment Variables Reference

```

# .env.example – Complete reference for all services

# — Database —————
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=forensics_db
POSTGRES_USER=forensics_user
POSTGRES_PASSWORD=change_me_in_production
DATABASE_URL=postgresql+asyncpg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@${POSTGRES_HOST}:${POSTGRES_PORT}/${POSTGRES_DB}

# — Redis —————
REDIS_URL=redis://redis:6379/0

# — RabbitMQ —————

```

```

RABBITMQ_HOST=rabbitmq
RABBITMQ_PORT=5672
RABBITMQ_USER=forensics
RABBITMQ_PASSWORD=change_me_in_production
RABBITMQ_VHOST=/
RABBITMQ_URL=amqp://${RABBITMQ_USER}:${RABBITMQ_PASSWORD}@${RABBITMQ_HOST}:${RABBITMQ_PORT}/${RABBITMQ_VHOST}

# — MinIO / S3 Object Storage —
MINIO_ENDPOINT=minio:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=change_me_in_production
MINIO_BUCKET=forensics-bucket
MINIO_USE_SSL=false

# — Auth Service —
JWT_SECRET_KEY=super_secret_key_minimum_32_chars_change_this
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7

# — Model Paths (Mounted Docker Volume) —
TEXT_MODEL_PATH=/models/deberta_v3_raid_v1.2
TEXT_MODEL_VERSION=v1.2
IMAGE_MODEL_GAN_PATH=/models/resnet50_cnn_detection_v2.0
IMAGE_MODEL_DIFF_PATH=/models/corvi_effnet_v1.0
AUDIO_MODEL_PATH=/models/aassist_l_v1.0

# — Service URLs (internal Docker network) —
TEXT_SERVICE_URL=http://text-service:8001
IMAGE_SERVICE_URL=http://image-service:8002
AUDIO_SERVICE_URL=http://audio-service:8003
EXPLAINABILITY_SERVICE_URL=http://explainability-service:8004
REPORT_SERVICE_URL=http://report-service:8005
AUTH_SERVICE_URL=http://auth-service:8006

# — Rate Limiting —
RATE_LIMIT_PER_MINUTE=60
RATE_LIMIT_PER_DAY=500

# — File Upload Limits —
MAX_TEXT_LENGTH=10000
MAX_IMAGE_SIZE_MB=10
MAX_AUDIO_SIZE_MB=30
MAX_AUDIO_DURATION_SECONDS=120
MAX_VIDEO_SIZE_MB=300

# — Fusion Weights (overrides calibrated defaults) —
FUSION_WEIGHT_TEXT=0.60
FUSION_WEIGHT_IMAGE=0.40
FUSION_WEIGHT_AUDIO=0.35
FUSION_WEIGHT_VIDEO=0.25

# — Logging —
LOG_LEVEL=INFO
LOG_FORMAT=json          # json | text

# — Frontend —
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_APP_NAME=AI Forensics Platform

```

*End of Sections 5–8 (Full LLD Edition)*

*Next: Sections 9–14 — Implementation Roadmap, Model Integration, Explainability Deep-Dive, Testing Strategy, DevOps, Final Recommendation*

## Section 9-14

---

# Multi-Modal AI Content Forensics Platform

## Software Engineering Blueprint — Sections 9–14

**Document Version:** 1.0  
**Type:** Internal Architecture & Engineering Reference  
**Scope:** Sections 9–14 of 14  
**Target Audience:** Development Team, Faculty Supervisor, External Examiners  
**Continuation of:** Sections 1–8

## SECTION 9 — IMPLEMENTATION ROADMAP

**Format:** 24-week roadmap for a 2-student team. Each week block lists: tasks, deliverables, dependencies, risk, and estimated effort. Phases are cumulative — never skip a phase.

### 9.1 Roadmap Overview

|                              |             |             |
|------------------------------|-------------|-------------|
| PHASE 1: Foundation          | Weeks 01–04 | <div></div> |
| PHASE 2: Text Modality       | Weeks 05–08 | <div></div> |
| PHASE 3: Image Modality      | Weeks 09–12 | <div></div> |
| PHASE 4: Audio Modality      | Weeks 13–16 | <div></div> |
| PHASE 5: Fusion & Reporting  | Weeks 17–20 | <div></div> |
| PHASE 6: Evaluation & Polish | Weeks 21–24 | <div></div> |

### 9.2 Phase 1 — Foundation & Infrastructure (Weeks 1–4)

#### Week 1 — Repository Setup & Docker Scaffolding

| Field           | Detail                                                                                                                                                                                                                       |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Zero-to-running local environment with all containers healthy                                                                                                                                                                |
| Student A Tasks | Create GitHub repo, branch strategy (main/dev/feature/*). Set up Docker Compose with postgres, redis, rabbitmq, minio containers. Write <code>.env.example</code> . Configure <code>Makefile</code> (make up / down / logs). |
| Student B Tasks | Scaffold Next.js 14 frontend (App Router). Create FastAPI gateway skeleton with <code>/health</code> endpoint. Set up GitHub Actions CI for linting only.                                                                    |
| Deliverables    | <code>docker-compose up</code> brings all infra containers healthy. <code>/health</code> returns 200. Next.js shows placeholder page.                                                                                        |
| Dependencies    | Docker Desktop installed. GitHub org created.                                                                                                                                                                                |
| Risk            |                                                                                                                                                                                                                              |

| Field  | Detail                                                                              |
|--------|-------------------------------------------------------------------------------------|
|        | Docker networking conflicts on Windows WSL2 — allocate 4h buffer for port conflicts |
| Effort | Student A: 20h / Student B: 18h                                                     |

## Week 2 — Database Schema & Alembic Migrations

| Field           | Detail                                                                                                                                                                                                              |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | All 8 database tables created via Alembic migrations                                                                                                                                                                |
| Student A Tasks | Write all SQLAlchemy ORM models (Section 8.5). Write Alembic migration files 001–008 in order. Run <code>alembic upgrade head</code> . Verify all indexes created via <code>\d+ table_name</code> in psql.          |
| Student B Tasks | Write shared Pydantic DTOs (Section 8.6). Create <code>shared/</code> Python package with <code>pyproject.toml</code> for editable install. Configure GitHub Actions to run <code>alembic check</code> on every PR. |
| Deliverables    | All 8 tables verified in PostgreSQL. <code>alembic current</code> shows head. CI fails if migrations are not up to date.                                                                                            |
| Dependencies    | Week 1 complete. PostgreSQL container healthy.                                                                                                                                                                      |
| Risk            | Enum type conflicts on re-runs — add <code>IF NOT EXISTS</code> guards                                                                                                                                              |
| Effort          | Student A: 22h / Student B: 16h                                                                                                                                                                                     |

## Week 3 — Auth Service & JWT Flow

| Field           | Detail                                                                                                                                                                                                                                                               |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Complete working authentication flow: register → login → get protected resource                                                                                                                                                                                      |
| Student A Tasks | Build auth-service FastAPI app: <code>/auth/register</code> , <code>/auth/login</code> , <code>/auth/refresh</code> , <code>/auth/logout</code> . Implement bcrypt hashing, JWT generation, refresh token storage in Redis. Write unit tests for all auth functions. |
| Student B Tasks | Integrate auth into API Gateway middleware. Create <code>GET /api/v1/auth/me</code> endpoint. Build Next.js login/register pages with form validation. Implement token storage in <code>httpOnly</code> cookie.                                                      |
| Deliverables    | User can register, log in, receive JWT, and access a protected route. Expired token returns 401. Logout invalidates refresh token. 12 unit tests passing.                                                                                                            |
| Dependencies    | Week 2 complete. Redis container healthy.                                                                                                                                                                                                                            |
| Risk            | CORS configuration for Next.js ↔ FastAPI — allocate 4h                                                                                                                                                                                                               |
| Effort          | Student A: 24h / Student B: 22h                                                                                                                                                                                                                                      |

## Week 4 — MinIO Storage & RabbitMQ Queue Scaffolding

| Field           | Detail                                                                                                                                                                                                                                                                            |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | File upload pipeline working end-to-end. Queue publish/consume cycle verified.                                                                                                                                                                                                    |
| Student A Tasks | Implement <code>StorageService</code> (MinIO upload/download/presign). Implement <code>QueueService</code> (publish to queues). Write <code>POST /api/v1/detect/image</code> endpoint that: validates MIME, uploads to MinIO, creates job in DB, publishes to queue, returns 202. |
| Student B Tasks | Build the Next.js file <code>UploadDropzone</code> component (react-dropzone). Display job ID and status after upload. Implement <code>GET /api/v1/jobs/{id}</code> polling. Build <code>JobStatusCard</code> component that auto-refreshes every 3 seconds.                      |
| Deliverables    | File upload → MinIO object stored → Job row in DB → Message in RabbitMQ queue. Frontend shows job polling. Detection stub consumer logs the message.                                                                                                                              |
| Dependencies    | Weeks 1–3 complete. MinIO and RabbitMQ containers healthy.                                                                                                                                                                                                                        |
| Risk            | <code>python-magic</code> requires libmagic system library in Docker — add to Dockerfile                                                                                                                                                                                          |
| Effort          | Student A: 26h / Student B: 24h                                                                                                                                                                                                                                                   |

## 9.3 Phase 2 — Text Detection Modality (Weeks 5–8)

## Week 5 — RAID Dataset Preparation & DeBERTa Fine-Tuning

| Field           | Detail                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Fine-tuned DeBERTa-v3-base checkpoint ready for integration                                                                                                                                                                                                                                                                                                                               |
| Student A Tasks | Download RAID dataset from HuggingFace (100K subset). Write <code>scripts/prepare RAID.py</code> — filter to English, balance classes (50K human / 50K AI), train/val/test split (80/10/10). Write <code>scripts/finetune deberta.py</code> using HuggingFace Trainer API. Run fine-tuning on Google Colab Pro A100 (~2h). Save checkpoint to <code>models/deberta_v3 RAID_v1.2/</code> . |
| Student B Tasks | Write training evaluation script: compute F1, AUROC, precision, recall. Generate confusion matrix. Save all metrics to <code>results/text_training_metrics.json</code> . Document hyperparameters (lr=2e-5, batch=16, epochs=4).                                                                                                                                                          |
| Deliverables    | Fine-tuned model checkpoint. Validation F1 ≥ 0.85 on RAID val split. Training metrics report.                                                                                                                                                                                                                                                                                             |
| Dependencies    | Google Colab Pro or Vast.ai A100 access. RAID dataset downloaded.                                                                                                                                                                                                                                                                                                                         |
| Risk            | OOM on Colab A100 if batch size too large — use <code>gradient_accumulation_steps=4</code>                                                                                                                                                                                                                                                                                                |

| Field      | Detail                               |
|------------|--------------------------------------|
| Cloud Cost | ~\$8 (Colab Pro compute unit budget) |
| Effort     | Student A: 20h / Student B: 14h      |

## Week 6 — Text Detection Service Implementation

| Field           | Detail                                                                                                                                                                                                                                                                                        |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | text-service container running DeBERTa inference end-to-end                                                                                                                                                                                                                                   |
| Student A Tasks | Implement <code>TextDetector</code> class (Section 8.9). Load model at startup using FastAPI lifespan event. Implement <code>POST /internal/detect</code> synchronous endpoint. Add preprocessing: tokenisation, 512-token truncation, sentence-boundary detection. Handle CUDA/CPU fallback. |
| Student B Tasks | Implement <code>SHAPTextExplainer</code> : initialise <code>shap.Explainer</code> once at startup with <code>masker=shap.maskers.Text(tokenizer)</code> . Generate top-20 token attributions per request. Build HTML heatmap renderer (inline CSS coloured spans).                            |
| Deliverables    | Text detection returns JSON with <code>ai_probability</code> , <code>confidence_score</code> , <code>verdict</code> . SHAP heatmap generated. Unit tests: 8 passing (including edge cases: empty text, non-English, truncation).                                                              |
| Dependencies    | Week 5 checkpoint in <code>/models/</code> .                                                                                                                                                                                                                                                  |
| Risk            | SHAP initialisation takes 30–60s the first time — init at startup, not per-request                                                                                                                                                                                                            |
| Effort          | Student A: 24h / Student B: 22h                                                                                                                                                                                                                                                               |

## Week 7 — Text API Integration & Frontend Visualization

| Field           | Detail                                                                                                                                                                                                                                                                          |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | User can submit text via dashboard and see results with token heatmap                                                                                                                                                                                                           |
| Student A Tasks | Wire Gateway <code>POST /api/v1/detect/text</code> to call text-service synchronously (httpx). Store result in <code>detection_results</code> table. Store explanation in <code>explanations</code> table. Return completed job response directly (no polling needed for text). |
| Student B Tasks | Build <code>TextResultVisualization</code> React component. Render token heatmap from <code>html_heatmap</code> field. Build bar chart (Recharts) for top AI-indicator tokens and their attribution scores. Add confidence meter UI component.                                  |
| Deliverables    | Full end-to-end: text input → submit → result + heatmap displayed within 3 seconds.                                                                                                                                                                                             |

| Field        | Detail                                                                                 |
|--------------|----------------------------------------------------------------------------------------|
| Dependencies | Weeks 5–6 complete.                                                                    |
| Risk         | Large SHAP heatmaps (>100 tokens) may be slow to render — paginate or truncate display |
| Effort       | Student A: 18h / Student B: 20h                                                        |

## Week 8 — Text Modality Evaluation & Cross-Dataset Testing

| Field           | Detail                                                                                                                                                                                                                                                   |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Documented evaluation results for text modality                                                                                                                                                                                                          |
| Student A Tasks | Download M4 English test set. Run text-service against M4 test samples (1,000 samples). Record F1, AUROC, precision, recall. Compare performance: RAID in-distribution vs M4 cross-domain. Write <code>results/text_cross_dataset_evaluation.md</code> . |
| Student B Tasks | Write 5 integration tests covering: normal text, AI text from GPT-4, paraphrased AI text, non-English input, text at exact 10,000 char limit. Set up test fixtures in <code>tests/conftest.py</code> .                                                   |
| Deliverables    | Cross-dataset evaluation report. 5 integration tests passing.                                                                                                                                                                                            |
| Dependencies    | Week 7 complete. M4 dataset downloaded.                                                                                                                                                                                                                  |
| Risk            | M4 performance significantly lower than RAID val — expected, document gap clearly                                                                                                                                                                        |
| Effort          | Student A: 16h / Student B: 18h                                                                                                                                                                                                                          |

## 9.4 Phase 3 — Image Detection Modality (Weeks 9–12)

### Week 9 — Model Download & Image Preprocessing Pipeline

| Field           | Detail                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Both image models loaded, preprocessing pipeline verified                                                                                                                                                                                                                                                                                                                              |
| Student A Tasks | Clone CNNDetection repo. Download ResNet-50 weights ( <code>blur_jpg_prob0.5.pth</code> ). Clone DMImageDetection repo. Download Corvi EfficientNet-B0 weights. Write <code>ImagePreprocessor</code> class: PIL load, RGBA → RGB, Grayscale → RGB, resize to 224×224, ImageNet normalization. Write <code>FrequencyPreprocessor</code> class: 2D FFT, high-pass filter, log-magnitude. |
| Student B Tasks | Download CIFAKE (500MB). Write <code>scripts/verify_image_models.py</code> — run both models on 100 CIFAKE samples, verify scores are in [0,1] range, measure inference time. Write results summary.                                                                                                                                                                                   |

| Field        | Detail                                                                                                    |
|--------------|-----------------------------------------------------------------------------------------------------------|
| Deliverables | Both models produce valid scores. ResNet-50 < 100ms/image. Corvi < 200ms/image. CIFAKE quick-test passes. |
| Dependencies | Phase 2 complete. GPU available locally or Colab.                                                         |
| Risk         | Corvi FFT preprocessing requires exact normalization constants — verify against paper                     |
| Effort       | Student A: 22h / Student B: 18h                                                                           |

## Week 10 — Image Detection Service & GradCAM

| Field           | Detail                                                                                                                                                                                                                                                                                                                                  |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | image-service container producing ensemble scores and GradCAM heatmaps                                                                                                                                                                                                                                                                  |
| Student A Tasks | Implement <code>ImageDetector</code> class with ensemble scoring. Implement async consumer from <code>forensics.image.detect</code> queue. Download file from MinIO. Run both models. Compute ensemble: $0.55 \times P_{\text{resnet}} + 0.45 \times P_{\text{corvi}}$ . Compute confidence from score divergence. Upload result to DB. |
| Student B Tasks | Implement GradCAM using <code>pytorch-grad-cam</code> library: target layer = <code>model.layer4[-1]</code> for ResNet-50. Generate heatmap PNG. Overlay on original image with <code>alpha=0.5</code> colormap. Upload overlay PNG to MinIO. Store <code>artifact_object_key</code> in <code>explanations</code> table.                |
| Deliverables    | Image detection pipeline end-to-end. GradCAM heatmap stored in MinIO. Ensemble score in DB.                                                                                                                                                                                                                                             |
| Dependencies    | Week 9 complete.                                                                                                                                                                                                                                                                                                                        |
| Risk            | GradCAM target layer name differs between model versions — verify layer name programmatically                                                                                                                                                                                                                                           |
| Effort          | Student A: 24h / Student B: 22h                                                                                                                                                                                                                                                                                                         |

## Week 11 — Image Frontend & Heatmap Visualization

| Field           | Detail                                                                                                                                                                                                                                                       |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | User uploads image, sees result with side-by-side GradCAM overlay                                                                                                                                                                                            |
| Student A Tasks | Wire Gateway <code>POST /api/v1/detect/image</code> to async flow. Implement presigned URL generation for heatmap. Return <code>heatmap_url</code> in job result response.                                                                                   |
| Student B Tasks | Build <code>HeatmapViewer</code> React component: side-by-side original and heatmap overlay. Add opacity slider (0–100%) to toggle heatmap intensity. Display <code>model_ensemble</code> breakdown (ResNet vs Corvi individual scores) in expandable panel. |

| Field        | Detail                                                                                                  |
|--------------|---------------------------------------------------------------------------------------------------------|
| Deliverables | Image upload → async processing → frontend displays heatmap within polling cycle. Opacity slider works. |
| Dependencies | Week 10 complete.                                                                                       |
| Risk         | Presigned URL expiry — set to 3600s; refresh on dashboard revisit                                       |
| Effort       | Student A: 14h / Student B: 22h                                                                         |

## Week 12 — Image Evaluation & AUC Benchmarking

| Field           | Detail                                                                                                                                                                                                                                                             |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Documented AUC results across CIFAKE and GenImage subset                                                                                                                                                                                                           |
| Student A Tasks | Download GenImage 3-generator subset (30GB). Run image-service on 500 CIFAKE samples + 500 GenImage samples. Compute AUC separately for: ResNet-50 alone, Corvi alone, ensemble. Plot ROC curves (matplotlib). Save to <code>results/image_evaluation.png</code> . |
| Student B Tasks | Write 6 integration tests: JPEG, PNG, WebP, RGBA input, corrupted file (expects 422), oversized file (expects 413).                                                                                                                                                |
| Deliverables    | ROC curve plot. Ensemble AUC ≥ individual model AUC. 6 integration tests passing.                                                                                                                                                                                  |
| Dependencies    | Week 11 complete. GenImage subset downloaded.                                                                                                                                                                                                                      |
| Risk            | GenImage subset download may take 8–12h on slow connection — start download in Week 11                                                                                                                                                                             |
| Effort          | Student A: 20h / Student B: 16h                                                                                                                                                                                                                                    |

## 9.5 Phase 4 — Audio Detection Modality (Weeks 13–16)

### Week 13 — ASVspoof Dataset & AASIST Setup

| Field           | Detail                                                                                                                                                                                                                                                                                          |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | AASIST model running on ASVspoof 2019 LA data, baseline EER verified                                                                                                                                                                                                                            |
| Student A Tasks | Download ASVspoof 2019 LA (14GB) from Edinburgh DataShare. Clone AASIST repo. Run official evaluation script on ASVspoof 2019 LA eval set. Verify EER ≈ 0.83%. Write <code>AudioPreprocessor</code> : load WAV/MP3/FLAC via torchaudio, resample to 16kHz mono, pad/truncate to 64,600 samples. |
| Student B Tasks | Download In-the-Wild dataset (4GB). Write <code>scripts/verify_aasist.py</code> — test AASIST on 50 bonafide + 50 spoof samples from ASVspoof, log EER. Document: what does the                                                                                                                 |

| Field        | Detail                                                                                                                  |
|--------------|-------------------------------------------------------------------------------------------------------------------------|
|              | AASIST graph attention output mean? Write a 1-page technical note.                                                      |
| Deliverables | AASIST running. Verified EER $\leq 1.0\%$ on ASVspoof LA eval. Audio preprocessing pipeline verified on MP3, WAV, FLAC. |
| Dependencies | Edinburgh DataShare access approved. 20GB free disk space.                                                              |
| Risk         | torchaudio FFmpeg backend version conflicts — pin <code>torchaudio==2.1.0</code> in requirements.txt                    |
| Effort       | Student A: 20h / Student B: 16h                                                                                         |

## Week 14 — Audio Detection Service Implementation

| Field           | Detail                                                                                                                                                                                                                                                                                                                                                        |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | audio-service container processing audio jobs from RabbitMQ                                                                                                                                                                                                                                                                                                   |
| Student A Tasks | Implement <code>AudioDetector</code> class. Implement async consumer from <code>forensics.audio.detect</code> queue. Full pipeline: download from MinIO $\rightarrow$ <code>AudioPreprocessor</code> $\rightarrow$ AASIST inference $\rightarrow$ sigmoid probability $\rightarrow$ update DB. Handle edge cases: silent audio, very short clips, non-speech. |
| Student B Tasks | Implement spectrogram visualization: <code>librosa.feature.melspectrogram</code> $\rightarrow$ <code>librosa.display.specshow</code> $\rightarrow$ matplotlib savefig to PNG buffer $\rightarrow$ upload to MinIO. Annotate spectrogram with frequency-band anomaly markers if score $> 0.6$ .                                                                |
| Deliverables    | Audio job processed end-to-end. EER logged for test batch. Spectrogram PNG stored in MinIO.                                                                                                                                                                                                                                                                   |
| Dependencies    | Week 13 complete.                                                                                                                                                                                                                                                                                                                                             |
| Risk            | librosa + soundfile + torchaudio version conflicts — test in clean venv first                                                                                                                                                                                                                                                                                 |
| Effort          | Student A: 26h / Student B: 22h                                                                                                                                                                                                                                                                                                                               |

## Week 15 — Audio Frontend & Async UX

| Field           | Detail                                                                                                                                                                                    |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | User uploads audio, polling UI shows progress, result displayed with spectrogram                                                                                                          |
| Student A Tasks | Wire Gateway <code>/detect/audio</code> endpoint. Implement async polling response for audio jobs. Add <code>estimated_completion_seconds: 30</code> in <code>JobCreatedResponse</code> . |
| Student B Tasks | Build <code>AudioResultVisualization</code> component. Display mel-spectrogram image. Add audio playback widget ( <code>&lt;audio&gt;</code> )                                            |

| Field        | Detail                                                                                        |
|--------------|-----------------------------------------------------------------------------------------------|
|              | HTML5 element via presigned URL). Show processing progress bar with estimated time countdown. |
| Deliverables | Audio upload → 202 response → polling → spectrogram + score displayed.                        |
| Dependencies | Week 14 complete.                                                                             |
| Risk         | Large audio files may need chunked upload via multipart — add server-side streaming if needed |
| Effort       | Student A: 14h / Student B: 20h                                                               |

## Week 16 — Audio Cross-Dataset Evaluation

| Field           | Detail                                                                                                                                                                                                                           |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Cross-dataset EER measured on In-the-Wild dataset                                                                                                                                                                                |
| Student A Tasks | Run AASIST on 500 samples from In-the-Wild dataset. Compute EER (in-the-wild expected: 15–20%). Compare against ASVspooof 2019 LA EER (0.83%). Document the generalization gap. Write <code>results/audio_evaluation.md</code> . |
| Student B Tasks | Write 5 integration tests: WAV, MP3, FLAC, silent file (expects low AI score), <1s clip (expects 422).                                                                                                                           |
| Deliverables    | Cross-dataset evaluation documented. Generalization gap acknowledged in methodology. 5 tests passing.                                                                                                                            |
| Dependencies    | Week 15 complete. In-the-Wild dataset downloaded.                                                                                                                                                                                |
| Risk            | EER > 20% on in-the-wild — expected; frame as a limitation, not a failure                                                                                                                                                        |
| Effort          | Student A: 16h / Student B: 16h                                                                                                                                                                                                  |

## 9.6 Phase 5 — Fusion Layer, Reports & System Integration (Weeks 17–20)

### Week 17 — Fusion Engine & Multimodal Endpoint

| Field           | Detail                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | <code>POST /detect/multimodal</code> returns a single fused verdict across all submitted modalities                                                                                                                                                                                                                                                                     |
| Student A Tasks | Implement <code>FusionEngine</code> class. Run calibration script on a synthetic multimodal test set (manually constructed 100 samples). Save calibrated weights to <code>config/fusion_weights.json</code> . Implement multimodal job orchestration in Gateway: fan-out to multiple queues, wait for all results, run fusion, write <code>fusion_results</code> table. |
| Student B Tasks |                                                                                                                                                                                                                                                                                                                                                                         |

| Field        | Detail                                                                                                                                                                                          |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|              | Build <code>MultiModalResultDashboard</code> component. Show each modality score in a radar chart (Recharts <code>RadarChart</code> ). Show fusion score prominently. Color-code verdict badge. |
| Deliverables | Multimodal endpoint returns fused result. Calibration weights saved. Radar chart displayed.                                                                                                     |
| Dependencies | Phases 2–4 complete. All detection services running.                                                                                                                                            |
| Risk         | Fan-out timeout if one modality service is slow — implement per-modality timeout (60s)                                                                                                          |
| Effort       | Student A: 26h / Student B: 22h                                                                                                                                                                 |

## Week 18 — Report Service & PDF Generation

| Field           | Detail                                                                                                                                                                                                                                                                   |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Downloadable PDF report generated for every completed job                                                                                                                                                                                                                |
| Student A Tasks | Implement <code>ReportGenerator</code> class. Write Jinja2 HTML templates for all 5 report pages. Write report CSS (WeasyPrint-compatible). Wire <code>POST /internal/report/generate/{job_id}</code> endpoint. Store PDF in MinIO. Generate 24h presigned download URL. |
| Student B Tasks | Build <code>ReportDownloadButton</code> React component. Wire <code>GET /api/v1/reports/{job_id}/pdf</code> endpoint in Gateway. Add "Generate Report" button on result page. Trigger report generation on job completion. Display download link when ready.             |
| Deliverables    | PDF report downloadable from the dashboard. Report contains all 5 sections. Embedded heatmap images render correctly.                                                                                                                                                    |
| Dependencies    | Week 17 complete. WeasyPrint system fonts installed in Docker image.                                                                                                                                                                                                     |
| Risk            | WeasyPrint SVG rendering issues — convert SVG explanations to PNG before embedding                                                                                                                                                                                       |
| Effort          | Student A: 28h / Student B: 18h                                                                                                                                                                                                                                          |

## Week 19 — System Integration Testing

| Field           | Detail                                                                                                                                                                                                                                                                                                     |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Full end-to-end integration tests passing across all modalities                                                                                                                                                                                                                                            |
| Student A Tasks | Write 3 full end-to-end integration test scenarios: (1) Text-only detection with SHAP. (2) Image detection with GradCAM. (3) Multimodal (text + image) with fusion. Each scenario: submit → poll until complete → verify all DB fields populated → verify MinIO artifacts exist → verify report generates. |
| Student B Tasks |                                                                                                                                                                                                                                                                                                            |

| Field        | Detail                                                                                                                                                                                              |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|              | Write Playwright E2E tests for the frontend: (1) Login flow. (2) Text upload and result display. (3) Image upload and heatmap display. (4) Report download. Run against local Docker Compose stack. |
| Deliverables | 3 backend E2E tests passing. 4 Playwright tests passing. No critical bugs outstanding.                                                                                                              |
| Dependencies | Week 18 complete.                                                                                                                                                                                   |
| Risk         | Timing-dependent tests flake on slow CI — add explicit wait conditions, not fixed sleeps                                                                                                            |
| Effort       | Student A: 22h / Student B: 22h                                                                                                                                                                     |

## Week 20 — Load Testing & Performance Baseline

| Field           | Detail                                                                                                                                                                                        |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Performance baseline documented; bottlenecks identified and addressed                                                                                                                         |
| Student A Tasks | Write Locust load test. Run: 10 concurrent users, 60-second text detection requests for 5 minutes. Record P50, P95, P99 latencies. Profile text-service with <code>py-spy</code> if P95 > 2s. |
| Student B Tasks | Add Prometheus metrics to all services: request count, latency histogram, queue depth gauge. Import Grafana dashboard template for FastAPI.                                                   |
| Deliverables    | Load test report: P95 text latency < 2s under 10 concurrent users. Grafana dashboard running.                                                                                                 |
| Dependencies    | Week 19 complete. Prometheus and Grafana containers healthy.                                                                                                                                  |
| Risk            | Single CPU-bound model causing high latency — consider <code>asyncio.to_thread()</code> for inference calls                                                                                   |
| Effort          | Student A: 18h / Student B: 20h                                                                                                                                                               |

## 9.7 Phase 6 — Evaluation, Polish & Documentation (Weeks 21–24)

### Week 21 — Academic Evaluation Matrix

| Field           | Detail                                                                                                                                                                                                                         |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | All evaluation results computed, tabulated, and ready for dissertation                                                                                                                                                         |
| Student A Tasks | Run full evaluation matrix: Text (RAID val + M4). Image (CIFAKE + GenImage subset). Audio (ASVspoof eval + In-the-Wild). Fusion (synthetic multimodal ROC vs individual modalities). Compute all metrics: F1, AUROC, EER, AUC. |

| Field           | Detail                                                                                                                                                                         |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Student B Tasks | Generate all visualizations: ROC curves, confusion matrices, precision-recall curves. Save all plots as high-res PNGs. Write <code>results/final_evaluation_report.md</code> . |
| Deliverables    | Complete evaluation table ready to copy into dissertation. All plots generated.                                                                                                |
| Effort          | Student A: 24h / Student B: 20h                                                                                                                                                |

## Week 22 — Security Hardening & UAT

| Field           | Detail                                                                                                                                                                             |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | Security audit passed. UAT with 3 non-team testers completed.                                                                                                                      |
| Student A Tasks | Run OWASP ZAP baseline scan. Fix any HIGH-severity findings. Verify: MIME validation, rate limiting (trigger 429), JWT tampering (should return 401), file upload bypass attempts. |
| Student B Tasks | Conduct UAT with 3 external testers (classmates). Provide test scenario cards. Collect feedback. Fix top-3 UX issues.                                                              |
| Deliverables    | ZAP scan: 0 HIGH findings. UAT feedback report. Top-3 UX fixes applied.                                                                                                            |
| Effort          | Student A: 20h / Student B: 22h                                                                                                                                                    |

## Week 23 — Documentation & Video Demo

| Field           | Detail                                                                                                                                                                                                             |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal            | All documentation complete. 5-minute demo video recorded.                                                                                                                                                          |
| Student A Tasks | Write final <code>README.md</code> with: project overview, setup instructions ( <code>make up</code> ), API usage examples, model download instructions. Write <code>CONTRIBUTING.md</code> . Export OpenAPI spec. |
| Student B Tasks | Record 5-minute demo video: login → text detection with SHAP → image detection with GradCAM → audio detection → multimodal fusion → PDF report download. Upload to YouTube (unlisted).                             |
| Deliverables    | README complete. Demo video link in README. OpenAPI spec exported to <code>docs/openapi.yaml</code> .                                                                                                              |
| Effort          | Student A: 16h / Student B: 16h                                                                                                                                                                                    |

## Week 24 — Final Submission Preparation

| Field | Detail                                           |
|-------|--------------------------------------------------|
| Goal  | Project submission ready. Presentation prepared. |

| Field           | Detail                                                                                                                                                                                                                    |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Student A Tasks | Tag final release <code>v1.0.0</code> on GitHub. Create GitHub Release with changelog. Push Docker images to GitHub Container Registry. Final <code>docker-compose up --pull always</code> smoke test from clean machine. |
| Student B Tasks | Prepare 20-slide presentation deck. Slides: Problem → Related Work → Architecture → Demo → Evaluation Results → Limitations → Future Work → Conclusion. Practice demo live.                                               |
| Deliverables    | v1.0.0 GitHub release. Docker images published. Presentation deck ready.                                                                                                                                                  |
| Effort          | Student A: 12h / Student B: 16h                                                                                                                                                                                           |

## 9.8 Milestone Summary Table

| Milestone            | Week | Deliverable                 | Success Criteria                           |
|----------------------|------|-----------------------------|--------------------------------------------|
| M1: Infrastructure   | 4    | Docker stack running        | <code>docker-compose up</code> → all green |
| M2: Text Detection   | 8    | Text service + SHAP         | F1 ≥ 0.85 on RAID val                      |
| M3: Image Detection  | 12   | Image ensemble + GradCAM    | Ensemble AUC ≥ 0.90 on CIFAKE              |
| M4: Audio Detection  | 16   | Audio service + spectrogram | EER ≤ 1.0% on ASVspoof eval                |
| M5: Fusion + Reports | 20   | Fusion + PDF reports        | E2E tests passing                          |
| M6: Final Submission | 24   | Tagged release v1.0.0       | Demo recorded; docs complete               |

# SECTION 10 — MODEL INTEGRATION STRATEGY

## 10.1 Text Detection — DeBERTa-v3-base (RAID Fine-Tuned)

| Property                | Detail                                                                                              |
|-------------------------|-----------------------------------------------------------------------------------------------------|
| Model                   | microsoft/deberta-v3-base (fine-tuned)                                                              |
| GitHub                  | <a href="https://github.com/liamdugan/raid">https://github.com/liamdugan/raid</a> + HuggingFace Hub |
| Parameters              | 86 million                                                                                          |
| Expected F1             | 0.87–0.92 on RAID val; 0.74–0.80 cross-domain (M4)                                                  |
| Expected Inference Time | 80–150ms on GPU; 400–800ms on CPU (512 tokens)                                                      |
| VRAM Required           | 2GB GPU / 4GB RAM CPU                                                                               |

```

# scripts/finetune_deberta.py – Run once on cloud GPU
from datasets import load_dataset
from transformers import (
    DebertaV2ForSequenceClassification, DebertaV2Tokenizer,
    TrainingArguments, Trainer,
)
import numpy as np
from sklearn.metrics import f1_score, roc_auc_score

MODEL_NAME = "microsoft/deberta-v3-base"
OUTPUT_DIR = "./models/deberta_v3_raid_v1.2"

tokenizer = DebertaV2Tokenizer.from_pretrained(MODEL_NAME)
model = DebertaV2ForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)

dataset = load_dataset("liamdugan/raid", split="train")
dataset = dataset.filter(lambda x: x["language"] == "en")

def tokenize(batch):
    return tokenizer(batch["generation"], truncation=True, max_length=512, padding="max_length")

tokenized = dataset.map(tokenize, batched=True, batch_size=1000)

training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    num_train_epochs=4,
    per_device_train_batch_size=16,
    gradient_accumulation_steps=2,
    learning_rate=2e-5,
    warmup_ratio=0.1,
    weight_decay=0.01,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    fp16=True,
    report_to="none",
)

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    probs = np.exp(logits) / np.sum(np.exp(logits), axis=-1, keepdims=True)
    return {
        "f1": f1_score(labels, preds),
        "auROC": roc_auc_score(labels, probs[:, 1]),
    }

trainer = Trainer(
    model=model, args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["validation"],
    compute_metrics=compute_metrics,
)
trainer.train()
trainer.save_model(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)

```

```

# Production inference wrapper
# services/text-service/app/services/text_detector.py
import time, asyncio, torch
from transformers import DebertaV2ForSequenceClassification, DebertaV2Tokenizer

MODEL_PATH = "/models/deberta_v3_raid_v1.2"
_model, _tokenizer, _device = None, None, None

def load_model_sync():
    global _model, _tokenizer, _device
    _device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    _tokenizer = DebertaV2Tokenizer.from_pretrained(MODEL_PATH)
    _model = DebertaV2ForSequenceClassification.from_pretrained(MODEL_PATH)
    _model.eval().to(_device)

```

```

async def predict_text(text: str) -> dict:
    return await asyncio.to_thread(_predict_sync, text)

def _predict_sync(text: str) -> dict:
    t0 = time.perf_counter()
    inputs = _tokenizer(text, return_tensors="pt", max_length=512, truncation=True, padding=True)
    inputs = {k: v.to(_device) for k, v in inputs.items()}
    with torch.no_grad():
        logits = _model(**inputs).logits[0]
        probs = torch.softmax(logits, dim=-1)
        ai_prob = probs[1].item()
        return {
            "ai_probability": round(ai_prob, 4),
            "confidence_score": round(float(probs.max()), 4),
            "verdict": _score_to_verdict(ai_prob),
            "processing_time_ms": int((time.perf_counter() - t0) * 1000),
            "device_used": str(_device),
        }

def _score_to_verdict(s: float) -> str:
    if s >= 0.80: return "AI_GENERATED"
    if s >= 0.60: return "LIKELY_AI"
    if s >= 0.40: return "INCONCLUSIVE"
    if s >= 0.20: return "LIKELY_HUMAN"
    return "HUMAN_GENERATED"

```

## 10.2 Image Detection — ResNet-50 + Corvi EfficientNet-B0 Ensemble

| Property                    | Detail                                                                                                      |
|-----------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Model 1</b>              | ResNet-50 (Wang et al. CNNDetection) — GAN detector                                                         |
| <b>Model 2</b>              | EfficientNet-B0 + FFT (Corvi et al.) — Diffusion detector                                                   |
| <b>GitHub 1</b>             | <a href="https://github.com/peterwang512/CNNDetection">https://github.com/peterwang512/CNNDetection</a>     |
| <b>GitHub 2</b>             | <a href="https://github.com/grip-unina/DMimageDetection">https://github.com/grip-unina/DMimageDetection</a> |
| <b>Ensemble AUC</b>         | 0.92–0.96 across GAN+Diffusion generators                                                                   |
| <b>Total Inference Time</b> | ~230ms on GPU (both models sequential)                                                                      |
| <b>VRAM Required</b>        | ~1.5GB for both models loaded simultaneously                                                                |

```

# services/image-service/app/services/image_detector.py
import io, time, asyncio, torch, torch.nn as nn
import torchvision.transforms as T
from PIL import Image
import numpy as np

class ResNetGANDetector:
    WEIGHTS_PATH = "/models/resnet50_cnn_detection_v2.0/blur_jpg_prob0.5.pth"
    TRANSFORM = T.Compose([
        T.Resize(256), T.CenterCrop(224), T.ToTensor(),
        T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])
    def load(self, device):
        import torchvision.models as models
        self._model = models.resnet50(pretrained=False)
        self._model.fc = nn.Linear(2048, 1)
        state = torch.load(self.WEIGHTS_PATH, map_location=device)
        self._model.load_state_dict(state["model"])
        self._model.eval().to(device)
        self._device = device
    def predict(self, img: Image.Image) -> float:

```

```

        tensor = self.TRANSFORM(img).unsqueeze(0).to(self._device)
        with torch.no_grad():
            return torch.sigmoid(self._model(tensor)[0, 0]).item()

class CorviDiffusionDetector:
    WEIGHTS_PATH = "/models/corvi_effnet_v1.0/model.pth"
    def load(self, device):
        import timm
        self._model = timm.create_model("efficientnet_b0", pretrained=False, num_classes=1)
        self._model.load_state_dict(torch.load(self.WEIGHTS_PATH, map_location=device))
        self._model.eval().to(device)
        self._device = device
    def _fft_features(self, img: Image.Image) -> torch.Tensor:
        gray = np.array(img.convert("L"), dtype=np.float32) / 255.0
        fft = np.fft.fftshift(np.fft.fft2(gray))
        mag = np.log1p(np.abs(fft))
        h, w = mag.shape
        mag[h//2-h//10:h//2+h//10, w//2-w//10:w//2+w//10] = 0
        mag = (mag - mag.min()) / (mag.max() - mag.min() + 1e-8)
        tensor = torch.from_numpy(mag).float().unsqueeze(0).repeat(3, 1, 1)
        return T.Resize((224, 224))(tensor.unsqueeze(0)).squeeze(0)
    def predict(self, img: Image.Image) -> float:
        tensor = self._fft_features(img).unsqueeze(0).to(self._device)
        with torch.no_grad():
            return torch.sigmoid(self._model(tensor)[0, 0]).item()

class ImageEnsembleDetector:
    GAN_WEIGHT = 0.55; DIFF_WEIGHT = 0.45
    def __init__(self):
        self._device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self._gan = ResNetGANDetector()
        self._diff = CorviDiffusionDetector()
    def load(self):
        self._gan.load(self._device)
        self._diff.load(self._device)
    async def predict(self, image_bytes: bytes) -> dict:
        return await asyncio.to_thread(self._predict_sync, image_bytes)
    def _predict_sync(self, image_bytes: bytes) -> dict:
        t0 = time.perf_counter()
        img = Image.open(io.BytesIO(image_bytes)).convert("RGB")
        p_gan = self._gan.predict(img)
        p_diff = self._diff.predict(img)
        p_ensemble = self.GAN_WEIGHT * p_gan + self.DIFF_WEIGHT * p_diff
        confidence = max(0.0, 1.0 - 2.0 * abs(p_gan - p_diff))
        return {
            "ai_probability": round(p_ensemble, 4),
            "confidence_score": round(confidence, 4),
            "verdict": _score_to_verdict(p_ensemble),
            "model_ensemble": [
                {"model": "ResNet50-GANDetector", "weight": 0.55, "score": round(p_gan, 4)},
                {"model": "Corvi-DiffusionDetector", "weight": 0.45, "score": round(p_diff, 4)},
            ],
            "processing_time_ms": int((time.perf_counter() - t0) * 1000),
        }

    def _score_to_verdict(s: float) -> str:
        if s >= 0.80: return "AI_GENERATED"
        if s >= 0.60: return "LIKELY_AI"
        if s >= 0.40: return "INCONCLUSIVE"
        if s >= 0.20: return "LIKELY_HUMAN"
        return "HUMAN_GENERATED"

```

## 10.3 Audio Detection — AASIST-L (Pre-Trained, Inference Only)

| Property | Detail                                                                            |
|----------|-----------------------------------------------------------------------------------|
| Model    | AASIST-L (Large variant, official pre-trained)                                    |
| GitHub   | <a href="https://github.com/clovaai/aasist">https://github.com/clovaai/aasist</a> |

| Property       | Detail                                                 |
|----------------|--------------------------------------------------------|
| Parameters     | 297,000 (under 1MB checkpoint)                         |
| Expected EER   | 0.83% on ASVspoof 2019 LA eval; ~15–20% on in-the-wild |
| Inference Time | 25–40ms per 4s clip on GPU; 80–120ms on CPU            |
| VRAM Required  | < 500MB                                                |

```
# services/audio-service/app/services/audio_detector.py
import io, time, asyncio, torch, torchaudio
import torchaudio.transforms as AT
from app.models.aasist import AASIST # Copy from official repo

WEIGHTS_PATH = "/models/aasist_l_v1.0/AASIST-L.pth"
SAMPLE_RATE = 16_000
FIXED_SAMPLES = 64_600 # ~4.03 seconds

class AASISTDetector:
    def load(self):
        import json
        self._device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        with open("/models/aasist_l_v1.0/AASIST-L.conf") as f:
            config = json.load(f)
        self._model = AASIST(**config["model"])
        state = torch.load(WEIGHTS_PATH, map_location=self._device)
        self._model.load_state_dict(state["model"])
        self._model.eval().to(self._device)

    def _preprocess(self, audio_bytes: bytes) -> torch.Tensor:
        waveform, sr = torchaudio.load(io.BytesIO(audio_bytes))
        if waveform.shape[0] > 1:
            waveform = waveform.mean(dim=0, keepdim=True)
        if sr != SAMPLE_RATE:
            waveform = AT.Resample(sr, SAMPLE_RATE)(waveform)
        waveform = waveform.squeeze(0)
        n = waveform.shape[0]
        if n < FIXED_SAMPLES:
            waveform = torch.nn.functional.pad(waveform, (0, FIXED_SAMPLES - n))
        else:
            waveform = waveform[:FIXED_SAMPLES]
        return waveform.unsqueeze(0)

    async def predict(self, audio_bytes: bytes) -> dict:
        return await asyncio.to_thread(self._predict_sync, audio_bytes)

    def _predict_sync(self, audio_bytes: bytes) -> dict:
        t0 = time.perf_counter()
        waveform = self._preprocess(audio_bytes).to(self._device)
        with torch.no_grad():
            _, logit = self._model(waveform)
            probs = torch.softmax(logit[0], dim=-1)
            spoof_prob = probs[1].item()
        return {
            "ai_probability": round(spoof_prob, 4),
            "confidence_score": round(float(probs.max()), 4),
            "verdict": _score_to_verdict(spoof_prob),
            "processing_time_ms": int((time.perf_counter() - t0) * 1000),
            "device_used": str(self._device),
        }

def _score_to_verdict(s: float) -> str:
    if s >= 0.80: return "AI_GENERATED"
    if s >= 0.60: return "LIKELY_AI"
    if s >= 0.40: return "INCONCLUSIVE"
    if s >= 0.20: return "LIKELY_HUMAN"
    return "HUMAN_GENERATED"
```

10.4 Video Detection — XceptionNet Frame-Level (Phase 4)

| Property       | Detail                                                                                          |
|----------------|-------------------------------------------------------------------------------------------------|
| Model          | XceptionNet (FaceForensics++ trained)                                                           |
| GitHub         | <a href="https://github.com/ondyari/FaceForensics">https://github.com/ondyari/FaceForensics</a> |
| Pipeline       | ffmpeg 2fps → MTCNN face crop → XceptionNet batch → temporal aggregation                        |
| Aggregation    | <code>P = 0.5×mean + 0.3×max + 0.2×vote_fraction</code>                                         |
| Expected AUC   | ~0.90 on FF++ c23; ~0.74 cross-dataset                                                          |
| Inference Time | ~7s for 30 frames at batch_size=8 on GPU                                                        |

10.5 Model Integration Performance Summary

| Modality | Model           | Expected Accuracy  | P95 Latency          | VRAM  | Fine-Tune?          |
|----------|-----------------|--------------------|----------------------|-------|---------------------|
| Text     | DeBERTa-v3-RAID | F1: 0.87–0.92      | < 2s (CPU)           | 2–4GB | Yes (once on cloud) |
| Image    | ResNet50+Corvi  | AUC: 0.92–0.96     | < 500ms (GPU)        | 1.5GB | No (pre-trained)    |
| Audio    | AASIST-L        | EER: 0.83% (bench) | < 100ms (CPU)        | < 1GB | No (pre-trained)    |
| Video    | XceptionNet     | AUC: ~0.90 (bench) | ~7s (GPU, 30 frames) | 2GB   | No (pre-trained)    |

SECTION 11 — EXPLAINABILITY

11.1 Method Decision Matrix

| Method                 | Modality   | Difficulty | Runtime | Usefulness | Status               |
|------------------------|------------|------------|---------|------------|----------------------|
| SHAP Token Attribution | Text       | Medium     | 2–5s    | —          | Phase 2              |
| GradCAM Heatmap        | Image      | Low        | <100ms  | —          | Phase 3              |
| Mel-Spectrogram        | Audio      | Low        | <50ms   | —          | Phase 4              |
| LIME                   | Text/Image | Medium     | 5–30s   |            | Optional fallback    |
| Attention Weights      | Text       | High       | Medium  |            | Future work          |
| Frame Grid             | Video      | Medium     | Medium  |            | Phase 4              |
| Saliency Maps          | Image      | Low        | Low     |            | Redundant vs GradCAM |

11.2 SHAP — Token Attribution for Text

Algorithm: Marginal contribution per token via permutation game with Text masker.

```

# services/text-service/app/services/shap_explainer.py
import asyncio, shap, torch, numpy as np
from transformers import DebertaV2Tokenizer, DebertaV2ForSequenceClassification

class SHAPTextExplainer:
    MAX_TOKENS = 256

    def __init__(self, model, tokenizer, device):
        self._model, self._tokenizer, self._device = model, tokenizer, device
        self._explainer = self._build_explainer()

    def _build_explainer(self):
        def predict_fn(texts):
            inputs = self._tokenizer(texts, return_tensors="pt",
                                     max_length=self.MAX_TOKENS, truncation=True, padding=True)
            inputs = {k: v.to(self._device) for k, v in inputs.items()}
            with torch.no_grad():
                probs = torch.softmax(self._model(**inputs).logits, dim=-1)
            return probs[:, 1].cpu().numpy()
        return shap.Explainer(predict_fn, shap.maskers.Text(self._tokenizer))

    async def explain(self, text: str) -> dict:
        return await asyncio.to_thread(self._explain_sync, text)

    def _explain_sync(self, text: str) -> dict:
        truncated = " ".join(text.split()[:self.MAX_TOKENS])
        sv = self._explainer([truncated])
        values, tokens = sv.values[0], sv.data[0]
        attrs = sorted([
            {"token": t, "attribution": round(float(v), 4), "position": i}
            for i, (t, v) in enumerate(zip(tokens, values))
        ], key=lambda x: abs(x["attribution"]), reverse=True)
        top_ai = [a["token"] for a in attrs[:20] if a["attribution"] > 0]
        return {
            "method": "SHAP",
            "token_attributions": attrs[:20],
            "top_ai_indicator_tokens": top_ai[:10],
            "html_heatmap": self._render_heatmap(tokens, values),
            "truncated": len(text.split()) > self.MAX_TOKENS,
        }

    def _render_heatmap(self, tokens, values) -> str:
        max_val = max(abs(values)) if len(values) > 0 else 1.0
        parts = []
        for tok, val in zip(tokens, values):
            if tok in ["[CLS]", "[SEP]", "[PAD]"]: continue
            intensity = abs(val) / (max_val + 1e-8)
            r, g, b = (255, 0, 0) if val > 0 else (0, 180, 0)
            tok_clean = tok.replace("_", " ").strip()
            parts.append(
                f'<span style="background:rgba({r},{g},{b},{intensity*0.7:.2f});'
                f'padding:1px 3px;border-radius:2px;" title="SHAP:{val:.3f}">{tok_clean}</span>'
            )
        return " ".join(parts)

```

## 11.3 GradCAM — Image Heatmap

```

# services/image-service/app/services/gradcam_explainer.py
import io, asyncio, numpy as np, torch
from PIL import Image
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image

class GradCAMExplainer:
    def __init__(self, model, target_layer_name="layer4"):
        target_layer = getattr(model, target_layer_name)[-1]
        self._cam = GradCAM(model=model, target_layers=[target_layer])

    async def explain(self, image_bytes: bytes) -> bytes:

```

```

        return await asyncio.to_thread(self._explain_sync, image_bytes)

    def _explain_sync(self, image_bytes: bytes) -> bytes:
        img = Image.open(io.BytesIO(image_bytes)).convert("RGB").resize((224, 224))
        img_arr = np.array(img, dtype=np.float32) / 255.0
        mean, std = np.array([0.485, 0.456, 0.406]), np.array([0.229, 0.224, 0.225])
        inp = torch.from_numpy((img_arr - mean) / std).transpose(2, 0, 1).float().unsqueeze(0)
        cam = self._cam(input_tensor=inp)[0]
        vis = show_cam_on_image(img_arr, cam, use_rgb=True)
        buf = io.BytesIO()
        Image.fromarray(vis).save(buf, format="PNG")
        return buf.getvalue()

    def get_highlighted_regions(self, cam: np.ndarray) -> list[dict]:
        h, w = cam.shape
        regions = {
            "top-left": cam[:h//2, :w//2].mean(), "top-right": cam[:h//2, w//2:].mean(),
            "bottom-left": cam[h//2:, :w//2].mean(), "bottom-right": cam[h//2:, w//2:].mean(),
            "center": cam[h//4:3*h//4, w//4:3*w//4].mean(),
        }
        return [{"region": n, "intensity": round(float(v), 4)}
                for n, v in sorted(regions.items(), key=lambda x: -x[1]) if v > 0.3]

```

## 11.4 Mel-Spectrogram — Audio Visualization

```

# services/audio-service/app/services/spectrogram_visualizer.py
import io, asyncio, numpy as np, librosa, librosa.display
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import torchaudio

class SpectrogramVisualizer:
    SR = 16_000
    async def generate(self, audio_bytes: bytes, ai_probability: float) -> bytes:
        return await asyncio.to_thread(self._gen_sync, audio_bytes, ai_probability)
    def _gen_sync(self, audio_bytes: bytes, ai_probability: float) -> bytes:
        waveform, sr = torchaudio.load(io.BytesIO(audio_bytes))
        if waveform.shape[0] > 1: waveform = waveform.mean(dim=0)
        y = waveform.numpy().squeeze()
        if sr != self.SR: y = librosa.resample(y, orig_sr=sr, target_sr=self.SR)
        y = y[:self.SR * 10]
        mel = librosa.feature.melspectrogram(y=y, sr=self.SR, n_mels=128, fmax=8000)
        mel_db = librosa.power_to_db(mel, ref=np.max)
        fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6), facecolor="#1a1a2e")
        fig.suptitle(f"Audio Forensics | AI Probability: {ai_probability*100:.1f}%",
                    color="white", fontsize=13, fontweight="bold")
        for ax in (ax1, ax2): ax.set_facecolor("#16213e"); ax.tick_params(colors="white")
        librosa.display.waveshow(y, sr=self.SR, ax=ax1, color="#e94560")
        ax1.set_title("Waveform", color="white")
        img = librosa.display.specshow(mel_db, sr=self.SR, x_axis="time", y_axis="mel",
                                       fmax=8000, ax=ax2, cmap="magma")
        fig.colorbar(img, ax=ax2, format="%+2.0f dB")
        ax2.set_title("Mel-Spectrogram (dB)", color="white")
        plt.tight_layout()
        buf = io.BytesIO()
        plt.savefig(buf, format="png", dpi=120, bbox_inches="tight", facecolor="#1a1a2e")
        plt.close(fig)
        return buf.getvalue()

```

## 11.5 Explanation Narrative Generator

```

# shared/services/narrative_generator.py
VERDICT_TEMPLATES = {
    "AI_GENERATED": "The {modality} content is classified as AI-generated ({score:.0%} probability, {confidence:.0%} confidence). {evidence}",
    "LIKELY_AI": "The {modality} content shows strong AI indicators ({score:.0%} probability). {evidence}",
}

```

```

    "INCONCLUSIVE": "Analysis is inconclusive ({score:.0%} AI probability). {evidence} Manual review recommended.",
    "LIKELY_HUMAN": "The {modality} content appears likely human-generated ({score:.0%} AI probability). {evidence}",
    "HUMAN_GENERATED": "No significant AI indicators detected ({score:.0%} probability). {evidence}",
}

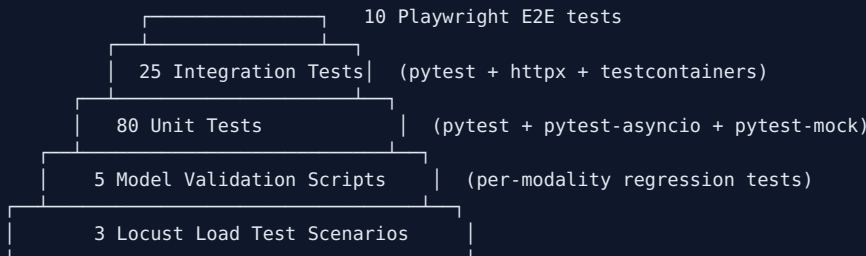
def generate_text_narrative(result: dict, explanation: dict) -> str:
    tokens = ", ".join(f"'{t}'" for t in explanation.get("top_ai_indicator_tokens", [])[:5])
    evidence = (f"AI indicators: {tokens}. These phrases are statistically over-represented in LLM outputs."
               if result["ai_probability"] >= 0.5
               else "Text shows natural variation consistent with human authorship.")
    return VERDICT_TEMPLATES[result["verdict"]].format(
        modality="text", score=result["ai_probability"],
        confidence=result["confidence_score"], evidence=evidence
    )

def generate_image_narrative(result: dict, regions: list[dict]) -> str:
    region_names = ", ".join(r["region"] for r in regions[:3])
    evidence = (f"GradCAM highlights {region_names} as primary artifact regions."
               if result["ai_probability"] >= 0.5
               else "No significant GAN upsampling or diffusion artifacts detected.")
    return VERDICT_TEMPLATES[result["verdict"]].format(
        modality="image", score=result["ai_probability"],
        confidence=result["confidence_score"], evidence=evidence
    )

```

## SECTION 12 — TESTING STRATEGY

### 12.1 Testing Pyramid



### 12.2 Unit Tests (80 tests)

```

# tests/unit/test_fusion_engine.py
import pytest
from app.services.fusion_engine import FusionEngine

class TestFusionEngine:
    def setup_method(self):
        self.engine = FusionEngine()

    def test_single_text_ai(self):
        r = self.engine.fuse({"text": 0.91})
        assert r.final_verdict == "AI_GENERATED"
        assert r.final_score == pytest.approx(0.91, abs=0.01)

    def test_high_agreement_high_confidence(self):
        r = self.engine.fuse({"text": 0.90, "image": 0.88})
        assert r.final_confidence > 0.70

    def test_high_disagreement_low_confidence(self):
        r = self.engine.fuse({"text": 0.95, "image": 0.10})
        assert r.final_confidence < 0.40

```

```

def test_boundary_inconclusive(self):
    r = self.engine.fuse({"text": 0.50})
    assert r.final_verdict == "INCONCLUSIVE"

def test_weights_sum_to_one(self):
    r = self.engine.fuse({"text": 0.80, "audio": 0.80})
    assert sum(r.weights_applied.values()) == pytest.approx(1.0, abs=0.001)

def test_empty_raises(self):
    with pytest.raises(ValueError): self.engine.fuse({})

def test_score_in_range(self):
    r = self.engine.fuse({"text": 1.0})
    assert 0.0 <= r.final_score <= 1.0
    assert 0.0 <= r.final_confidence <= 1.0

# tests/unit/test_verdict_mapping.py
def test_verdict_thresholds():
    from app.services.text_detector import _score_to_verdict
    assert _score_to_verdict(0.95) == "AI_GENERATED"
    assert _score_to_verdict(0.80) == "AI_GENERATED"    # boundary inclusive
    assert _score_to_verdict(0.799) == "LIKELY_AI"
    assert _score_to_verdict(0.50) == "INCONCLUSIVE"
    assert _score_to_verdict(0.30) == "LIKELY_HUMAN"
    assert _score_to_verdict(0.10) == "HUMAN_GENERATED"

```

## 12.3 Integration Tests (25 tests)

```

# tests/integration/test_text_detection_api.py
import pytest
from httpx import AsyncClient
from app.main import app

class TestTextDetectionEndpoint:
    @pytest.mark.asyncio
    async def test_valid_text_returns_result(self, auth_headers):
        async with AsyncClient(app=app, base_url="http://test") as c:
            r = await c.post("/api/v1/detect/text",
                            json={"text": "The ramifications of advanced ML " * 10},
                            headers=auth_headers)
            assert r.status_code == 200
            data = r.json()
            assert 0.0 <= data["results"][0]["ai_probability"] <= 1.0

    @pytest.mark.asyncio
    async def test_short_text_returns_422(self, auth_headers):
        async with AsyncClient(app=app, base_url="http://test") as c:
            r = await c.post("/api/v1/detect/text",
                            json={"text": "Too short."}, headers=auth_headers)
            assert r.status_code == 422

    @pytest.mark.asyncio
    async def test_no_auth_returns_401(self):
        async with AsyncClient(app=app, base_url="http://test") as c:
            r = await c.post("/api/v1/detect/text", json={"text": "Some text here."})
            assert r.status_code == 401

    @pytest.mark.asyncio
    async def test_explanation_present_when_requested(self, auth_headers):
        async with AsyncClient(app=app, base_url="http://test") as c:
            r = await c.post("/api/v1/detect/text",
                            json={"text": "Furthermore it is important to note " * 20, "explain": True},
                            headers=auth_headers)
            assert r.json()["results"][0]["explanation"] is not None

    @pytest.mark.asyncio
    async def test_image_wrong_mime_returns_400(self, auth_headers):
        async with AsyncClient(app=app, base_url="http://test") as c:
            r = await c.post("/api/v1/detect/image",

```

```

        files={"file": ("test.jpg", b"%PDF-1.4 fake", "application/pdf")},
        headers=auth_headers)
assert r.status_code == 400
assert r.json()["error"] == "INVALID_MIME_TYPE"

```

## 12.4 Model Validation Scripts

```

# scripts/validate_models.py
FIXTURES = {
    "text": [
        ("The mitochondria is the powerhouse of the cell. ", "HUMAN_GENERATED", 0.0, 0.40),
        ("Furthermore, it is worth noting that the utilization " * 5, "LIKELY_AI", 0.60, 1.0),
    ],
}
def validate_text(fixtures):
    from app.services.text_detector import load_model_sync, _predict_sync
    load_model_sync()
    passed = 0
    for text, expected_verdict, lo, hi in fixtures:
        r = _predict_sync(text)
        ok = lo <= r["ai_probability"] <= hi and r["verdict"] == expected_verdict
        print(f"  [{'PASS' if ok else 'FAIL'}] {expected_verdict} | Got: {r['verdict']} ({r['ai_probability']:.3f})")
        passed += ok
    return passed == len(fixtures)

```

## 12.5 Load Testing (Locust)

```

# tests/load/locustfile.py
from locust import HttpUser, task, between

class ForensicsUser(HttpUser):
    wait_time = between(1, 3)
    def on_start(self):
        r = self.client.post("/api/v1/auth/login",
            json={"email": "loadtest@test.com", "password": "loadtest_pass"})
        self.headers = {"Authorization": f"Bearer {r.json()['access_token']}"}

    @task(5)
    def detect_text_fast(self):
        self.client.post("/api/v1/detect/text",
            json={"text": "AI text detection benchmark " * 10, "explain": False},
            headers=self.headers)

    @task(2)
    def detect_text_with_shap(self):
        self.client.post("/api/v1/detect/text",
            json={"text": "Furthermore the utilization of advanced ML " * 15, "explain": True},
            headers=self.headers)

    @task(1)
    def list_jobs(self):
        self.client.get("/api/v1/jobs", headers=self.headers)

```

```

# Run load test
locust -f tests/load/locustfile.py \
  --host http://localhost:8000 \
  --users 10 --spawn-rate 2 --run-time 5m \
  --headless --csv results/load_test_$(date +%Y%m%d)
# Target SLOs:
# Text P95 (no SHAP) < 2000ms | Text P95 (with SHAP) < 6000ms
# Job list P95 < 100ms | Error rate < 1%

```

## 12.6 Security Testing Checklist

| Test                       | Tool                                                  | Pass Criteria        |
|----------------------------|-------------------------------------------------------|----------------------|
| Dependency vulnerabilities | <code>pip-audit -r requirements.txt</code>            | 0 CRITICAL / 0 HIGH  |
| OWASP baseline             | <code>zap-baseline.py -t http://localhost:8000</code> | 0 HIGH alerts        |
| SQL injection              | SQLMap <code>--level 2</code>                         | No injections found  |
| JWT tampering              | Manual: alter payload, re-sign wrong key              | Returns 401          |
| File type bypass           | Upload <code>.php</code> as <code>image/jpeg</code>   | Returns 400          |
| Rate limit                 | 70 req/min                                            | 429 after request 61 |
| Secrets in code            | <code>git secrets --scan</code>                       | 0 findings           |
| Plaintext passwords        | Check DB directly                                     | Only bcrypt hashes   |

## 12.7 UAT Plan (10 Scenarios)

| #  | Scenario                         | Action               | Expected                         | Accept Criteria |
|----|----------------------------------|----------------------|----------------------------------|-----------------|
| 1  | Registration                     | Fill form, submit    | Account created                  | No error        |
| 2  | Submit AI text (GPT-4 essay)     | Paste text, detect   | AI_GENERATED, heatmap visible    | Score > 0.70    |
| 3  | Submit human text (hand-written) | Paste text, detect   | HUMAN or INCONCLUSIVE            | Score < 0.50    |
| 4  | Upload AI image (Midjourney)     | Drop PNG file        | LIKELY_AI, GradCAM visible       | Score > 0.60    |
| 5  | Upload real photo                | Drop personal JPEG   | HUMAN or INCONCLUSIVE            | Score < 0.50    |
| 6  | Upload TTS audio (ElevenLabs)    | Drop MP3             | LIKELY_AI, spectrogram visible   | Score > 0.60    |
| 7  | Download PDF report              | Click button         | PDF downloaded with all sections | File > 50KB     |
| 8  | View job history                 | Navigate to job list | Past jobs listed                 | ≥ 3 jobs shown  |
| 9  | Upload invalid file type         | Drop PDF as image    | Clear error shown                | No crash        |
| 10 | Session expiry                   | Wait 16min, detect   | Redirected to login              | Graceful logout |

# SECTION 13 — DEVOPS

## 13.1 Dockerfile (Multi-Stage, All Services)

```
# services/text-service/Dockerfile
FROM python:3.11-slim AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends gcc libpq-dev libmagic1 \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
```

```

RUN pip install --upgrade pip && pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim AS runtime
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends libpq5 libmagic1 ffmpeg curl \
    && rm -rf /var/lib/apt/lists/*
COPY --from=builder /install /usr/local
COPY ./app ./app
RUN useradd -m -u 1001 appuser && chown -R appuser:appuser /app
USER appuser
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
    CMD curl -f http://localhost:8001/internal/health || exit 1
EXPOSE 8001
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8001", \
    "--workers", "1", "--loop", "uvloop"]

```

## 13.2 Docker Compose

```

# docker-compose.yml (key services – full version in repo)
version: "3.9"
x-common-env: &common-env
  DATABASE_URL: postgresql+asyncpg://forensics_user:${POSTGRES_PASSWORD}@postgres:5432/forensics_db
  REDIS_URL: redis://redis:6379/0
  RABBITMQ_URL: amqp://forensics:${RABBITMQ_PASSWORD}@rabbitmq:5672/
  MINIO_ENDPOINT: minio:9000
  MINIO_ACCESS_KEY: ${MINIO_ACCESS_KEY}
  MINIO_SECRET_KEY: ${MINIO_SECRET_KEY}
  MINIO_BUCKET: forensics-bucket

services:
  postgres:
    image: postgres:16-alpine
    environment: {POSTGRES_DB: forensics_db, POSTGRES_USER: forensics_user, POSTGRES_PASSWORD: "${POSTGRES_PASSWORD}"}
    volumes: ["/data/postgres:/var/lib/postgresql/data"]
    healthcheck: {test: ["CMD", "pg_isready", "-U", "forensics_user"], interval: 30s, retries: 5}

  redis:
    image: redis:7-alpine
    volumes: ["/data/redis:/data"]
    healthcheck: {test: ["CMD", "redis-cli", "ping"], interval: 30s}

  rabbitmq:
    image: rabbitmq:3.13-management-alpine
    environment: {RABBITMQ_DEFAULT_USER: forensics, RABBITMQ_DEFAULT_PASS: "${RABBITMQ_PASSWORD}"}
    ports: ["127.0.0.1:15672:15672"]
    healthcheck: {test: ["CMD", "rabbitmq-diagnostics", "check_running"], interval: 30s}

  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    environment: {MINIO_ROOT_USER: "${MINIO_ACCESS_KEY}", MINIO_ROOT_PASSWORD: "${MINIO_SECRET_KEY}"}
    volumes: ["/data/minio:/data"]
    ports: ["127.0.0.1:9001:9001"]

  api-gateway:
    build: ./services/api-gateway
    ports: ["8000:8000"]
    environment:
      <<: *common-env
      JWT_SECRET_KEY: ${JWT_SECRET_KEY}
      TEXT_SERVICE_URL: http://text-service:8001
      IMAGE_SERVICE_URL: http://image-service:8002
      AUDIO_SERVICE_URL: http://audio-service:8003
      REPORT_SERVICE_URL: http://report-service:8005
      AUTH_SERVICE_URL: http://auth-service:8006
    depends_on:
      postgres: {condition: service_healthy}
      redis: {condition: service_healthy}
      rabbitmq: {condition: service_healthy}

```

```

text-service:
  build: ./services/text-service
  expose: ["8001"]
  environment: {<<: *common-env, TEXT_MODEL_PATH: /models/deberta_v3_raid_v1.2}
  volumes: ["/models:/models:ro"]
  deploy: {resources: {limits: {memory: 5G}}}

image-service:
  build: ./services/image-service
  expose: ["8002"]
  environment: {<<: *common-env, IMAGE_MODEL_GAN_PATH: /models/resnet50_cnn_detection_v2.0,
IMAGE_MODEL_DIFF_PATH: /models/corvi_effnet_v1.0}
  volumes: ["/models:/models:ro"]
  deploy:
    resources:
      limits: {memory: 7G}
      reservations:
        devices: [{driver: nvidia, count: 1, capabilities: [gpu]}]

prometheus:
  image: prom/prometheus:v2.51.0
  volumes: ["/config/prometheus.yml:/etc/prometheus/prometheus.yml:ro", "/data/prometheus:/prometheus"]
  ports: ["127.0.0.1:9090:9090"]

grafana:
  image: grafana/grafana:10.3.0
  volumes: ["/data/grafana:/var/lib/grafana"]
  environment: {GF_SECURITY_ADMIN_PASSWORD: "${GRAFANA_PASSWORD}"}
  ports: ["127.0.0.1:3001:3000"]

```

### 13.3 GitHub Actions CI/CD

```

# .github/workflows/ci.yml
name: CI
on:
  push: {branches: [main, dev]}
  pull_request: {branches: [main, dev]}
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11", cache: pip}
      - run: pip install ruff mypy
      - run: ruff check services/ shared/
      - run: mypy services/ shared/ --ignore-missing-imports

  test:
    runs-on: ubuntu-latest
    needs: lint
    services:
      postgres:
        image: postgres:16-alpine
        env: {POSTGRES_DB: test_db, POSTGRES_USER: test_user, POSTGRES_PASSWORD: test_pass}
        ports: ["5432:5432"]
        options: "--health-cmd pg_isready --health-interval 10s --health-retries 5"
      redis:
        image: redis:7-alpine
        ports: ["6379:6379"]
        options: "--health-cmd 'redis-cli ping' --health-interval 10s"
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11", cache: pip}
      - run: pip install -e shared/ && pip install pytest pytest-asyncio pytest-cov pytest-mock httpx
      - name: Run migrations
        env:
          DATABASE_URL: postgresql+asyncpg://test_user:test_pass@localhost:5432/test_db

```

```

    run: cd shared/db && alembic upgrade head
- name: Run tests
  env:
    DATABASE_URL: postgresql+asyncpg://test_user:test_pass@localhost:5432/test_db
    REDIS_URL: redis://localhost:6379/0
    JWT_SECRET_KEY: test_secret_key_minimum_32_characters_here
  run: pytest services/ shared/ --cov=. --cov-report=xml -v --ignore=tests/load

build:
  runs-on: ubuntu-latest
  needs: test
  if: github.ref == 'refs/heads/main'
  strategy:
    matrix:
      service: [api-gateway, text-service, image-service, audio-service, report-service, auth-service]
  steps:
    - uses: actions/checkout@v4
    - uses: docker/setup-buildx-action@v3
    - uses: docker/login-action@v3
      with:
        registry: ghcr.io, username: "${{ github.actor }}", password: "${{ secrets.GITHUB_TOKEN }}"
    - uses: docker/build-push-action@v5
      with:
        context: ./services/${{ matrix.service }}
        push: true
        tags: "ghcr.io/${{ github.repository_owner }}/forensics-${{ matrix.service }}:${{ github.sha }}"
        cache-from: type=gha
        cache-to: type=gha,mode=max

```

## 13.4 Prometheus Configuration

```

# config/prometheus.yml
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: api-gateway
    static_configs: [{targets: ["api-gateway:8000"]}]}
  - job_name: text-service
    static_configs: [{targets: ["text-service:8001"]}]}
  - job_name: image-service
    static_configs: [{targets: ["image-service:8002"]}]}
  - job_name: audio-service
    static_configs: [{targets: ["audio-service:8003"]}]}
  - job_name: rabbitmq
    static_configs: [{targets: ["rabbitmq:15692"]}]}
metrics_path: /metrics

```

```

# Add to each service's main.py
from prometheus_fastapi_instrumentator import Instrumentator
from prometheus_client import Counter, Histogram

DETECTIONS = Counter("forensics_detections_total", "Total detections", ["modality", "verdict"])
LATENCY = Histogram("forensics_latency_seconds", "Inference latency", ["modality"],
    buckets=[0.1, 0.5, 1.0, 2.5, 5.0, 10.0])
Instrumentator().instrument(app).expose(app)

```

## 13.5 Structured Logging

```

# shared/logging_config.py
import structlog

def configure_logging(log_format: str = "json"):
    processors = [
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
    ]

```

```

    structlog.processors.TimeStamper(fmt="iso"),
]
processors.append(
    structlog.processors.JSONRenderer() if log_format == "json"
    else structlog.dev.ConsoleRenderer()
)
structlog.configure(processors=processors,
                    logger_factory=structlog.PrintLoggerFactory())

# Usage:
# logger = structlog.get_logger()
# logger.info("detection_complete", job_id=job_id, score=0.87, latency_ms=340)

```

## 13.6 Alerting Rules

```

# config/alerting_rules.yml
groups:
- name: forensics
  rules:
    - alert: ServiceDown
      expr: up == 0
      for: 1m
      labels: {severity: critical}
      annotations: {summary: "Service {{ $labels.job }} is down"}

    - alert: HighDetectionLatency
      expr: histogram_quantile(0.95, forensics_latency_seconds_bucket) > 5
      for: 5m
      labels: {severity: warning}
      annotations: {summary: "P95 latency > 5s for {{ $labels.modality }}" }

    - alert: QueueBacklog
      expr: forensics_queue_depth > 50
      for: 2m
      labels: {severity: warning}
      annotations: {summary: "Queue {{ $labels.queue_name }} has {{ $value }} pending"}

    - alert: HighErrorRate
      expr: rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m]) > 0.05
      for: 3m
      labels: {severity: critical}
      annotations: {summary: "Error rate > 5% on {{ $labels.job }}" }

```

## 13.7 Versioning & Release Strategy

```

VERSIONING: Semantic Versioning (MAJOR.MINOR.PATCH)
v1.0.0 → Initial FYP release
v1.1.0 → Add audio modality
v1.2.0 → Add fusion engine
v2.0.0 → Breaking API change

BRANCH STRATEGY: GitHub Flow
main      ← Always deployable (protected)
dev       ← Integration branch
feature/* ← Feature branches
hotfix/*  ← Emergency fixes

COMMIT FORMAT (Conventional Commits):
feat(audio): add AASIST integration
fix(gateway): resolve MIME validation for WebP
docs(api): update OpenAPI spec
test(text): add M4 cross-dataset evaluation
chore(deps): upgrade torch to 2.2.0

DOCKER TAGS:
:latest      ← Tracks main

```

```
:v1.0.0      ← Immutable release
:sha-abc1234 ← Commit SHA

RELEASE PROCESS:
1. PR: feature/* → dev (1 review + CI green)
2. PR: dev → main (1 review + CI green + smoke test)
3. git tag -a v1.x.x -m "Changelog"
4. GitHub Release auto-generated from commits
5. Docker images pushed (tagged :latest + :v1.x.x)
6. SSH deploy: docker compose pull && docker compose up -d
```

## SECTION 14 — FINAL RECOMMENDATION

### 14.1 Executive Summary

For a 1–3 student FYP team with limited GPU, 6–8 months, and goals of academic rigor plus industry relevance, this section provides the exact non-negotiable choices. Every decision prioritizes: (1) achievability in the given timeline, (2) maximum resume signal, (3) academic rigor, (4) real-world applicability.

### 14.2 Exact Datasets

| Priority                               | Modality | Dataset                                              | Action                           |
|----------------------------------------|----------|------------------------------------------------------|----------------------------------|
| <div><div></div><div>MUST</div></div>  | Text     | <b>RAID (100K subset)</b> — ACL 2024, MIT license    | Download Week 1 from HuggingFace |
| <div><div></div><div>MUST</div></div>  | Text     | <b>M4 English test</b> — cross-domain eval           | Download Week 7                  |
| <div><div></div><div>MUST</div></div>  | Image    | <b>CIFAKE (120K)</b> — 500MB, CC0                    | Download Week 1                  |
| <div><div></div><div>MUST</div></div>  | Image    | <b>GenImage (30GB subset)</b> — 3 generators         | Download Week 11                 |
| <div><div></div><div>MUST</div></div>  | Audio    | <b>ASVspoof 2019 LA (14GB)</b> — canonical benchmark | Request access Week 1            |
| <div><div></div><div>MUST</div></div>  | Audio    | <b>In-the-Wild (4GB)</b> — generalization test       | Download Week 13                 |
| <div><div></div><div>SKIP</div></div>  | Image    | ArtiFact (500GB) — too large                         | Use GenImage                     |
| <div><div></div><div>SKIP</div></div>  | Audio    | ASVspoof 2021 DF (130GB) — too large                 | Use 2019 LA                      |
| <div><div></div><div>AVOID</div></div> | Video    | Any large video dataset — Phase 4 stretch only       | Skip unless time permits         |

## 14.3 Exact Models

| Modality               | Model                                                       | Reason                                                            |
|------------------------|-------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Text</b>            | <code>microsoft/deberta-v3-base</code> fine-tuned on RAID   | Best accuracy/size ratio, SHAP-compatible, HuggingFace ecosystem  |
| <b>Image GAN</b>       | ResNet-50 (CNNDetection <code>blur_jpg_prob0.5.pth</code> ) | Proven, 25M params, sub-100ms, GradCAM-compatible                 |
| <b>Image Diffusion</b> | Corvi EfficientNet-B0                                       | Lightweight, frequency-based, trainable on single RTX 3090        |
| <b>Audio</b>           | AASIST-L (pre-trained)                                      | 297K params, 30ms inference, near-SOTA EER, pre-trained available |
| <b>Video (Phase 4)</b> | XceptionNet (FF++ trained)                                  | Academic standard, MTCNN-compatible, widely cited                 |
| <b>AVOID</b>           | DIRE                                                        | 12GB VRAM at inference — impossible for web service               |
| <b>AVOID</b>           | Binoculars                                                  | Two 7B models simultaneously — hardware infeasible                |
| <b>AVOID</b>           | Any custom transformer                                      | 6–18 months work — use HuggingFace                                |

## 14.4 Exact GitHub Repositories

| Component                | Repository                               | Integration                                                  |
|--------------------------|------------------------------------------|--------------------------------------------------------------|
| RAID dataset             | <code>liamdugan/raid</code>              | HuggingFace datasets API                                     |
| DeBERTa fine-tune        | <code>huggingface/transformers</code>    | <code>Trainer</code> API, copy script from Section 10.1      |
| ResNet-50 GAN detector   | <code>peterwang512/CNNDetection</code>   | Copy <code>networks/resnet.py</code> + download weights      |
| Corvi diffusion detector | <code>grip-unina/DMimageDetection</code> | Copy model architecture + download weights                   |
| GradCAM                  | <code>jacobgil/pytorch-grad-cam</code>   | <code>pip install grad-cam</code> — 5 lines                  |
| AASIST audio detector    | <code>clovaai/aasist</code>              | Clone, copy <code>models/AASIST.py</code> + config + weights |
| MTCNN face detection     | <code>timesler/facenet-pytorch</code>    | <code>pip install facenet-pytorch</code>                     |
| XceptionNet video        | <code>ondyari/FaceForensics</code>       | Download weights + <code>xception.py</code>                  |
| SHAP                     | <code>slundberg/shap</code>              | <code>pip install shap</code>                                |
| API                      | <code>tiangolo/fastapi</code>            | <code>pip install fastapi[standard]</code>                   |
| Queue                    | <code>mosquito/aio-pika</code>           | <code>pip install aio-pika</code>                            |
| Object storage           | <code>minio/minio-py</code>              | <code>pip install minio</code>                               |
| PDF reports              | <code>Kozea/WeasyPrint</code>            | <code>pip install weasyprint</code>                          |

| Component    | Repository                   | Integration                     |
|--------------|------------------------------|---------------------------------|
| Load testing | <code>locustio/locust</code> | <code>pip install locust</code> |
| Linting      | <code>astral-sh/ruff</code>  | <code>pip install ruff</code>   |

## 14.5 Exact Architecture

| Decision         | Choice                                | Reason                                                   |
|------------------|---------------------------------------|----------------------------------------------------------|
| API Framework    | FastAPI (Python 3.11)                 | Async, auto OpenAPI, type-safe, standard for ML APIs     |
| Frontend         | Next.js 14, TypeScript, App Router    | Strongest resume signal. SSR + polling in one framework  |
| Database         | PostgreSQL 16                         | ACID, JSONB, array types for flexible result storage     |
| ORM              | SQLAlchemy 2.0 async + Alembic        | Type-safe, async-native, migration management            |
| Queue            | RabbitMQ (durable + DLX)              | Better than Redis Streams for ACK + retry                |
| Object Storage   | MinIO (S3-compatible)                 | Self-hosted, zero cloud cost, swap to AWS S3 via env var |
| Cache            | Redis 7                               | JWT blacklist + rate limit counters only                 |
| Containerization | Docker Compose (single host)          | Right tool for FYP. Do NOT use Kubernetes.               |
| Sync vs Async    | Text: sync / Image+Audio: async queue | Text < 2s. Image/Audio need queue for backpressure       |
| Model loading    | Singleton at startup                  | Never load per-request. FastAPI lifespan events.         |
| GPU assignment   | Image service only                    | Image benefits most. Text/Audio fast on CPU.             |
| Explainability   | SHAP + GradCAM + Spectrogram          | Practical, documented, visualizable in browser           |
| Fusion           | Weighted linear (calibrated LR)       | Interpretable, fast, validated                           |
| Reports          | WeasyPrint + Jinja2                   | No paid deps. Full CSS control.                          |
| Monitoring       | Prometheus + Grafana                  | Docker-native, importable dashboards                     |
| CI/CD            | GitHub Actions                        | Free for public repos, standard industry toolchain       |
| Linting          | Ruff                                  | 100× faster than black+flake8, single tool               |

## 14.6 Deployment Strategy

```
# Development (Weeks 1–20) – single command
make up           # docker compose up -d --build
make migrate      # docker compose exec api-gateway alembic upgrade head
make test         # pytest services/ shared/ -v
make logs s=text  # docker compose logs -f text-service

# Staging (Weeks 21–24) – DigitalOcean $24/mo Droplet (4vCPU, 8GB RAM)
# nginx + Let's Encrypt SSL (certbot auto-renew)
# GitHub Actions CD: SSH deploy on merge to main
# docker compose pull && docker compose up -d --remove-orphans

# DO NOT USE:
# Kubernetes → Overkill. 3× setup time for no FYP benefit.
# Terraform → Overkill for single VM.
# AWS ECS/GCP → Adds cost and complexity. Use DigitalOcean.
# Multiple envs → One staging environment sufficient for FYP.
```

## 14.7 Final Implementation Checklist

```
PHASE 1 – FOUNDATION
[ ] GitHub repo + branch strategy (main/dev/feature/*)
[ ] docker-compose.yml: all containers healthy
[ ] All 8 Alembic migrations applied and verified
[ ] Auth: register → login → JWT → protected route
[ ] File upload → MinIO → job in DB → RabbitMQ message verified
[ ] GitHub Actions CI: lint + tests on every PR

PHASE 2 – TEXT
[ ] RAID dataset downloaded and balanced (100K English)
[ ] DeBERTa fine-tuned on cloud GPU (budget ≤ $10)
[ ] text-service: F1 ≥ 0.85 on RAID validation
[ ] SHAP heatmap generating and rendering in UI
[ ] M4 cross-dataset evaluation documented

PHASE 3 – IMAGE
[ ] ResNet-50 + Corvi models downloaded and verified
[ ] image-service: ensemble scoring end-to-end
[ ] GradCAM heatmap overlay in frontend UI
[ ] AUC on CIFAKE + GenImage subset documented

PHASE 4 – AUDIO
[ ] AASIST baseline EER ≤ 1.0% on ASVspoof 2019 LA
[ ] audio-service: async processing via RabbitMQ
[ ] Mel-spectrogram displayed in frontend
[ ] In-the-Wild cross-dataset EER documented

PHASE 5 – SYSTEM
[ ] FusionEngine calibrated, weights saved to config/
[ ] PDF report generating with embedded visuals
[ ] 3 backend E2E integration tests passing
[ ] 4 Playwright frontend tests passing
[ ] P95 text detection < 2s under 10 concurrent users
[ ] Prometheus + Grafana dashboards running

PHASE 6 – SUBMISSION
[ ] OWASP ZAP: 0 HIGH severity findings
[ ] UAT: 3 testers, top-3 issues fixed
[ ] All evaluation metrics computed and tabulated
[ ] README with `make up` setup instructions
[ ] 5-minute demo video recorded and linked
[ ] GitHub Release v1.0.0 tagged
[ ] Docker images pushed to GHCR
```

## 14.8 Resume Value Map

| Technology Used                       | Job Description Signal                                   |
|---------------------------------------|----------------------------------------------------------|
| FastAPI + async Python                | "Python backend, REST APIs, async programming"           |
| SQLAlchemy + PostgreSQL + Alembic     | "Database design, ORM, schema migrations"                |
| RabbitMQ async queues                 | "Message queuing, distributed systems, async processing" |
| Docker Compose + multi-stage builds   | "Containerization, Docker, microservices"                |
| GitHub Actions CI/CD                  | "DevOps, CI/CD pipelines, automated testing"             |
| HuggingFace Transformers (DeBERTa)    | "LLM fine-tuning, NLP, transformer models"               |
| PyTorch (ResNet, AASIST, GradCAM)     | "Deep learning, PyTorch, computer vision, audio ML"      |
| SHAP + GradCAM explainability         | "Explainable AI, XAI, model interpretability"            |
| Next.js 14 TypeScript                 | "Full-stack, React, TypeScript, modern web development"  |
| Prometheus + Grafana                  | "Observability, monitoring, MLOps, SRE"                  |
| Pydantic v2 DTOs                      | "Type safety, API contracts, data validation"            |
| MinIO S3-compatible                   | "Cloud storage, object storage, AWS S3"                  |
| Locust load testing                   | "Performance testing, SLO validation"                    |
| Multi-modal fusion (SHAP calibration) | "ML system design, ensemble methods, calibration"        |

Blueprint Complete — 14 of 14 Sections

Documents:

- \* blueprint\_sections\_1\_to\_4.md
- \* blueprint\_sections\_5\_to\_8.md
- \* blueprint\_sections\_9\_to\_14.md\*